



university of
 groningen

faculty of science
and engineering

Nested Session Types in Turing Complete Calculi

Master's thesis

June 28, 2026

Student: S. Wortelboer

Primary supervisor: Prof. Dr. J.A. Pérez

Secondary supervisor: Dr. D. Frumin

Contents

1	Introduction	3
2	Background	4
2.1	Introduction	4
2.2	Nested Session Types (NSTs)	4
2.2.1	Process Language	5
2.2.2	Process Configurations	6
2.2.3	Example	10
2.2.4	Properties	10
2.3	Turing Completeness and Minsky Machines	13
2.3.1	Reachable instructions	13
2.3.2	Examples	14
3	Turing completeness of the process language	16
3.1	Introduction	16
3.1.1	Limitation of Nested Types	16
3.2	Encoding of Minsky machines	18
3.2.1	Encoding of registers	18
3.2.2	Encoding of instructions	20
3.2.3	Encoding of configurations	23
3.2.4	Examples	24
3.3	Correctness of the encoding	26
3.3.1	Soundness	27
3.3.2	Completeness	28
3.4	Turing completeness	31
4	Counting with nested types	32
4.1	Introduction	32
4.1.1	Complementing nested types with processes	32
4.2	Encoding of registers	35
4.3	Instantiation of state-aware instructions	36
4.3.1	Encoding of state-aware instructions	37
4.3.2	Encoding the instantiation tree	39
4.4	Encoding of the configuration	44
4.4.1	Process Interactions of the Encoding	45
4.4.2	Examples	49
4.4.3	Correctness	51

5	Discussion	54
6	Conclusion	56
	Appendices	58
A	Proof of Theorem 3.1	59
A.1	Register processes	60
A.1.1	Register encoding	60
A.1.2	Register process definitions	60
A.1.3	Constituent register processes	61
A.2	Instruction processes	62
A.2.1	Instruction encoding	62
A.2.2	Instruction process definition	63
A.2.3	Constituent instruction processes	63
A.3	Encoding of configuration	64
B	Proof of Theorem 4.1	65
B.1	Register processes	66
B.1.1	Register encoding	66
B.1.2	Register process definitions	66
B.1.3	Constituent register processes	67
B.2	Instruction processes	68
B.2.1	Instruction encoding	68
B.2.2	Instruction process definitions (Figure 4.2)	69
B.2.3	Constituent instruction processes	70
B.3	State tree processes	72
B.3.1	State tree encoding	72
B.3.2	State tree process definition	73
B.3.3	Constituent state tree processes	73
B.4	Encoding of the configuration	78

Chapter 1

Introduction

Session types [1] provide a logical framework for reasoning about concurrent computation. Through the Curry-Howard correspondence, they establish a link between logic and concurrency, where types are propositions and programs serve as their proof. Combined with linear logic, session types can ensure properties that programs adhere to protocols and ensure deadlock freedom/progress.

While session types introduce guarantees on the behaviour of a program, they also restrict the expressiveness of the underlying program. Hence, the session types that are least restrictive are preferable. One such type system is called nested session types [2]. It combines prenex polymorphism, recursion, polymorphic type construction and nested types and shows that an algorithm for type equivalence is decidable. In the hierarchy of session types, nested types [3] are only superseded in terms of expressiveness by two-counter types for which type equivalence is not decidable. Hence, nested types are the most expressive session types where type equivalence is still decidable.

In this thesis, we establish the role of session types in two-counter/Minsky machines. We present a minimal encoding where the state of registers is captured on the side of processes only, where branching happens on the process level of NST (language of nested session types). We also present an encoding that captures state on the level of session types and processes using one-counters to capture the state of the registers, complemented by nested types to allow progress.

In Chapter 2 we will give the process calculus for NST and Minsky machines and discuss their most important properties. In Chapter 3, a minimal encoding is given and proof of the correctness of the encoding. In Chapter 4 will give the encoding using nested types capturing state in session types and discuss the correctness of the encoding as a consequence of the captured state of registers in the session types. In Chapter 5, the differences between the two encodings are discussed, as well as the relation of nested types to two-counter types with regard to the encoding of Minsky machines. Chapter 6 summarises the conclusions of the results presented in the thesis.

Chapter 2

Background

2.1 Introduction

This chapter gives a brief overview and formal specification of Nested Session Types and Minsky machines, the specific construction used to study Turing completeness.

2.2 Nested Session Types (NSTs)

Session Types are type-based specifications of protocols for concurrent message-passing processes. Building upon the Curry-Howard correspondence between session types and linear logic [4], Nested Session Types [2] extend session types with nested, recursive polymorphic types. This results in an expressive way to define session protocols through the combination of prenex polymorphism and recursion, without imposing restrictions on polymorphic type constructions in combination with prenex polymorphism.

Definition 2.1 (Types). The language of nested session types (NSTs) is defined by:

$A, B, C ::= \oplus \{\textcolor{red}{1}: A_l\}_{l \in L}$	send label $l_i \in L$
$ \&\{\textcolor{red}{1}: A_l\}_{l \in L}$	receive label $l_i \in L$
$ A \otimes B$	send channel of type A
$ A \multimap B$	receive channel of type A
$ \mathbf{1}$	close channel
$ \alpha$	type variable
$ V[\theta]$	type name

Following [4], the type system allows us to express protocols for sending and receiving of labels and channels and to specify the terminated protocol. These constructs are then extended with types for type variables and type definitions, which are comparable to channels and recursive process definitions in processes, respectively. The type name $V[\theta]$ deserves elaboration: it refers to an instantiation of a type definition $V[\bar{\alpha}] \triangleq A$, where $\bar{\alpha}$ refers to the type parameters

contained in A . Type definitions are contained in the set of signatures Σ :

$$\Sigma ::= \cdot \mid V[\bar{\alpha}] = A$$

In the type name $V[\theta]$, θ refers to a substitution of type variables, that is, $\theta = A/\alpha, \dots$. Any substitution that replaces all type parameters is called $\mathcal{V}$ -closing. Substitutions follow the judgment

$$\mathcal{V}' \vdash \theta : \mathcal{V}$$

with $\mathcal{V}$ -closing being defined as $\mathcal{V}' = \cdot$.

Type names can have empty substitutions, $V[]$, such type names are only $\mathcal{V}$ -closing when A has no free names, with type definition $V[] = A$. Moreover, whenever a substitution is written like $V[A, B]$ for a type name $V[\alpha, \beta]$, it constitutes a substitution along the order of the type variables, hence it is equivalent to $V[A/\alpha, B/\beta]$.

In NSTs, any type that is not a type name is structural; for example, $V_1[\theta_1] \otimes V_2[\theta_2]$. Any type that is structural and not a type variable is contractive and hence a valid type.

The meta-theory of NSTs is well developed. One important property is that type equivalence is decidable. Also, an algorithm for type equivalence exists; see [2] for details. Their expressive power has been studied in [3].

2.2.1 Process Language

The language of types is then coupled with a formal process language, which we will call ASP. This language is based on the intuitionistic variant of the Curry-Howard correspondence [4], under an asynchronous communication discipline. Due to its logical foundations, communication in ASP will be shown to be deadlock-free.

Definition 2.2 (Processes). Given a set of channel/names ranged over by $x, y, z, \dots$, the syntax of processes is defined as follows.

$P, Q ::= x.\mathbf{l}_i; P$	send label $\mathbf{l}_i$ over x
$\mathbf{case} \ x \ (\mathbf{l}_i \Rightarrow Q_i)_{i \in L}$	receive a label over x
$\mathbf{send} \ x \ y; P$	send channel y over x
$y \leftarrow \mathbf{recv} \ x; P$	receive a channel over x
$\mathbf{close} \ x$	closed channel x
$\mathbf{wait} \ x; P$	wait for closing of x
$x \leftrightarrow y$	forward x along y
$x \leftarrow f[\theta] \ \bar{y}; P$	instantiate process definition $f[] \in \Sigma$

The operational semantics will be given later on for processes encapsulated into *configurations*, which will involve also messages exchanged between processes (cf. Section 2.2.2).

The typing judgment for processes is of the form

$$\mathcal{V}; \Delta \vdash_{\Sigma} P :: (x : A)$$

where $\mathcal{V}$ is the environment of type variables, Δ is the linear environment of channels (a sequence of assignments $x : A$, no weakening nor contraction), P is the process, x is the intuitionistic channel of type A , and Σ is a signature that contains type and process definitions. Process definitions are of the form $\Delta \vdash f[\bar{\alpha}] = P :: (x : A)$, where f is the name of the process and P its definition, with Δ being the channels used by f . Also, $\bar{\alpha}$ is a sequence of type variables that P , Δ , and A can refer to; they represent prenex polymorphism.

The intuitionistic interpretation of session types ensures that there is no need for the explicit definition of dual types. This is because typing induces an interpretation of types where protocols can either be used or implemented by a process. That is, the judgement $\mathcal{V}; \Delta \vdash P :: (x : A)$ means that P implements a protocol A along the intuitionistic channel x , using channels described in Δ . The intent is that $x : A$ can be used by at most another process implementing the dual of A .

The typing rules for ASP are given in Figure 2.1; they allow the construction of a sequential process. Each process action for sending or receiving labels/channels has two rules: one that exercises action using a channel in the antecedent (i.e., to the left of ‘ $\vdash$ ’) and one that applies to the intuitionistic channel. Sending actions make use of the type equivalence relation to allow channels using structurally different types to be exchanged. The action ‘**close** x ’ for terminating a process is given over the intuitionistic channel, as that channel is tied to the lifecycle of the process.

The last three rules in Figure 2.1 concern channel forwarding and process definitions. The forwarder process $x \leftrightarrow y$ can be typed provided that x and y have structurally different but equivalent types.

The rule for process instantiation is defined with the preconditions of a process definition that will be instantiated, a type equivalence relation similar to the other rules, with the added substitution and finally as a continuation process that makes use of the process instantiated. Process definitions are analogous to the type definitions and allow the formation of nested processes to match the nested types. Process definitions, in the same way as type definitions, need to be structural; the process P_f needs to be typable. Similar to sending and forwarding, type equivalence allows the rewriting of the substituted types in the antecedent, allowing it to match a prefix or external process with equivalent protocol. Finally, the instantiation of $f[]$ matches the type of channel x by substitution on both sides, where the substitution on the type B of x is given as a precondition, while for the process definition, the same substitution is passed as an argument being applied to the same type B .

Whenever type equivalence naturally follows (i.e. equivalence follows from definition or identity/substitution), the abbreviation $;\mathcal{V} \vdash (\Delta) \diamond$ is used, meaning $;\mathcal{V} \vdash \Delta \equiv \Delta$.

2.2.2 Process Configurations

Processes in ASP are not directly cut against each other, but are lifted to process configurations, denoted $\mathcal{S}$. Process configurations handle the asynchronous nature of ASP by adding messages that are an intermediary medium to transfer actions performed between processes.

First, we define *messages*; they can be seen as processes that perform one action.

$$\begin{array}{c}
\frac{(k \in L) \quad \mathcal{V}; \Delta \vdash P :: (x : A_k)}{\mathcal{V}; \Delta \vdash x.\mathbf{k}; P :: (x : \oplus\{\mathbf{1} \Rightarrow A_l\}_{l \in L})} \oplus R \\
\\
\frac{(\forall l \in L) \quad \mathcal{V}; \Delta, x : A_l \vdash Q_l :: (z : C)}{\mathcal{V}; \Delta, x : \oplus\{\mathbf{1} : A_l\}_{l \in L} \vdash \mathbf{case} \ x \ (\mathbf{1} \Rightarrow Q_l)_{l \in L} :: (z : C)} \oplus L \\
\\
\frac{(\forall l \in L) \quad \mathcal{V}; \Delta \vdash P_l :: (x : A_l)}{\mathcal{V}; \Delta \vdash \mathbf{case} \ x \ (\mathbf{1} \Rightarrow Q_l)_{l \in L} :: (x : \&\{\mathbf{1} \Rightarrow A_l\}_{l \in L})} \& R \\
\\
\frac{(k \in L) \quad \mathcal{V}; \Delta \vdash P :: (x : A_k)}{\mathcal{V}; \Delta, x : \&\{\mathbf{1} \Rightarrow A_l\}_{l \in L} \vdash x.\mathbf{k}; Q :: (z : C)} \& L \\
\\
\frac{\cdot; \mathcal{V} \vdash A \equiv A' \quad \mathcal{V}; \Delta \vdash P :: (x : B)}{\mathcal{V}; \Delta, y : A \vdash \mathbf{send} \ x \ y; P :: (x : A' \otimes B)} \otimes R \\
\\
\frac{\mathcal{V}; \Delta, y : A, x : B \vdash Q :: (z : C)}{\mathcal{V}; \Delta, x : A \otimes B \vdash y \leftarrow \mathbf{recv} \ x; Q :: (z : C)} \otimes L \\
\\
\frac{\mathcal{V}; \Delta, y : A \vdash P :: (x : B)}{\mathcal{V}; \Delta \vdash y \leftarrow \mathbf{recv} \ x; P :: (x : A \multimap B)} \multimap R \\
\\
\frac{\cdot; \mathcal{V} \vdash A \equiv A' \quad \mathcal{V}; \Delta, x : B \vdash Q :: (z : C)}{\mathcal{V}; \Delta, x : A' \multimap B, y : A \vdash \mathbf{send} \ x \ y; Q :: (z : C)} \multimap L \\
\\
\frac{}{\mathcal{V}; \cdot \vdash \mathbf{close} \ x :: (x : \mathbf{1})} \mathbf{1} R \quad \frac{\mathcal{V}; \Delta \vdash Q :: (z : C)}{\mathcal{V}; \Delta, x : \mathbf{1} \vdash \mathbf{wait} \ x; Q :: (z : C)} \mathbf{1} L \\
\\
\frac{\cdot; \mathcal{V} \vdash A \equiv A'}{\mathcal{V}; y : A \vdash x \leftrightarrow y :: (x : A')} \text{id} \\
\\
\frac{\overline{y' : B'} \vdash f[\bar{a}] = P_f :: (x' : B) \in \Sigma \quad \cdot; \mathcal{V} \vdash \Delta' \equiv \overline{y : B'[\theta]} \quad \mathcal{V}; \Delta, x : B[\theta] \vdash Q :: (z : C)}{\mathcal{V}; \Delta, \Delta' \vdash x \leftarrow f[\theta] \ \bar{y}; Q :: (z : C)} \text{def}
\end{array}$$

Figure 2.1: Typing rules for processes

Definition 2.3 (Messages).

$$\begin{aligned}
M ::= & x.\mathbf{k}; \ x \leftrightarrow x' \\
& | \ \mathbf{send} \ x \ y; \ x \leftrightarrow x' \\
& | \ \mathbf{close} \ x
\end{aligned}$$

Only certain actions, like sending and closing, actually send messages; actions like receiving and waiting are the dual of such actions. This leads to the following formal definition of process configurations.

Definition 2.4 (Configurations).

$$\mathcal{S} ::= (\mathcal{S}_1, \mathcal{S}_2)^x \mid \text{proc}(x, P) \mid \text{msg}(x, M)$$

Each configuration follows the judgment $\Delta \models_{\Sigma} \mathcal{S} :: (x : A)$ for some signature of process definitions Σ . Note that the definition here deviates slightly from the original paper to allow for tree-shaped process configurations, where originally only linear-shaped configurations were possible (not to be confused with linearity).

Definition 2.5 (Typing rules configuration).

$$\frac{\Delta_1 \models_{\Sigma} \mathcal{S}_1 :: (x : A) \quad x : A, \Delta_2 \models_{\Sigma} \mathcal{S}_2 :: \Delta_3}{\Delta_1, \Delta_2 \models_{\Sigma} (\mathcal{S}_1, \mathcal{S}_2)^x :: (y : B)} \text{ comp}$$

$$\frac{.; \Delta \vdash P :: (x : A)}{\Delta \models_{\Sigma} \text{proc}(x, P) :: (x : A)} \text{ proc} \quad \frac{.; \Delta \vdash M :: (x : A)}{\Delta \models_{\Sigma} \text{msg}(x, M) :: (x : A)} \text{ msg}$$

In the configuration type rules, any message and process can be hoisted to become a singleton configuration, using `proc` and `msg` rules. Any configurations can then be composed, where the left configuration offers a channel to the right process, and any channel not offered by the left configuration is forwarded on the left. As configurations should be executable, it requires that any configuration be $\mathcal{V}$ -closing. At no point is it possible to model a parallel composition.

The configuration type rules allow for linearly chained configurations and tree-like dependencies. As a consequence, it becomes necessary to define a structural congruence to allow for associativity and commutativity and ensure that messages and processes are able to reduce when receiving, even when they are not composed together.

Definition 2.6 (Structural congruence).

$$\begin{aligned} (\mathcal{S}_3, (\mathcal{S}_2, \mathcal{S}_1)^x)^y &\equiv ((\mathcal{S}_3, \mathcal{S}_2)^y, \mathcal{S}_1)^x && \text{if } y \in fn(\mathcal{S}_2) \\ (\mathcal{S}_3, (\mathcal{S}_2, \mathcal{S}_1)^x)^y &\equiv (\mathcal{S}_2, (\mathcal{S}_3, \mathcal{S}_1)^y)^x && \text{if } y \in fn(\mathcal{S}_1) \end{aligned}$$

Essentially, the singleton configurations have a partial ordering based on the relation of consumer and producer. The congruence preserves this partial ordering, and whenever this partial ordering is implied, the parentheses are dropped from the presentation.

With the system of configurations defined, the semantics can be defined.

Definition 2.7 (Reductions). See Figure 2.2.

The reductions follow a pattern of actions that create messages and their dual, which consumes the messages, with the exception of the forward reductions and process definition instantiation. In the forward reductions, we denote the subscript c to denote the inclusion of channel c as a free antecedent channel in either the body M of the message or the continuation process Q .

To facilitate nested types in ASP, a finite set of process definitions Σ which can be instantiated, replacing the channels and type variables. Recall from the type rules, a process P_f can be instantiated with name f by substituting type parameters $\bar{\alpha}$ in the types of A and $\bar{B}$. The instantiation action on $f[]$ results in the spawning of a new process P_f to the right, such that it is composed against the continuation process Q , with the appropriate substitutions dictated by θ , but also the renaming of the intuitionistic channel. This ensures type preservation. As a convention, whenever we omit the continuation Q , it implies $Q = x \leftrightarrow x'$. In the reduction, we can skip the renaming and directly identify it with x .

$$\begin{array}{ll}
\text{proc}(x, x.\mathbf{k}; P) \rightarrow \text{proc}(x', P[x'/x]), \text{msg}(x, x.\mathbf{k}; x \leftrightarrow x') & (\oplus S) \\
\text{msg}(x, x.\mathbf{k}; x \leftrightarrow x'), \text{proc}(y, \mathbf{case} \ x \ (\mathbf{1} \Rightarrow Q_i)) \rightarrow \text{proc}(y, Q_k[x'/x]) & (\oplus C) \\
\\
\text{proc}(d, x.\mathbf{k}; P) \rightarrow \text{msg}(x, x.\mathbf{k}; x \leftrightarrow x'), \text{proc}(d, P[x'/x]) & (\& S) \\
\text{proc}(x, \mathbf{case} \ x \ (\mathbf{1} \Rightarrow Q_i)), \text{msg}(x, x.\mathbf{k}; x \leftrightarrow x') \rightarrow \text{proc}(x', Q_k[x'/x]) & (\& C) \\
\\
\text{proc}(x, \mathbf{send} \ x \ w; P) \rightarrow \text{proc}(x', P[x'/x]), \text{msg}(x, \mathbf{send} \ x \ w; x \leftrightarrow x') & (\otimes S) \\
\text{msg}(x, \mathbf{send} \ x \ w; x \leftrightarrow x'), \text{proc}(y, w' \leftarrow \mathbf{recv} \ x; Q) \rightarrow \text{proc}(y, Q[x'/x, w/w']) & (\otimes C) \\
\\
\text{proc}(y, \mathbf{send} \ x \ w; P) \rightarrow \text{msg}(x, \mathbf{send} \ x \ w; x \leftrightarrow x'), \text{proc}(y, P[x'/x]) & (\multimap S) \\
\text{proc}(x, w' \leftarrow \mathbf{recv} \ x; Q), \text{msg}(x, \mathbf{send} \ x \ w; x \leftrightarrow x') \rightarrow \text{proc}(x', Q[x'/x, w/w']) & (\multimap C) \\
\\
\text{proc}(x, \mathbf{close} \ x) \rightarrow \text{msg}(x, \mathbf{close} \ x) & (\mathbf{1} S) \\
\text{msg}(x, \mathbf{close} \ x), \text{proc}(y, \mathbf{wait} \ x; P) \rightarrow \text{proc}(y, P) & (\mathbf{1} C) \\
\\
\text{msg}(d, M), \text{proc}(c, c \leftrightarrow d) \rightarrow \text{msg}(c, M[c/d]) & (\text{id}^+ C) \\
\text{proc}(c, c \leftrightarrow d), \text{msg}(e, M_c) \rightarrow \text{msg}(e, M_c[d/c]) & (\text{id}^- C) \\
\\
\text{proc}(d, Q), \text{proc}(c, c \leftrightarrow d) \rightarrow \text{proc}(c, Q[c/d]) & (\text{id}^+ S) \\
\text{proc}(c, c \leftrightarrow d), \text{proc}(e, Q_c) \rightarrow \text{proc}(e, Q_c[d/c]) & (\text{id}^- S) \\
\\
\text{proc}(z, x \leftarrow f[\theta] \ \bar{d}; Q) \rightarrow \text{proc}(a, P_f[a/x, \bar{d}/\bar{y}, \theta]), \text{proc}(z, Q[a/x]) & (\text{def} C)
\end{array}$$

Figure 2.2: Reduction semantics for configurations

2.2.3 Example

Before moving to show the properties of ASP, we present an example to understand the semantics.

First, we look at the asynchronous nature in a tree-like configuration. Consider the following process configuration

$$\begin{aligned} \mathcal{S} = & \text{proc}(x, x \leftarrow \text{recv } a; e \leftarrow \text{recv } b; R), \\ & \text{proc}(y, \text{send } y \ c; Q), \\ & \text{proc}(z, \text{send } x \ a; c \leftarrow \text{recv } y; P) \end{aligned}$$

Immediately, we can see that there are two possible reductions. The asynchronous nature allows us to reduce the middle process even though the right process is not ready to receive, wanting to send its own message. Both reductions can happen simultaneously, as the messages can be consumed once a process is ready to receive. We can reduce the process as follows

$$\begin{aligned} \rightarrow^2 & \text{proc}(x, x \leftarrow \text{recv } a; e \leftarrow \text{recv } b; R), \\ & \text{proc}(y', Q[y'/y]), \text{msg}(y, \text{send } y \ c; y \leftrightarrow y'), \\ & \text{msg}(x, \text{send } x' \ a; x \leftrightarrow x'), \text{proc}(z, c \leftarrow \text{recv } y; P[x'/x]) \end{aligned}$$

The new configuration can be further reduced after application of the congruence rules. The message over channel x has a partial order with the left-most process, while the message over channel y has a partial order with the right-most process. This allows the rearranging of the configuration, which allows the application of further reductions

$$\begin{aligned} \equiv & \text{proc}(x, x \leftarrow \text{recv } a; e \leftarrow \text{recv } b; R), \text{msg}(x, \text{send } x' \ a; x \leftrightarrow x') \\ & \text{proc}(y', Q[y'/y]), \\ & \text{msg}(y, \text{send } y \ c; y \leftrightarrow y'), \text{proc}(z, c \leftarrow \text{recv } y; P[x'/x]) \end{aligned}$$

The configuration can then be further reduced by the consumption of the messages

$$\begin{aligned} \rightarrow^2 & \text{proc}(x', e \leftarrow \text{recv } b; R), \\ & \text{proc}(y', Q[y'/y]), \\ & \text{proc}(z, P[y'/y, x'/x]) \end{aligned}$$

The new configuration cannot possibly progress only on Q or P , while the left-most process cannot reduce as there is no message or process composed that allows the process to receive along channel e . If the receiving actions were reversed, the left-most process would not be able to consume the message. Essentially, the next action of a process along some external channel stops it from progressing, while receiving along an internal channel can only be reduced when there is no external channel blocking progress in the dual process.

2.2.4 Properties

The language of ASP is defined in such a way that it ensures type safety (type preservation and progress). We add to this evidence of confluence, which ensures deterministic behaviour with regard to the choice of order of reduction.

Type Preservation

Starting with type preservation, it is the property that after a transformation of a configuration, the antecedent environment and the intuitionistic channel remain unchanged in the judgment.

Theorem 2.1 (Type preservation). *For a well-typed configuration in $\Delta' \models_{\Sigma} S :: \Delta$:*

1. *If $S \equiv S'$, then $\Delta' \models_{\Sigma} S' :: \Delta$;*
2. *If $S \rightarrow S'$, then $\Delta' \models_{\Sigma} S' :: \Delta$.*

Proof. Type preservation in structural congruence follows naturally from the choice in which order to compose the configuration, where channels are always uniquely shared between 2 singleton configurations in a composition. Reordering the configurations does not change the partial order and, as a consequence, won't affect the external channels.

Type preservation in reductions follows from a case analysis on the operational semantics from Definition 2.7. Consider the reduction $\otimes S$ for example

$$\text{proc}(x, \text{send } x \ w; P) \rightarrow \text{proc}(x', P[x'/x]), \text{msg}(x, \text{send } x \ w; x \leftrightarrow x')$$

On the left side the process follows the judgment $\Delta, w : A \models_{\Sigma} \text{proc}(x, \text{send } x \ w; P) :: (x : A \otimes B)$. If we invert the rules proc and $\otimes R$, the judgment for process P follows as $;; \Delta \vdash P :: (x : B)$. We can construct the judgment for the message as $w : B, x' : A \models_{\Sigma} \text{msg}(x, \text{send } x \ w; x \leftrightarrow x') :: (x : A \otimes B)$. Renaming x in P and using the comp rule gives us $\Delta, w : A \models_{\Sigma} \text{proc}(x', P[x'/x]), \text{msg}(x, \text{send } x \ w; x \leftrightarrow x') :: (x : A \otimes B)$ which means that the type signature is preserved. Any reduction rule can be further lifted to an arbitrary S using the comp rule, as it only internalises channels without adding restrictions to the reduction rule and congruence rules to allow any possible ordering of singletons in the configurations that are typable. $\square$

Progress and Poisedness

In order to understand progress, it is necessary to define the property of a poised process, message and configuration.

A poised process is a process that tries to receive a message on some channel or tries to forward a channel. Any message is poised over the channel it is trying to apply its action, for example, $\text{msg}(x, x.\mathbf{k}; x \leftrightarrow x')$ is poised on x while $\text{msg}(x, x'.\mathbf{k}; x \leftrightarrow x')$ is poised on x' . As a singleton in a configuration, these are not able to reduce without some internal process or message composed against the channel over which they are poised.

Hence, a poised configuration is defined as a configuration where all processes in the configuration are poised, and all messages are poised against some external channel. An external channel is the set of free names in the configuration. To elaborate, any process that is poised cannot progress as it is waiting to receive; this can only occur if there is some non-poised process, essentially its dual, to create a message, or the message already exists. As messages are poised against external channels, the messages are not received by one of the poised processes in the configuration, and hence, no reduction can occur.

Theorem 2.2 (Progress). *For a well-typed configuration in $\Delta' \models_{\Sigma} \mathcal{S} :: \Delta$. Either $\mathcal{S}$ is poised, or $\mathcal{S} \rightarrow \mathcal{S}'$.*

Proof. Progress can be proven by induction on the left-to-right partial ordering of $\mathcal{S}$. By the congruence rules, it is possible to split the configuration as $(\text{proc}(x, P), \mathcal{S}_r)$.

For the base case, any singleton configuration. For any message, it is given that it is poised and as both channels are external, the configuration is poised. For progress, it either sends or receives. A process that sends can always reduce, while a process that receives is poised and cannot reduce in the absence of any message. This proves the base case.

For the inductive step, we need to show that either $(\mathcal{S}_s, \mathcal{S}_r)$ is poised or reduces, where $\mathcal{S}_s$ is a singleton. From the inductive hypothesis, we know that either $\mathcal{S}_r \rightarrow \mathcal{S}'_r$ or $\mathcal{S}_r$ is poised. In the case of the former we conclude $\mathcal{S} \rightarrow \mathcal{S}'$ where $\mathcal{S}' = (\text{proc}(x, P), \mathcal{S}'_r)$. In the case of the latter, either the singleton is poised or not. When it is not poised, we are done. However, in the case of a poised singleton, that means we have to consider the three cases:

1. The singleton is poised over an external channel. Hence, $\mathcal{S}$ is poised;
2. The singleton is a message poised over a free name of $\mathcal{S}_r$. By the typing rules, this implies that there is a poised process P that receives the message. Hence, the configuration can reduce;
3. The singleton is a process poised over a free name of $\mathcal{S}_r$. By the typing rules, this implies that there is a poised message M . Hence, the configuration can reduce.

□

Confluence

ASP is concurrent and asynchronous, and as a consequence, multiple lines of reduction may exist at the same time. While the choice of reductions can lead to each state, at no point do reductions affect other reductions, nor are choices made that lead to different outcomes.

Theorem 2.3 (Confluence). *Suppose a configuration $\mathcal{S}$ in ASP. If $\mathcal{S} \rightarrow^* \mathcal{S}_1$ and $\mathcal{S} \rightarrow^* \mathcal{S}_2$, then there exists $\mathcal{S}'$ such that $\mathcal{S}_1 \rightarrow^* \mathcal{S}'$ and $\mathcal{S}_2 \rightarrow^* \mathcal{S}'$.*

The proof lies in the properties of linearity, its preservation and absence of choices or race conditions. The definition of the reduction tell us that for some process that can reduce either $\text{proc}(x, P) \rightarrow \text{proc}(x', P'), \text{msg}(x, M)$ or $\text{proc}(x, P) \rightarrow \text{proc}(x, P'), \text{msg}(y, M)$, or in case of process definition instantiation $\text{proc}(x, P) \rightarrow \text{proc}(x, P'), \text{proc}(y, Q)$. The first two cases only enable the converse where messages are consumed $\text{proc}(x, P), \text{msg}(y, M) \rightarrow \text{proc}(x, P')$ or $\text{proc}(x, P), \text{msg}(x, M) \rightarrow \text{proc}(x', P')$. The latter cases are just the natural progressions similar to that of the sequential reduction of actions on a single process. Hence, if a reduction is possible, it will remain possible regardless of the application of alternative reductions. Moreover, given two reductions, the order of application does not matter, leading to the same result.

2.3 Turing Completeness and Minsky Machines

A Turing machine is a class of automata that is able to implement any computer algorithm. It consists of a finite table of instructions and a strip of paper on which the instructions can manipulate symbols. The question of whether such a machine halts for any algorithm and input is undecidable, or in other words, there is no algorithm that can precisely answer this question with “yes” or “no”.

Turing machines define a measure of the expressiveness of a system. If a model can correctly simulate a Turing machine, then it is called Turing-complete. One such model is the Minsky machine [5] (also known as two-counter machines). Minsky machines are as expressive as Turing machines, and therefore their associated termination problem is undecidable.

A Minsky machine has a finite set of instructions that manipulate two registers holding natural numbers by *incrementing* or *decrementing* them. Whenever an instruction is successful, the machine moves to the next one; otherwise, it jumps to an arbitrary (undefined) instruction. When attempting to decrement a register that is 0, the machine will *jump* to another instruction. A Minsky machine terminates when the current instruction is not defined. A formal definition is given below, following [6].

Definition 2.8 (Minsky machine). A Minsky machine $\mathcal{M}$ is defined as a set of instructions $(1 : I_1), \dots, (n : I_n)$. The instructions are defined as:

$$\begin{aligned} I ::= & \text{INC}(j) && \text{increment on register } j \in \{0, 1\} \\ & | \text{DECJ}(j, k) && \text{decrement or jump on register } j \in \{0, 1\}, k \in [1..n] \end{aligned}$$

The state of a Minsky machine $\mathcal{M}$ is defined as the tuple $(i, m_0, m_1)^\mathcal{M}$ where $i, m_0, m_1 \in \mathbb{N}$. The tuple refers to the current instruction i , the register 0 with value m_0 and the register 1 with value m_1 .

The semantics of the machine is defined as:

$$\begin{aligned} & \frac{i : \text{INC}(j) \quad m'_j = m_j + 1 \quad m'_{1-j} = m_{1-j}}{(i, m_0, m_1)^\mathcal{M} \rightarrow (i + 1, m'_0, m'_1)^\mathcal{M}} \text{Incr} \\ & \frac{i : \text{DECJ}(j, k) \quad m_j > 0 \quad m'_j = m_j - 1 \quad m'_{1-j} = m_{1-j}}{(i, m_0, m_1)^\mathcal{M} \rightarrow (i + 1, m'_0, m'_1)^\mathcal{M}} \text{Decr} \\ & \frac{i : \text{DECJ}(j, k) \quad m_j = 0}{(i, m_0, m_1)^\mathcal{M} \rightarrow (k, m_0, m_1)^\mathcal{M}} \text{Jump} \end{aligned}$$

We will use (i, m_0, m_1) when $\mathcal{M}$ is implied. Furthermore, whenever $i > n$ then $(i, m_0, m_1) \nrightarrow$.

2.3.1 Reachable instructions

For any Minsky machine, the instructions tell us what the next state of an instruction can be, while the register complement instructions in choosing the exact instruction. Considering this, it is possible to view the instructions and their continuations as a non-deterministic finite state machine.

Definition 2.9. Instruction i in Minsky machine $\mathcal{M}$ is reachable when:

$$\exists i', m'_0, m'_1, m_0, m_1. (i', m'_0, m'_1) \rightarrow (i, m_0, m_1)$$

The set $L_{\mathcal{M}}$ contains all reachable i for some Minsky machine $\mathcal{M}$.

Lemma 2.1. *The set of reachable instructions $L_{\mathcal{M}}$ is finite.*

Proof. An instruction i is reachable for some $\mathcal{M}$ when there is a configuration that reduces to the configuration with i . A reduction is only possible when $i' \leq n$ and each instruction has at most 2 possible continuation instructions (i.e. $i' + 1$ and k), which are by definition reachable and in the set $L_{\mathcal{M}}$. Hence as there are finite instructions and for each reducible instruction there are finite reachable continuation instructions, the set $L_{\mathcal{M}}$ is finite for some $\mathcal{M}$. $\square$

The lemma tells us that, for any running Minsky machine, the instruction state of any configuration has an upper bound and no instruction can reduce to an instruction not in $L_{\mathcal{M}}$.

2.3.2 Examples

To clarify the semantics of Minsky machines, we consider a small example to illustrate this, which will also serve as an example for the encodings given later.

First, we define the machine $\mathcal{M}$ as

$$1 : \text{INC}(1), 2 : \text{INC}(1), 3 : \text{INC}(0), 4 : \text{DECJ}(1, 3), 5 : \text{DECJ}(1, 1)$$

The set of reachable instructions of this machine is $L = \{1, 2, 3, 4, 5, 6\}$. From any instruction, it is always possible to go to the next instruction, hence $2, 3, 4, 5, 6 \in L$. From the decrement instructions, it is also possible to jump to $1, 3 \in L$.

Essentially, this machine consists of 2 potential loops. The first cycle is from instruction 3 to 4, and the second cycle is from 1 to 5. We call it a cycle as the `decj` can jump to an instruction before it, allowing the machine to potentially return to the instruction. The machine has only one potential way of terminating when starting with an instruction where $i \leq n$, which is when it decrements the second register and moves to the out-of-bound instruction 6. The machine can be used to show both terminating and non-terminating behaviour.

Termination

For the terminating case, we consider the starting state $(3, 0, 1)$, which uses the second cycle before terminating. The machine can first reduce as follows

$$(3, 0, 1) \rightarrow (4, 1, 1) \rightarrow (5, 1, 0) \rightarrow (1, 1, 0)$$

The state increments the first register, after it decrements the second register to 0, which makes it jump to instruction 1. This puts the configuration at the beginning of the second loop. The configuration can further reduce

$$\rightarrow (2, 1, 1) \rightarrow (3, 1, 2)$$

The configuration is reduced by incrementing the second register twice. At this point, we are back at the same instruction 3; however, the state of the

second register is now greater than what we started with. The state of the first register does not matter to the behaviour as the `decj` instruction only applies to the second register. Continuing the reduction

$$\rightarrow (4, 2, 2) \rightarrow (5, 2, 1) \rightarrow (6, 2, 0) \nrightarrow$$

The difference with the initial set of reductions is that the second register can be decremented twice, whereas before the branching, reaching instruction 5 caused it to jump. After the last decrement, the configuration is out of bounds as instruction 6 is not defined, hence the configuration can not reduce further, and the machine terminates.

$$(3, 0, 1) \rightarrow^8 (6, 2, 0) \nrightarrow$$

Non-Termination

For the non-terminating case, we consider the starting state $(3, 0, 0)$, which will use the first cycle. The machine first reduces as follows

$$(3, 0, 0) \rightarrow (4, 1, 0) \rightarrow (3, 1, 0)$$

The configuration reduces by incrementing the first register, and then jumping back to 3 as the second register is 0. From here, the cycle repeats as the only thing that happened is that the first register is incremented, while the branching happened on the second register, which remained unchanged. Hence, we can state

$$\forall n \in \mathbb{N}. (3, n, 0) \rightarrow^2 (3, n + 1, 0)$$

This implies that the machine never terminates, and the first register continues to increase.

$$(3, 0, 0) \rightarrow^{2m} (3, m, 0) \rightarrow$$

Chapter 3

Turing completeness of the process language

3.1 Introduction

In this chapter, we present an encoding of Minsky machines in the formal language of Nested Session Types (ASP) and show its correctness. Hence, this correct encoding tells us that ASP is Turing-complete.

3.1.1 Limitation of Nested Types

Given that the main contribution of [2] is nested session types, it is therefore logical to exploit the nested type system in the encoding. The session types could be used to capture the semantics of the Minsky machine, and the correctness of the encoding would follow naturally. However, modelling the behaviour of two-counter machines using a nested type is not possible as such types cannot express two-counter types, as established by Gay et al. [3].

We can illustrate the problem by looking at the branching in some Minsky machine, where $i : \text{DECJ}(j, k)$ can decrement or jump depending on the value in register j . We can capture the states of registers in nested types as type parameters, resulting in a counter type like $C[\alpha, \beta]$ with the two parameters referring to the current values of the two registers. Here, our first observation

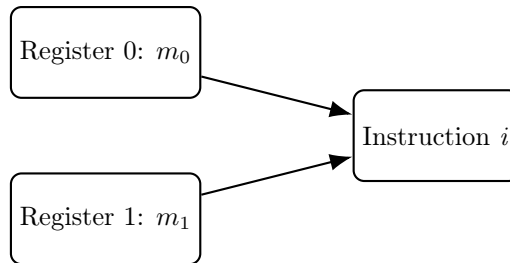


Figure 3.1: Structure of the process configuration of the encoded Minsky machine

is that we need to differentiate between

- states of zero, which offer a jump for a register, and
- states greater than zero, which offer a decrement.

Hence, the following types follow $C^{0,0}[]$, $C^{s,0}[\alpha]$, $C^{0,s}[\beta]$ and $C^{s,s}[\alpha, \beta]$ for the different states, where the superscripts respectively denote the states of zero (denoted 0) and greater than zero (denoted s). This way, depending on the state, each session type offers choices like increment, decrement or jump for each register; for example, for a register with state 0 only increment and jump are offered, while with a state s both increment and decrement are offered.

The issue that arises is that in nested types, nothing can be stated about the type parameter. For example, when the first register of type $C^{s,0}[\alpha]$ is decremented, the concrete instantiation of α will determine whether the subsequent nested type is $C^{0,0}[]$ or $C^{s,0}[\alpha']$. Furthermore, it is not possible to construct the predecessor α' of α . Logically, a state between 0 and s (say, 1) could capture this transition, but this would mean that we need to capture the transition from s to 1 as well. As nested types are constrained by a finite set of type definitions and finite type parameters in a definition, there is no possibility of capturing the unbounded states of Minsky machines for all transitions.

The above means that, under the design for registers given before, nested types by themselves cannot correctly represent the state distinction needed to implement decrement and jump. An alternative design would use the type parameters as push-down states for decrementing. Here, the interpretation would be to use α and β to be the subsequent state after decrementing a register, for example:

$$C^{s,s}[\alpha, \beta] = \&\{\dots \\ \text{decr}_0 : \alpha, \\ \text{decr}_1 : \beta\}$$

In addition to the fact that α and β capture the states for the registers indirectly and independently, the issue now comes down to the types after incrementing. Essentially, the counter type refers to a tree of possible ways to decrement the registers. Consider a state of 1 and 0 for the registers which would correspond to the type $C^{s,0}[C^{0,0}[]]$. The state of incrementing the second register would then constitute the following type $C^{s,s}[C^{s,0}[C^{0,0}[]], C^{0,s}[C^{0,0}[]]]$. When defining the type $C^{s,0}[\alpha]$ for the incrementing case, α can be forwarded to the first parameter of $C^{s,s}[\cdot]$; however, the second parameter depends on the actual instantiation of α to reconstruct it. We therefore reach the same conclusion as before.

Given that nested types cannot capture two-counter types, we are only able to show the Turing completeness from the perspective of the process language. This chapter will focus on showing this approach to Turing completeness, using only recursion of the nested types (i.e. using $V[]$ without type parameters). In the next chapter, an alternative encoding that uses session types to count, supported by process synchronisations, will be developed. This alternative encoding is more complex than the one given next, as additional process structures are required to enable the counting in session types due to the shift in how states of registers are shared with the instruction.

3.2 Encoding of Minsky machines

Programs in ASP are constrained by session types, hence we introduce a counter type for interacting with registers first.

Definition 3.1 (Counter protocol). The definition of the session type for a register is given as:

$$C[] \triangleq \&\{\text{inc}: C[], \text{decj}: \oplus\{\text{jump}: C[], \text{next}: C[]\}\} \quad (3.1)$$

The recursive protocol is simple: it describes an infinitely interactable register process that offers a choice to either increment or decrement-jump, respectively **inc** and **decj**. In the case of a decrement, it then communicates whether the current state of the register is zero or non-zero through the labels **jump** and **next**, respectively, which will correspond with the next instruction that needs to be executed for each case. In all cases, it ends with the process continuing to be a register.

3.2.1 Encoding of registers

The protocol $C[]$ can be used to define the encoding of a register.

Definition 3.2 (Encoding register). Given some $m \in \mathbb{N}$, the encoding of m into a configuration $\mathcal{S}$ in ASP is given as¹:

$$\llbracket m \rrbracket_r = \begin{cases} \text{proc}(r, R^0) & \text{if } m = 0 \\ \llbracket m - 1 \rrbracket_p, \text{proc}(r, R^s) & \text{if } m > 0 \end{cases} \quad (3.2)$$

The set of process definitions $\Sigma_{\mathcal{M}}$ is defined as:

$$\begin{aligned} \cdot \vdash z[] &= R^0 :: (r : C[]) \in \Sigma_{\mathcal{M}} & (\text{zero}) \\ p : C[] \vdash s[] &= R^s :: (r : C[]) \in \Sigma_{\mathcal{M}} & (\text{incr}) \end{aligned}$$

with the following definitions for R^0 and R^s :

$$\begin{aligned} R^0 &= \text{case } r \text{ (inc} \Rightarrow c \leftarrow z[]; r \leftarrow s[] \text{ c, decj} \Rightarrow r.\text{jump}; r \leftarrow z[]) \\ R^s &= \text{case } r \text{ (inc} \Rightarrow c \leftarrow s[] \text{ p; } r \leftarrow s[] \text{ c, decj} \Rightarrow r.\text{next}; r \leftrightarrow p) \end{aligned}$$

Note that an instantiation of $z[]$ over channel r would correspond to $\llbracket 0 \rrbracket_r$. Similarly, for $s[]$ would instantiate to $\llbracket m + 1 \rrbracket_r$ when provided with p that corresponds to the process $\llbracket m \rrbracket_p$.

The encoding for the case $m = 0$ encodes a poised process that waits for a label. In case of increment, the encoding replicates its own state of zero by instantiating $z[]$ and then continues to be the increment through the instantiation of $s[]$, which corresponds to the value of 1 for the register after. In case of decrement, it sends the label **jump** to indicate that it is not possible to decrement and then continues to be zero by instantiating $z[]$.

When $m > 0$, the encoding works in the same way. However, when incrementing the definition $s[]$ is used, which corresponds to the state m , which is

¹The subscript r refers to the internal renaming of channel r

then passed to the next increment process, such that we get to $m + 1$. When decrementing, instead of **jump**, **next** is sent: this indicates a successful decrement. Afterwards, the encoding forwards the previous value of $m - 1$ to the instruction process.

We show that the encoding is typable in ASP.

Lemma 3.1. *Given any $\llbracket m \rrbracket_r$, from Definition 3.2, then*

$$\cdot \models_{\Sigma, \mathcal{M}} \llbracket m \rrbracket_r :: (r : C[])$$

Proof. Appendix A.1 gives type derivations for the processes of the encoding. $\square$

The semantics can be formally presented as a Lemma.

Lemma 3.2. *For some process in ASP $\llbracket m \rrbracket_r$ (Definition 3.2), it is poised and can only progress under one of the following conditions:*

1. $\llbracket m \rrbracket_r, \text{msg}(r', r.\text{inc}; r' \leftrightarrow r) \rightarrow^3 \llbracket m + 1 \rrbracket_{r'}$
2. $\llbracket m \rrbracket_r, \text{msg}(r', r.\text{decj}; r' \leftrightarrow r) \rightarrow^3 \llbracket m - 1 \rrbracket_{r''}, \text{msg}(r', r'.\text{next}; r' \leftrightarrow r'')$, if $m > 0$
3. $\llbracket 0 \rrbracket_r, \text{msg}(r', r.\text{decj}; r' \leftrightarrow r) \rightarrow^3 \llbracket 0 \rrbracket_{r''}, \text{msg}(r', r'.\text{jump}; r' \leftrightarrow r'')$

Proof. Based on Definition 3.2 for $\llbracket m \rrbracket_r$, there are two possibilities to consider (states $m > 0$ and $m = 0$):

Case 1 ($m > 0$). By definition $\llbracket m \rrbracket_r = \text{proc}(r, R^s), \llbracket m - 1 \rrbracket_p$. The process configuration is poised and can only progress once it receives either a message with the label **inc** or **decj** over the external channel r .

For the case using label **inc**:

$$\begin{aligned} & \llbracket m - 1 \rrbracket_p, \text{proc}(r, R^s), \text{msg}(r', r.\text{inc}; r' \leftrightarrow r) \\ & \rightarrow \llbracket m - 1 \rrbracket_p, \text{proc}(r', c \leftarrow s[] p; r' \leftarrow s[] c) \\ & \rightarrow \llbracket m - 1 \rrbracket_p, \text{proc}(c, R^s[c/p]), \text{proc}(r', r' \leftarrow s[] c) \\ & = \llbracket m \rrbracket_c, \text{proc}(r', r' \leftarrow s[] c;) \\ & \rightarrow \llbracket m \rrbracket_c, \text{proc}(r', R^s[c/p]) \\ & = \llbracket m + 1 \rrbracket_{r'} \end{aligned}$$

For the case **decj**:

$$\begin{aligned} & \llbracket m - 1 \rrbracket_p, \text{proc}(r, R^s), \text{msg}(r', r.\text{decj}; r' \leftrightarrow r), \\ & \rightarrow \llbracket m - 1 \rrbracket_p, \text{proc}(r', r'.\text{next}; r' \leftrightarrow p), \\ & \rightarrow \llbracket m - 1 \rrbracket_p, \text{proc}(r'', r'' \leftrightarrow p), \text{msg}(r', r'.\text{next}; r' \leftrightarrow r'') \\ & \rightarrow \llbracket m - 1 \rrbracket_{r''}, \text{msg}(r', r'.\text{next}; r' \leftrightarrow r''), \end{aligned}$$

Case 2 ($m = 0$). By definition $\llbracket 0 \rrbracket_r = \text{proc}(r, R^0)$. Like with the case $m > 0$, the process is poised and is awaiting the label **inc** or **decj**.

For the case using label **inc**:

$$\text{proc}(r, R^0), \text{msg}(r', r.\text{inc}; r' \leftrightarrow r)$$

$$\begin{aligned}
&\rightarrow \text{proc}(r', c \leftarrow z[]; r' \leftarrow s[] \ c) \\
&\rightarrow \text{proc}(c, R^0[c/r]), \text{proc}(r', r' \leftarrow s[] \ c) \\
&= \llbracket 0 \rrbracket_c, \text{proc}(r', r' \leftarrow s[] \ c) \\
&\rightarrow \llbracket 0 \rrbracket_c, \text{proc}(r', R^s[r'/r, c/p]) \\
&= \llbracket 1 \rrbracket_{r'}
\end{aligned}$$

For the case **decj**:

$$\begin{aligned}
&\text{proc}(r, R^0), \text{msg}(r', r.\text{decj}; r' \leftrightarrow r) \\
&\rightarrow \text{proc}(r', r'.\text{jump}; r' \leftarrow z[]) \\
&\rightarrow \text{proc}(r'', r'' \leftarrow z[]), \text{msg}(r', r'.\text{jump}; r' \leftrightarrow r'') \\
&\rightarrow \text{proc}(r'', R^0[r''/r]), \text{msg}(r', r'.\text{jump}; r' \leftrightarrow r'') \\
&= \llbracket 0 \rrbracket_{r''}, \text{msg}(r', r'.\text{jump}; r' \leftrightarrow r'')
\end{aligned}$$

□

3.2.2 Encoding of instructions

Having defined the encoding of registers, we now present the corresponding encoding of the instructions that define the machine's transitions. The encoded instruction is a recursive process that instantiates the next instruction process after manipulating a register. In ASP, we require the use of process definitions to achieve this. The set of process definitions for any program is finite in ASP. From Lemma 2.1, it is given that whenever a Minsky machine reduces, it can only move to a finite set of reachable instructions. This ensures that we only have to define process definitions for a finite number of instructions $i \in L_{\mathcal{M}}$.

Definition 3.3 (Encoding instructions). Given some $i \in \mathbb{Z}^+$ and Minsky machine $\mathcal{M}$, the encoding of i into a configuration $\mathcal{S}$ in ASP is given as²:

$$\llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} = \text{proc}(x, P_i) \quad (3.3)$$

The set of process definitions $\Sigma_{\mathcal{M}}$ is defined as:

$$r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash p_{i \in L_{\mathcal{M}}}[] = P_i :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \quad (3.4)$$

with the following definitions for P_i :

$$P_{i \in \mathbb{Z}^+} = \begin{cases} r_j.\text{inc}; x \leftarrow p_{i+1}[] \ r_0 \ r_1 \ y & \text{if } i : \text{INC}(j) \text{ with } j \in \{0, 1\} \\ r_j.\text{decj}; \text{case } r_j \ (& \\ \quad \text{jump} \Rightarrow x \leftarrow p_k[] \ r_0 \ r_1 \ y, & \\ \quad \text{next} \Rightarrow x \leftarrow p_{i+1}[] \ r_0 \ r_1 \ y) & \text{if } i : \text{DECJ}(j, k) \text{ with } j \in \{0, 1\} \\ \text{wait } y; Q_i & \text{if } i > n \end{cases} \quad (3.5)$$

²The subscripts r_0, r_1 refer to the internal renaming of the channels for registers 0 and 1, in that order. The channels x and y never reduce, and as a consequence, no renaming is necessary.

There are three scenarios to consider:

- The encoding of an increment instruction i refers to a process that sends a label **inc** to the register j on channel r_j , where $j \in \{0, 1\}$ following Definition 2.8. Afterwards, the process continues to be $\llbracket i + 1 \rrbracket_{r_0, r_1}^{\mathcal{M}}$ by instantiating $p_{i+1}[]$;
- Similarly, the encoding of a decrement instruction i sends a label **decj**. However, before instantiating its continuation ($\llbracket i + 1 \rrbracket_{r_0, r_1}^{\mathcal{M}}$ or $\llbracket k \rrbracket_{r_0, r_1}^{\mathcal{M}}$), the register channel tells the instruction which to instantiate through the receiving of a label of **next** or **jump**, respectively;
- Finally, there is the case of an out-of-bound instruction. Here, the encoding waits for channel y to close and then executes an arbitrary process Q_i . This process is poised, and this state can be maintained by making y external, preventing progress in such a scenario, simulating termination. This construction is necessary to ensure typeability in ASP, while also preserving the semantics of termination in the machine.³

We show that the encoding is typable in ASP.

Lemma 3.3. *Given any $\llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}}$ then, from Definition 3.3, then*

$$r_0 : C[], r_1 : C[], y : \mathbf{1} \models_{\Sigma, \mathcal{M}} \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} :: (x : \mathbf{1})$$

Proof. Appendix A.2 gives type derivations for the processes of the encoding. $\square$

The semantics can be formally presented as a lemma.

Lemma 3.4. *For some process in ASP $\llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}}$ (Definition 3.3), one of the following reductions holds:*

1. $\llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} \rightarrow^2 \text{msg}(r'_j, r_j.\text{inc}; r'_j \leftrightarrow r_j), \llbracket i + 1 \rrbracket_{r'_0, r'_1}^{\mathcal{M}}$ where $r_{1-j} = r'_{1-j}$, when $i : \text{INC}(j)$ in $\mathcal{M}$.
2. $\llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} \rightarrow \text{msg}(r'_j, r_j.\text{decj}; r'_j \leftrightarrow r_j), \text{proc}(x, Q)$, when $i : \text{DECJ}(j, k)$ in $\mathcal{M}$, where the singleton configuration containing Q is poised and can only progress under one of the following conditions:
 - (a) $\text{msg}(r'_j, r'_j.\text{jump}; r'_j \leftrightarrow r''_j), \text{proc}(x, Q) \rightarrow^2 \llbracket k \rrbracket_{r'_0, r'_1}^{\mathcal{M}}$, when interacting with a register for which $m_j = 0$
 - (b) $\text{msg}(r'_j, r'_j.\text{next}; r'_j \leftrightarrow r''_j), \text{proc}(x, Q) \rightarrow^2 \llbracket i + 1 \rrbracket_{r'_0, r'_1}^{\mathcal{M}}$, when interacting with a register for which $m_j > 0$.

where $r_{1-j} = r'_{1-j}$

3. $\text{msg}(y, \text{close } y), \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} \rightarrow \text{proc}(x, Q_i)$ where Q_i , when $i > n$, is some residual process.

³The process Q_i can be arbitrary, provided it is typable in ASP and is unique for each i . For example, Q_i can be defined as a process that does i increments, followed by a decrement, followed by the construction of a shared process definition for all Q_i . As the process is never reached (due to the poised wait action prefixing it), it is not necessary to consider the exact implementation.

Proof. The instruction i in the machine $\mathcal{M}$ can either be defined as $i : \text{INC}(j)$, $i : \text{DECJ}(j, k)$ or be not defined for which $i > n$.

Case 1 (Increment). By definition:

$$\llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} = \text{proc}(x, r_j.\text{inc}; x \leftarrow p_{i+1} \parallel r_0 \ r_1 \ y)$$

where $j \in \{0, 1\}$. There is only one line of reductions available:

$$\begin{aligned} & \text{proc}(x, r_j.\text{inc}; x \leftarrow p_{i+1} \parallel r_0 \ r_1) \\ & \rightarrow \text{msg}(r'_j, r_j.\text{inc}; r'_j \leftrightarrow r_j), \text{proc}(x, x \leftarrow p_{i+1} \parallel r'_0 \ r'_1) \\ & \rightarrow \text{msg}(r'_j, r_j.\text{inc}; r'_j \leftrightarrow r_j), \text{proc}(x, P_{r'_0, r'_1}^{i+1}) \end{aligned}$$

with $r_{1-j} = r'_{1-j}$. Hence, by definition of the encoding:

$$\llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} \rightarrow^2 \text{msg}(r'_j, r_j.\text{inc}; r'_j \leftrightarrow r_j), \llbracket i+1 \rrbracket_{r'_0, r'_1}^{\mathcal{M}}$$

Case 2 (Decrement). By definition:

$$\begin{aligned} \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} = & \text{proc}(x, r_j.\text{decj}; \text{case } r_j \ (\text{jump} \Rightarrow x \leftarrow p_k \parallel r_0 \ r_1 \ y, \\ & \text{next} \Rightarrow x \leftarrow p_{i+1} \parallel r_0 \ r_1 \ y)) \end{aligned}$$

where $j \in \{0, 1\}$. There is only one line of reduction available, leading to a poised state.

$$\begin{aligned} & \text{proc}(x, r_j.\text{decj}; \text{case } r_j \ (\text{jump} \Rightarrow x \leftarrow p_k \parallel r_0 \ r_1, \text{next} \Rightarrow x \leftarrow p_{i+1} \parallel r_0 \ r_1)) \\ & \rightarrow \text{msg}(r'_j, r_j.\text{decj}; r'_j \leftrightarrow r_j), \\ & \text{proc}(x, \text{case } r'_j \ (\text{jump} \Rightarrow x \leftarrow p_k \parallel r'_0 \ r'_1, \text{next} \Rightarrow x \leftarrow p_{i+1} \parallel r'_0 \ r'_1)) \end{aligned}$$

with $r_{1-j} = r'_{1-j}$. The process can progress when the message with label **decj** is consumed, and either a message with label **jump** or **next** exists. The messages can not mutually exist, or otherwise, it would violate linearity and typeability.

For case using label **jump**:

$$\begin{aligned} & \text{msg}(r'_j, r'_j.\text{jump}; r'_j \leftrightarrow r''_j), \\ & \text{proc}(x, \text{case } r'_j \ (\text{jump} \Rightarrow x \leftarrow p_k \parallel r'_0 \ r'_1, \text{next} \Rightarrow x \leftarrow p_{i+1} \parallel r'_0 \ r'_1)) \\ & \rightarrow \text{proc}(x, x \leftarrow p_k \parallel r''_0 \ r''_1) \\ & \rightarrow \text{proc}(x, P_{r''_0, r''_1}^k) \end{aligned}$$

Similarly for **next**:

$$\begin{aligned} & \text{msg}(r'_j, r'_j.\text{next}; r'_j \leftrightarrow r''_j), \\ & \text{proc}(x, \text{case } r'_j \ (\text{jump} \Rightarrow x \leftarrow p_k \parallel r'_0 \ r'_1, \text{next} \Rightarrow x \leftarrow p_{i+1} \parallel r'_0 \ r'_1)) \\ & \rightarrow \text{proc}(x, x \leftarrow p_{i+1} \parallel r''_0 \ r''_1) \\ & \rightarrow \text{proc}(x, P_{r''_0, r''_1}^{i+1}) \end{aligned}$$

Hence, by definition, one of the following holds:

1. $\text{msg}(r'_j, r'_j.\text{jump}; r'_j \leftrightarrow r''_j), \text{proc}(x, Q) \rightarrow^2 \llbracket k \rrbracket_{r''_0, r''_1}^{\mathcal{M}}$

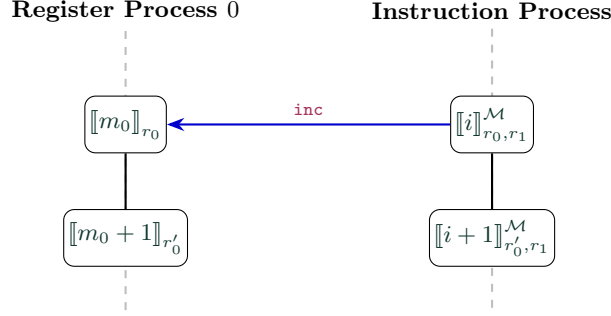


Figure 3.2: Flow diagram of an instruction $i : \text{INC}(0)$ on register 0 with some value m_0 .

$$2. \text{msg}(r'_j, r'_j.\text{next}; r'_j \leftrightarrow r''_j), \text{proc}(x, Q) \rightarrow^2 [i + 1]_{r'_0, r'_1}^{\mathcal{M}}$$

Case 3 (Out-of-bounds). By definition $[i]_{r_0, r_1}^{\mathcal{M}} = \text{proc}(x, \text{wait } y; Q_i)$. The process is poised and can only progress once it receives a close message along y , hence:

$$\text{proc}(x, \text{wait } y; Q_i), \text{msg}(y, \text{close } y) \rightarrow Q_i$$

□

3.2.3 Encoding of configurations

The encodings of instructions and registers can be put together to form the encoding of a configuration of a Minsky machine.

Definition 3.4 (Encoding). For any Minsky machine configuration $(i, m_0, m_1)^{\mathcal{M}}$, a mapping is defined to an ASP process configuration.

$$[(\cdot)]^{\mathcal{M}} : \mathbb{Z}^+ \times \mathbb{N} \times \mathbb{N} \mapsto \mathcal{S} \quad (3.6)$$

Using Definition 3.2 and 3.3 the concrete encoding is defined as:

$$[(i, m_0, m_1)]^{\mathcal{M}} = [m_0]_{r_0}, [m_1]_{r_1}, [i]_{r_0, r_1}^{\mathcal{M}} \quad (3.7)$$

Before proving the correctness of the semantics, we show that the encoding is typable in ASP.

Theorem 3.1 (Typable). *Given any $(i, m_0, m_1)^{\mathcal{M}}$, from Definition 3.4, then*

$$y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} [(i, m_0, m_1)]^{\mathcal{M}} :: (x : \mathbf{1})$$

Proof. Appendix A gives type derivations for the processes of the encoding. □

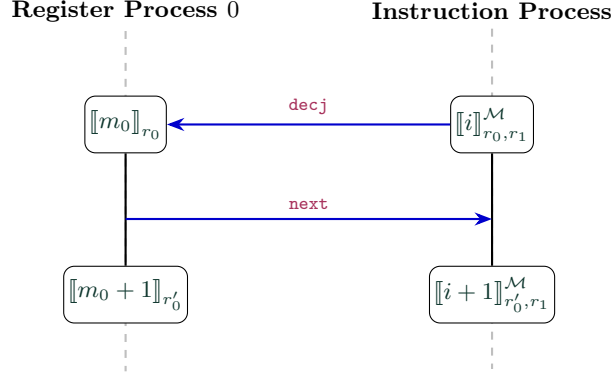


Figure 3.3: Flow diagram of an instruction $i : \text{DECJ}(0, k)$ on register 0 with value $m_0 > 0$.

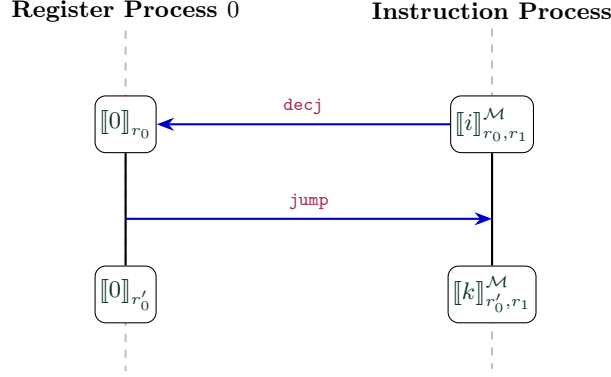


Figure 3.4: Flow diagram of an instruction $i : \text{DECJ}(0, k)$ on register 0 with value 0.

3.2.4 Examples

The semantics of the encoding are illustrated in Figures 3.3, 3.4 and 4.7 as sequence diagrams. In each diagram, a poised register process, either an instantiation of R^0 or R^s , waits for a message from the instruction process i , an instantiation of P_i . In case of an increment, both processes are able to reduce after sending the message. The register process becomes an instance of R^s , and the instruction process becomes P_{i+1} . The case of a decrement is similar to the incrementing case, but the instruction process has to wait for the register to produce a message informing it which branch to pick. The state for the register process began as an instance of R^s and becomes an instance of R^0 or R^s , depending on the value of m , which started out greater than 0 for decrementing. The case for jumping is similar to that of decrementing, with the modified precondition that $m = 0$ and thus always being an instance of R^0 and postcondition that the next instruction is k and an instance of P_k .

We can further verify the behaviour through the analysis of the example

machine from Section 2.2.3. Recall the definition of the machine $\mathcal{M}$ given as

$$1 : \text{INC}(1), 2 : \text{INC}(1), 3 : \text{INC}(0), 4 : \text{DECJ}(1, 3), 5 : \text{DECJ}(1, 1)$$

Termination

The first example starts at state $(3, 0, 1)$ and demonstrates termination and different ways of branching of the machine. The encoding is given as

$$\llbracket (3, 0, 1) \rrbracket^{\mathcal{M}} = \llbracket 0 \rrbracket_{r_0}, \llbracket 1 \rrbracket_1, \llbracket 3 \rrbracket_{r_0, r_1}^{\mathcal{M}}$$

Each component in the encoding corresponds to a process:

1. $\llbracket 0 \rrbracket_{r_0}$ is an instance of R^0 ;
2. $\llbracket 1 \rrbracket_{r_1}$ is an instance of R^1 that depends on $\llbracket 0 \rrbracket_p$ which is an instance of R^0 ;
3. $\llbracket 3 \rrbracket_{r_0, r_1}^{\mathcal{M}}$ is an instance of P_3 that depends on the processes listed above and the external channel y .

The instance of the instruction process is the only non-poised process in the configuration and reduces to $\llbracket 4 \rrbracket_{r_0, r_1}^{\mathcal{M}}$ (Lemma 3.4), enabling the reduction of the first register and the further reduction of the instruction process itself. Reducing the first register process allows the configuration to match the next state $\llbracket (4, 1, 1) \rrbracket^{\mathcal{M}} < r'_0, r_1 >$. However, choosing the reduction of the instruction process skips this state and instead makes progress towards the next state $\llbracket (5, 1, 0) \rrbracket^{\mathcal{M}}$. In ASP, we are free to choose any order of reduction (Theorem 2.3) and, at any point, the configuration can converge to a state that matches the encoding. Choosing the reduction of the instruction process allows the demonstration of the dependency it has on the register process it tries to decrement. The instruction process will get stuck at some poised process Q awaiting a message on which step to take next from the second register process. The reduction of the instruction now allows the reduction of the second register as well. Once the second register reduces to $\llbracket 0 \rrbracket_{r'_1}$ (Lemma 3.2), the instruction receives the message with label next, which indicates a successful decrement, reducing to $\llbracket 5 \rrbracket_{r'_0, r'_1}^{\mathcal{M}}$.

Further reductions on the instruction process let it jump to instruction 1, followed by two increments on the second register and one increment on the first register. This brings the instruction process to $\llbracket 4 \rrbracket_{r''_0, r''_1}^{\mathcal{M}}$. Reducing the instruction once more, and it will get stuck on an instance of Q , hence the second register needs to reduce next. In this case, the register needs to reduce by first two increment messages followed by the decrement message generated by instruction 4. The order of the consumption of the message by the process is cemented by the renaming of the channels. It reduces to $\llbracket 1 \rrbracket_{r'''_1}$ which allows the instruction to move to $\llbracket 5 \rrbracket_{r''_0, r'''_1}^{\mathcal{M}}$. Then again, the instruction reduces, followed by the reduction of the second register, allowing it to move to its final poised state $\llbracket 6 \rrbracket_{r''_0, r''_1}^{\mathcal{M}}$ as it tries to receive on the external channel y .

The only reduction possible is that on the first register, which consists of two reductions consuming 2 increment messages. The final configuration becomes $\llbracket (6, 2, 0) \rrbracket^{\mathcal{M}}$ and is poised along y , halting further progress.

While in this example, we consistently prioritised the instruction process when reducing, it is entirely possible to choose a different order such that every intermediate state in the original machine is passed in the encoding instead of being skipped.

Non-termination

For the non-termination example, we only have to consider some state $\llbracket(3, n, 0)\rrbracket^{\mathcal{M}}$ and see that it returns with an updated state for the first register. The encoding is given as

$$\llbracket(3, n, 0)\rrbracket^{\mathcal{M}} = \llbracket n \rrbracket_{r_0}, \llbracket 0 \rrbracket_1, \llbracket 3 \rrbracket_{r_0, r_1}^{\mathcal{M}}$$

Each component in the encoding corresponds to an instance of a process:

1. $\llbracket n \rrbracket_{r_0}$ is an instance of R^0 or R^s depending whether $n = 0$ or $n > 0$ respectively;
2. $\llbracket 0 \rrbracket_r$ is an instance of R^0 ;
3. $\llbracket 3 \rrbracket_{r_0, r_1}^{\mathcal{M}}$ is an instance of P_3 .

Like with the termination example, only the instruction process can reduce to $\llbracket 4 \rrbracket_{r'_0, r_1}^{\mathcal{M}}$, which produces reductions for the first register process and further reduction of the instruction process. Similar to the termination example, we can choose to prioritise the reduction of the instruction, which in turn requires the second register before it can reach the next state $\llbracket 3 \rrbracket_{r'_0, r'_1}^{\mathcal{M}}$. The configuration does not match the encoding, unless the first register is reduced. However, it is also possible to skip this state and reduce the instruction process. At any point, it is possible to reduce the register and converge to a configuration that does match the encoding by allowing the first register to reduce and consume the incrementing messages. For example, where we have $\llbracket 3 \rrbracket_{r''_0, r'_1}^{\mathcal{M}}$ where the encoding started with $\llbracket(3, n, 0)\rrbracket^{\mathcal{M}}$ having produced m increment messages. The first register can then reduce to $\llbracket n + m \rrbracket_{r''_0}$ which bring the configuration to match $\llbracket(3, n + m, 0)\rrbracket^{\mathcal{M}}$.

Like with the termination example, a different order can be chosen such that every intermediate state of the machine is passed in the reductions of the encoding instead of being skipped.

3.3 Correctness of the encoding

Giving the encoding alone is not proof that we can actually simulate the Minsky machine. The correctness of the semantics between the two systems needs to be established by checking whether reductions for any Minsky machine configuration are captured in our encoding (soundness) and whether our ASP process configuration does not have additional reductions that lead to states that do not correspond to states of the Minsky machine (completeness).

ASP has two properties that can be exploited to simplify the proofs that are needed. The first is confluence (Theorem 2.3), which ensures that the order of reduction does not matter if there are multiple reductions possible. After a choice between multiple reductions, they will always be able to converge to a

common state through a series of reductions. Hence, it can be assumed that if some desired state is reached through some series of reductions, there are no side effects with regard to other possible reductions and states. The second property is the asynchrony of ASP. The consequence, in combination with confluence, allows for the processes in a configuration to be studied independently and the conditions for such a process to progress.

To this degree, we have already established Lemmas 3.2 and 3.4 for the individual behaviour of the encoded instructions and registers. These results can be put together without additional behaviour and complement each other to introduce the proofs for the full encoding.

3.3.1 Soundness

First, we want to show that for any reduction in a Minsky machine, there exists a corresponding line of process reductions that leads to the same encoded subsequent state.

Theorem 3.2 (Soundness). *For some Minsky machine $\mathcal{M}$ using Definition 3.4: if $(i, m_0, m_1) \rightarrow (i', m'_0, m'_1)$, then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \llbracket (i', m'_0, m'_1) \rrbracket^{\mathcal{M}}$.*

Proof. By case analysis, using Definition 2.8. For some Minsky machine $\mathcal{M}$, a reducible instruction i can be defined as either $i : \text{INC}(j)$ or $i : \text{DECJ}(j, k)$. In the latter case, the outcome of the reduction depends on whether m_j is zero or greater than zero. Hence, the following cases need to be considered:

1. Increment: $i : \text{INC}(j)$
2. Decrement: $i : \text{DECJ}(j, k)$ and $m_j > 0$
3. Jump: $i : \text{DECJ}(j, k)$ and $m_j = 0$

Only the case for the first register is given (the analysis for the other is similar).

Case 1 (Increment). Given an instruction $i : \text{INC}(0)$, then the reduction of the Minsky machine is $(i, m_0, m_1) \rightarrow (i + 1, m_0 + 1, m_1)$. The encoding gives us:

$$\begin{aligned} \llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} &= \llbracket m_0 \rrbracket_{r_0}, \llbracket m_1 \rrbracket_{r_1}, \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} && \text{(Definition 3.4)} \\ &\rightarrow^3 \llbracket m_0 \rrbracket_{r_0}, \llbracket m_1 \rrbracket_{r_1}, \text{msg}(r'_0, r_0, \text{inc}; r'_0 \leftrightarrow r_0), \llbracket i + 1 \rrbracket_{r'_0, r_1}^{\mathcal{M}} \\ &\rightarrow^3 \llbracket m_0 + 1 \rrbracket_{r'_0}, \llbracket m_1 \rrbracket_{r_1}, \llbracket i + 1 \rrbracket_{r'_0, r_1}^{\mathcal{M}} = \llbracket (i + 1, m_0 + 1, m_1) \rrbracket^{\mathcal{M}} \end{aligned}$$

First, it can be shown using Lemma 3.4 that we can reduce the instruction to the next instruction, producing a message of **inc**. Then, using Lemma 3.2, the message can be consumed by the first register, which leads by definition to the next state of the configuration of the machine.

Case 2 (Decrement). Given an instruction $i : \text{DECJ}(0, k)$ and $m_0 > 0$, then the reduction of the machine is $(i, m_0, m_1) \rightarrow (i + 1, m_0 - 1, m_1)$. The encoding gives us:

$$\begin{aligned} \llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} &= \llbracket m_0 \rrbracket_{r_0}, \llbracket m_1 \rrbracket_{r_1}, \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} && \text{(Definition 3.4)} \\ &\rightarrow \llbracket m_0 \rrbracket_{r_0}, \llbracket m_1 \rrbracket_{r_1}, \text{msg}(r'_0, r_0, \text{decj}; r'_0 \leftrightarrow r_0), \text{proc}(x, Q) \end{aligned}$$

$$\begin{aligned}
&\rightarrow^3 \llbracket m_0 - 1 \rrbracket_{r_0''}, \text{msg}(r', r'.\text{next}; r' \leftrightarrow r''), \llbracket m_1 \rrbracket_{r_1}, \text{proc}(x, Q) \\
&\rightarrow^2 \llbracket m_0 - 1 \rrbracket_{r_0''}, \llbracket m_1 \rrbracket_{r_1}, \llbracket i + 1 \rrbracket_{r_0'', r_1}^{\mathcal{M}} = \llbracket (i + 1, m_0 - 1, m_1) \rrbracket^{\mathcal{M}}
\end{aligned}$$

First, it can be shown using Lemma 3.4 that the encoding of i leads to a poised process with a message with label **decj**, the poised process awaits either **jump** or **next**. Then, using Lemma 3.2, the message **decj** is consumed by the first register, which produces the message **next** over the channel on which Q expects the message, leading to the next instruction. By definition, this leads to the next state of the configuration of the machine.

Case 3 (Jump). Given an instruction $i : \text{DECJ}(0, k)$ and $m_0 = 0$, then the reduction of the machine is $(i, 0, m_1) \rightarrow (k, 0, m_1)$. The encoding gives us:

$$\begin{aligned}
\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} &= \llbracket 0 \rrbracket_{r_0}, \llbracket m_1 \rrbracket_{r_1}, \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} && \text{(Definition 3.4)} \\
&\rightarrow \llbracket 0 \rrbracket_{r_0}, \llbracket m_1 \rrbracket_{r_1}, \text{msg}(r_0', r_0.\text{decj}; r_0' \leftrightarrow r_0), \text{proc}(x, Q) \\
&\rightarrow^3 \llbracket 0 \rrbracket_{r_0''}, \text{msg}(r', r'.\text{jump}; r' \leftrightarrow r''), \llbracket m_1 \rrbracket_{r_1}, \text{proc}(x, Q) \\
&\rightarrow^2 \llbracket 0 \rrbracket_{r_0''}, \llbracket m_1 \rrbracket_{r_1}, \llbracket k \rrbracket_{r_0'', r_1}^{\mathcal{M}} = \llbracket (k, 0, m_1) \rrbracket^{\mathcal{M}}
\end{aligned}$$

Similar to the previous case, the encoding reduces to creating a poised process Q , awaiting message **jump** or **next**, and a message with label **decj**. Using Lemma 3.2, the process of the first register creates a message with label **jump** and continues to be 0. This is consumed by the process Q , which reduces to the instruction corresponding to a jump, which, by definition, leads to the next state of the configuration of the machine.

□

The result extends to reduction paths of arbitrary length:

Corollary 3.1 (Soundness). *For some Minsky machine $\mathcal{M}$: if $(i, m_0, m_1) \rightarrow^+ (i', m'_0, m'_1)$, then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \llbracket (i', m'_0, m'_1) \rrbracket^{\mathcal{M}}$.*

Proof. Immediate from Theorem 3.2. □

3.3.2 Completeness

The next property ensures that the encoded state does not introduce additional behaviour that deviates from the behaviour of the Minsky machine.

Theorem 3.3 (Completeness). *If $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \mathcal{S}$ for a process in ASP, then there exists (i', m'_0, m'_1) such that $\mathcal{S} \rightarrow^+ \llbracket (i', m'_0, m'_1) \rrbracket^{\mathcal{M}}$ and $(i, m_0, m_1) \rightarrow^+ (i', m'_0, m'_1)$ using Definition 3.4.*

Proof. Given any encoding of (i, m_0, m_1) we know that for $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \mathcal{S}$ that $\mathcal{S} = \llbracket (i', m'_0, m'_1) \rrbracket^{\mathcal{M}}$ or $\mathcal{S} \neq \llbracket (\cdot) \rrbracket^{\mathcal{M}}$. The former follows from Corollary 3.1.

The latter is an intermediary state of the machine for which we can show that $\mathcal{S} \rightarrow^+ \llbracket (i', m'_0, m'_1) \rrbracket^{\mathcal{M}}$. Using Lemmas 3.2 and 3.4 and Theorem 2.1, the composition of the intermediate state $\mathcal{S}$ can be derived inductively.

Given some (i, m_0, m_1) and either $i : \text{INC}(j)$ or $i : \text{DECJ}(j, k)$ ⁴, then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}}$ reduces to $\rightarrow^2 \llbracket m_0 \rrbracket_r, \llbracket m_1 \rrbracket_r, \text{msg}(r'_j, r_j.\text{inc}; r'_j \leftrightarrow r_j), \llbracket i + 1 \rrbracket_{r_0, r_1}^{\mathcal{M}} [r'_j / r_j]$ or $\rightarrow$

$\llbracket m_0 \rrbracket_r, \llbracket m_1 \rrbracket_r, \text{msg}(r'_j, r_j.\text{decj}; r'_j \leftrightarrow r_j), \text{proc}(x, Q)$ respectively. Note that in the case of incrementing, two steps in the reduction were taken; we essentially skip this internal step in the proof, as confluence allows us to apply this step at any point. By skipping, we arrive at the next encoded state faster, which is the goal. We continue to skip internal reductions as long as they do not skip poised states (e.g. Q in a decrement instruction) or the next encoded state.

In the case of incrementing, we rearrange the configuration using congruence as follows $\mathcal{S}_0, \mathcal{S}_1, \llbracket i+1 \rrbracket_{r_0, r_1}^{\mathcal{M}}$ where $\mathcal{S}_j = \llbracket m_j \rrbracket_r, \text{msg}(r'_j, r_j.\text{inc}; r'_j \leftrightarrow r_j)$ and $\mathcal{S}_{1-j} = \llbracket m_j \rrbracket_r$ putting the message adjacent to the consuming process. The configuration either reduces along $\mathcal{S}_j$ or if $i+1 \leq n$ then can reduce $\llbracket i+1 \rrbracket_{r_0, r_1}^{\mathcal{M}}$. The former implies that $\mathcal{S}$ is equal to the encoded state of $(i+1, m_0+1, m_1)$, which is already proven. The latter is essentially the inductive step. Hence $\mathcal{S}_0, \mathcal{S}_1, \llbracket i+1 \rrbracket_{r_0, r_1}^{\mathcal{M}}$ is a candidate for $\mathcal{S}$ for which it can further reduce when $i+1 \leq n$.

The case of decrementing is similar to the incrementing case, but the instruction process becomes poised before reaching the next state. Hence, $\mathcal{S}_0, \mathcal{S}_1, \text{proc}(x, Q)$ where $\mathcal{S}_j = \llbracket m_j \rrbracket_r, \text{msg}(r'_j, r_j.\text{decj}; r'_j \leftrightarrow r_j)$ and $\mathcal{S}_{1-j} = \llbracket m_j \rrbracket_r$. Here only $\mathcal{S}_j$ can reduce, mainly $\mathcal{S}_j \rightarrow^3 \llbracket m'_j \rrbracket_r, \text{msg}(r'_j, r'_j.\text{1}; r'_j \leftrightarrow r'_j)$ where $m'_j = m_j - 1$ and $\text{1} = \text{next}$ if $m_j > 0$, otherwise $m'_j = 0$ and $\text{1} = \text{jump}$. Once $\mathcal{S}_j$ has reduced, the process Q can be reduced; however, this would conclude the next encoded state. Hence $\mathcal{S}'_0, \mathcal{S}'_1, \text{proc}(x, Q)$ as a candidate of $\mathcal{S}$ where $\mathcal{S}'_j = \llbracket m'_j \rrbracket_r, \text{msg}(r'_j, r'_j.\text{1}; r'_j \leftrightarrow r'_j)$ and $\mathcal{S}_{1-j} = \llbracket m_{1-j} \rrbracket_r$. Note that we do not have to consider the configuration with $\mathcal{S}_j$ as a candidate, as it will always be possible to reduce to this state.

From these base case we can inductively iterate to create subsequent states.

From the incrementing case, we observe that its candidate can further reduce to a case that is not equate the next state of the encoding of the machine. Consider $\mathcal{S}_0, \mathcal{S}_1, \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}}$ whenever $i : \text{INC}(j)$, then $\rightarrow^2 \mathcal{S}'_0, \mathcal{S}'_1, \llbracket (i+1) \rrbracket_{r'_j/r_j}^{\mathcal{M}}$ where $\mathcal{S}'_j = \mathcal{S}_j, \text{msg}(r'_j, r_j.\text{inc}; r'_j \leftrightarrow r_j)$ and $\mathcal{S}_{1-j} = \mathcal{S}'_{1-j}$ following a similar structure as the base case. Note that the messages in $\mathcal{S}_j$ are consumed first, followed by the added message in $\mathcal{S}'_j$. We can create new candidates as long as $i : \text{INC}(<)i+1 > j'$ using these steps, alternatively $i : \text{DECJ}(<, i) + 1 > j'k'$.

The decrementing case does not allow for additional candidates to be created, but does stop the inductive incrementing case from generating new cases when the next instruction is $i : \text{DECJ}(j, k)$. Hence from $\mathcal{S}_0, \mathcal{S}_1, \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} \rightarrow^4 \mathcal{S}'_0, \mathcal{S}'_1, \text{proc}(x, Q)$ where $\mathcal{S}'_j = \mathcal{S}_j, \text{msg}(r'_j, r_j.\text{decj}; r'_j \leftrightarrow r_j)$ and $\mathcal{S}'_{1-j} = \mathcal{S}_{1-j}$. We can draw the same conclusions as from the base case and the inductive incrementing case.

The construction of the candidates starts at a state (i, m_0, m_1) . In the analysis, candidates that matched the encoding were excluded, while it is entirely possible for the encoding to consume messages and reach candidate states that are not included so far. Essentially, this means that $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^* \llbracket (i', m'_0, m'_1) \rrbracket^{\mathcal{M}} \rightarrow +\mathcal{S}$ where $(i, m_0, m_1) \rightarrow^+ (i', m'_0, m'_1)$. The point at which messages are consumed does not change the result because of confluence (Theorem 2.3). Note that candidates only have to contain at least one message and that both increment and decrement messages can be consumed.

⁴The case $i > n$ does not have to be considered as the completeness implies termination, as otherwise the encoding would reduce to a state that does correspond to the state of a Minsky machine.

We state that the set of candidates is complete and no other reductions are possible.

Hence, the following candidates for $\mathcal{S}$ are considered (with $j = 0$ as $j = 1$ is analogous), when $\mathcal{S} \neq \llbracket (\cdot) \rrbracket^{\mathcal{M}}$:

1. $\mathcal{S} = \llbracket m'_0 - 1 \rrbracket_{r_0}, \text{msg}(r_j, r_j.\text{next}; r_j \leftrightarrow r'_j),$
 $\llbracket m'_1 \rrbracket_{r_1}, \text{msg}(r_1^1, r_1.\text{inc}; r_1^1 \leftrightarrow r_1), \dots, \text{msg}(r_1^{n_1}, r_1^{n_1-1}.\text{inc}; r_1^{n_1} \leftrightarrow r_1^{n_1-1}),$
 $\text{proc}(x, Q), \text{ with } m'_0 > 0;$
2. $\mathcal{S} = \llbracket 0 \rrbracket_{r_0}, \text{msg}(r_j, r_j.\text{jump}; r_j \leftrightarrow r'_j),$
 $\llbracket m'_1 \rrbracket_{r_1}, \text{msg}(r_1^1, r_1.\text{inc}; r_1^1 \leftrightarrow r_1), \dots, \text{msg}(r_1^{n_1}, r_1^{n_1-1}.\text{inc}; r_1^{n_1} \leftrightarrow r_1^{n_1-1}),$
 $\text{proc}(x, Q);$
3. $\mathcal{S} = \llbracket m'_0 \rrbracket_{r_0}, \text{msg}(r_0^1, r_0.\text{inc}; r_0^1 \leftrightarrow r_0), \dots, \text{msg}(r_0^{n_0}, r_0^{n_0-1}.\text{inc}; r_0^{n_0} \leftrightarrow r_0^{n_0-1}),$
 $\llbracket m'_1 \rrbracket_{r_1}, \text{msg}(r_1^1, r_1.\text{inc}; r_1^1 \leftrightarrow r_1), \dots, \text{msg}(r_1^{n_1}, r_1^{n_1-1}.\text{inc}; r_1^{n_1} \leftrightarrow r_1^{n_1-1}),$
 $\llbracket i' + n_0 + n_1 \rrbracket_{r_0^{n_0}, r_1^{n_1}}^{\mathcal{M}}, \text{ with either } n_0 > 0 \text{ or } n_1 > 0;$
4. $\mathcal{S} = \llbracket m'_0 \rrbracket_{r_0}, \text{msg}(r_0^1, r_0.\text{inc}; r_0^1 \leftrightarrow r_0), \dots, \text{msg}(r_0^{n_0-1}, r_0^{n_0-2}.\text{inc}; r_0^{n_0-1} \leftrightarrow$
 $r_0^{n_0-2}),$
 $\text{msg}(r_0^{n_0}, r_0^{n_0-1}.\text{decj}; r_0^{n_0} \leftrightarrow r_0^{n_0-1}),$
 $\llbracket m'_1 \rrbracket_{r_1}, \text{msg}(r_1^1, r_1.\text{inc}; r_1^1 \leftrightarrow r_1), \dots, \text{msg}(r_1^{n_1}, r_1^{n_1-1}.\text{inc}; r_1^{n_1} \leftrightarrow r_1^{n_1-1}),$
 $\text{proc}(x, Q_{r_0^{n_0}, r_1^{n_1}}), \text{ with } n_0 > 0.$

What was shown so far is delaying the application that leads to the next subsequent state of the encoding, which we know from soundness (Corollary 3.1), to lead to the next state of the encoding. Moreover, as a consequence of confluence (Theorem 2.3), each reduction that leads to the next state is preserved and can be applied to reach the next state of the encoding.

Case 1. As the encoding of the instructions follows the definition of the machine, instructions i' to $i' + n_1$ are only increments, with n_1 increments on the second register. This means that $(i', m'_0, m'_1) \rightarrow^+ (i' + n_1, m'_0 - 1, m'_1 + n_1)$.

Using Lemma 3.2 and applying it n_1 times to (the encoding of) register 1, followed by applying Lemma 3.4, it can be shown that:

$$\begin{aligned} \mathcal{S} &\rightarrow^+ \llbracket m'_0 - 1 \rrbracket_{r'_0}, \llbracket m'_1 + n_1 \rrbracket_{r_1^m}, \llbracket i' + n_1 + 1 \rrbracket_{r'_0, r_1^m}^{\mathcal{M}} \\ &= \llbracket (i' + n_1 + 1, m'_0 - 1, m'_1 + n_1) \rrbracket^{\mathcal{M}} \end{aligned}$$

Then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \mathcal{S} \rightarrow^+ \llbracket (i' + n_1 + 1, m'_0 - 1, m'_1 + n_1) \rrbracket^{\mathcal{M}},$ with $(i, m_0, m_1) \rightarrow^+ (i' + n_1 + 1, m'_0 - 1, m'_1 + n_1).$

Case 2. As the encoding of the instructions follows the definition of the machine, instructions i' to $i' + n_1$ are only increments, with n_1 increments on the second register. This means that $(i', 0, m'_1) \rightarrow^+ (i' + n_1, 0, m'_1 + n_1).$

Using Lemma 3.2 and applying it n_1 times to (the encoding of) register 1, followed by applying Lemma 3.4, it can be shown that:

$$\begin{aligned} \mathcal{S} &\rightarrow^+ \llbracket 0 \rrbracket_{r'_0}, \llbracket m'_1 + n_1 \rrbracket_{r_1^m}, \llbracket k \rrbracket_{r'_0, r_1^m}^{\mathcal{M}} \\ &= \llbracket (k, 0, m'_1 + n_1) \rrbracket^{\mathcal{M}} \end{aligned}$$

Then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \mathcal{S} \rightarrow^+ \llbracket (k, 0, m'_1 + n_1) \rrbracket^{\mathcal{M}},$ with $(i, m_0, m_1) \rightarrow^+ (k, 0, m'_1 + n_1).$

Case 3. As the encoding of the instructions follows the definition of the machine, instructions i' to $i' + n_0 + n_1$ are only increments, with n_0 and n_1 increments on the first and second registers, respectively. This means that $(i', m'_0, m'_1) \rightarrow^+ (i' + n_0 + n_1, m'_0 + n_0, m'_1 + n_1)$.

Using Lemma 3.2 and applying it n_0 times to (the encoding of) register 0 and n_1 times to (the encoding of) register 1, it can be shown that:

$$\begin{aligned} \mathcal{S} &\rightarrow^+ \llbracket m'_0 + n_0 \rrbracket_{r_0^n}, \llbracket m'_1 + n_1 \rrbracket_{r_1^m}, \llbracket i' + n_0 + n_1 \rrbracket_{r_0^n, r_1^m}^{\mathcal{M}} \\ &= \llbracket (i' + n_0 + n_1, m'_0 + n_0, m'_1 + n_1) \rrbracket^{\mathcal{M}} \end{aligned}$$

Then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \mathcal{S} \rightarrow^+ \llbracket (i' + n_0 + n_1, m'_0 + n_0, m'_1 + n_1) \rrbracket^{\mathcal{M}}$, with $(i, m_0, m_1) \rightarrow^+ (i' + n_0 + n_1, m'_0 + n_0, m'_1 + n_1)$.

Case 4. As the encoding of the instructions follows the definition of the machine, instructions i' to $i' + n_0 + n_1 - 1$ are only increments followed by instruction $i' + n_0 + n_1$, which decrements the first register, with n_0 increments on the first register and n_1 increments on the second. This means that either $(i', m'_0, m'_1) \rightarrow^+ (i' + n_0 + n_1, m'_0 + n_0 - 1, m'_1 + n_1)$ or $(i', m'_0, m'_1) \rightarrow^+ (k, 0, m'_1 + n_1)$ (whenever $m'_0 = 0 \wedge n_0 = 0$).

Using Lemma 3.2 and applying it n_0 times to (the encoding of) the first register and n_1 times to (the encoding of) the second register, we obtain:

$$\begin{aligned} \mathcal{S} &\rightarrow^* \llbracket m'_0 + n_0 \rrbracket_{r_0^{n_0}}, \llbracket m'_1 + n_1 \rrbracket_{r_1^{n_1}}, \\ &\text{msg}(r_0^{n_0}, r_0^{n_1+1} \cdot \text{decj}; r_0^{n_0} \leftrightarrow r_0^{n_1+1}), \llbracket i' + n_0 + n_1 \rrbracket_{r_0^{n_0+1}, r_1^{n_1}}^{\mathcal{M}} = \mathcal{S}' \end{aligned}$$

Depending on whether $m'_0 + n_0 > 0$, the instruction moves $i' + n_0 + n_1 + 1$ and otherwise to k as declared in the decrement. For the decrement case:

$$\begin{aligned} \mathcal{S}' &\rightarrow^+ \llbracket m'_0 + n_0 \rrbracket_{r_0^{n_0}}, \llbracket m'_1 + n_1 \rrbracket_{r_1^{n_1}}, \llbracket i' + n_0 + n_1 \rrbracket_{r_0^{n_0}, r_1^{n_1}}^{\mathcal{M}} \\ &= \llbracket (i' + n_0 + n_1 + 1, m'_0 + n_0 - 1, m'_1 + n_1) \rrbracket^{\mathcal{M}} \end{aligned}$$

Then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \mathcal{S} \rightarrow^+ \llbracket (i' + n_0 + n_1 + 1, m'_0 + n_0 - 1, m'_1 + n_1) \rrbracket^{\mathcal{M}}$, with $(i, m_0, m_1) \rightarrow^+ (i' + n_0 + n_1 + 1, m'_0 + n_0 - 1, m'_1 + n_1)$. For the jump case:

$$\mathcal{S}' \rightarrow^+ \llbracket 0 \rrbracket_{r_0'}, \llbracket m'_1 + n_1 \rrbracket_{r_1^m}, \llbracket k \rrbracket_{r_0', r_1^m}^{\mathcal{M}} = \llbracket (k, 0, m'_1 + n_1) \rrbracket^{\mathcal{M}}$$

Then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \mathcal{S} \rightarrow^+ \llbracket (k, 0, m'_1 + n_1) \rrbracket^{\mathcal{M}}$, with $(i, m_0, m_1) \rightarrow^+ (k, 0, m'_1 + n_1)$.

□

3.4 Turing completeness

The encoding and its proofs of correctness show that ASP can simulate Minsky machines. As Minsky machines are known to be Turing complete, it can simply be stated that ASP are Turing complete.

Corollary 3.2. *ASP is Turing complete.*

Chapter 4

Counting with nested types

4.1 Introduction

In this chapter, we will complement the encoding developed in the previous chapter by using the nested types to define a more precise protocol that captures the state and transitions of the two-counter machine.

4.1.1 Complementing nested types with processes

Chapter 3 discussed one of the results by Gay et al. [3], namely that two-counter session types cannot be expressed with nested types by themselves. As a consequence, it is only possible to show that ASP is Turing complete from the perspective of the process language. While session types alone cannot show Turing completeness, they can still play an important role in the encoding that implies correctness of the encoding by capturing its state and transitions.

The nested types introduced in Chapter 3, like $C^{s,s}[\alpha, \beta]$, fell short to capture the protocol of two-counter machines. They could either capture increments or decrements, but not both. Herein lies the problem, as a protocol that only offers one and relies on other mechanisms of the language (say, processes) to perform the other, does not show the correctness of the encoding.

Rather than devising a protocol that captures both counters, an alternative approach is to have a protocol that captures one counter while ensuring that the

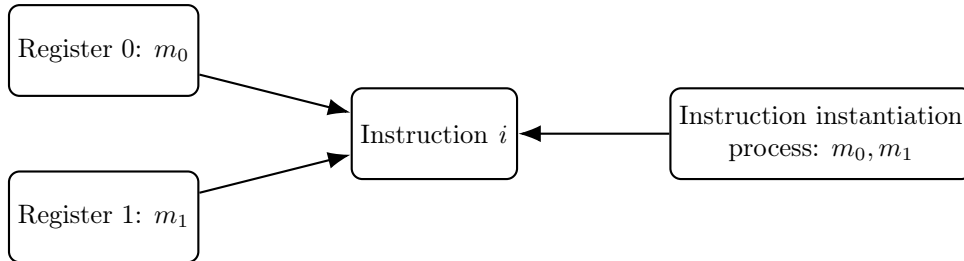


Figure 4.1: Structure of the process configuration of the encoded Minsky machine

behaviour of the two-counters is jointly captured at the process level. This is a “hybrid” approach, where control is shared between protocols and processes; it can address the concerns of capturing both transitions of decrement and increments.

The required one-counter protocol can be expressed in nested types as Peano arithmetic. Peano arithmetic starts with the natural number 0 and uses the inductive successor function $S(n)$ to create any subsequent natural number. This can be translated directly to a nested type and forms the basis of a one-counter protocol, with the session types $0[]$ and $S[\alpha]$. The reason why this works is that it simply functions as a push-down type; it is not necessary to infer from α what the next state is, as the type variable represents the value before incrementing, and thus is the next state.

We can make the protocol concrete by adding the choices for each state:

Definition 4.1 (Polymorphic counter protocol). We define the (polymorphic) counter type as follows:

$$\begin{aligned} 0[] &= \&\{\text{incr}: S[0[]], \text{jump}: 0[]\} && \text{(zero)} \\ S[\alpha] &= \&\{\text{incr}: S[S[\alpha]], \text{decr}: \alpha\} && \text{(successor)} \end{aligned}$$

The protocol captures: increments (by wrapping the current state with the successor $S[\cdot]$); jump (by staying at the state $0[]$); and also decrement (by continuing to be the unwrapped/previous state α). The explicit inclusion of jump is chosen to highlight this step, even though it should not change the state of the register, only influencing the instruction value. This way, an encoding of natural numbers into types follows naturally:

Definition 4.2 (Encoding counter protocol). Using Definition 4.1, the definition of encoding a natural number into the counter type is given as:

$$\llbracket m \rrbracket = \begin{cases} 0[] & \text{if } m = 0 \\ S[\llbracket m - 1 \rrbracket] & \text{if } m > 0 \end{cases} \quad (4.1)$$

The implementation of the counter protocol into a register process will be given in Definition 4.3.

State-aware Instructions

Unlike the protocol in Definition 3.1, the new counter protocol does not share the current state (by sending `next` or `jump`), but rather expects processes on both sides to be aware of the state (in contrast, in the encoding of Chapter 3, only the register processes were aware). This design has consequences on how the branching is implemented in the instruction process.

With the instruction process being *state-aware*, we can no longer use a singular process for each instruction i defined in the machine $\mathcal{M}$. This comes down to the fact that the behavior of $i : \text{DECJ}(j, k)$ depends on the value of register j : if j holds the value 0, then it needs to be encoded as a process sending a `jump` label to the encoding of register j ; otherwise, if the value of j is greater than 0, then the encoding needs to send the `decr` label.

Furthermore, the step after sending the label is somewhat more complicated. How do we instantiate the next instruction? This comes down to the same

problem that we encountered with two-counter types: do we transition to a state that becomes zero or greater than zero? It is not possible to share this state, as we cannot infer, even at the process level, something about the type parameter, nor can we send the type. Instantiation of the next instruction process is also not possible in the register process, as, even though it is aware of its own state, it does not know about the state of the other register, nor can we send the type to instantiate the next instruction.

This is where we introduce the nested type for instantiation. Where nested session types failed to capture the transition in decrements, we can create an agnostic type to instantiate the instruction without revealing the state after decrementing, using processes to deal with the transition.

Consider the type $T^{s,s}[\alpha, \beta]$ that corresponds to a state of a Minsky-machine where both registers hold values greater than 0, where α and β correspond to the state of the first registers, respectively. When an increment happens, for example on $j = 0$, then it simply transitions to $T^{s,s}[S[\alpha], \beta]$ as the instruction can instantiate the next instruction in such a case. However, when decrementing, also on $j = 0$, it simply forwards α and $S[\beta]$, and continues to be the instruction process which simply terminates. Such a nested type would match one-counter session types, where the one-counter correctly captures the semantics of a two-counter machine, and the nested types allow the processes to instantiate the subsequent instruction. The concrete type definition then becomes:

$$\begin{aligned} T^{s,s}[\alpha, \beta] = \&\{ \text{exp}_0 : T^{s,s}[S[\alpha], \beta] \\ &\text{exp}_1 : T^{s,s}[\alpha, S[\beta]] \\ &\text{ins}_0 : \alpha \multimap S[\beta] \multimap \mathbf{1} \\ &\text{ins}_1 : S[\alpha] \multimap \beta \multimap \mathbf{1} \} \end{aligned}$$

As this type lets the process take care of the transition, it is agnostic in the sense that it does not say anything about the state after decrementing. As such, and to clearly distinguish between this type and the one-counter types, instead of using labels like incr_j and decr_j , the labels exp_j (expand on register j) and ins_j (instantiate instruction after decrementing register j) are used respectively.

While the definition of $T^{s,s}[\alpha, \beta]$ is given, it is not necessarily complete, as processes might need additional options to allow maintaining the underlying data structure when incrementing and possibly decrementing. This will be discussed in Section 4.1.1.

Finally, it has to be noted that the duality between the one-counter types and the instantiation types is non-optional. Take away the one-counter types and the instantiation type after decrementing offers only a generic type of $\mathbf{1} \multimap \mathbf{1}$ with the absence of any state. This makes the type parameters used in the incrementing superfluous, as they are only pushed down but never used. On the other hand, without the nested type, it is not possible to instantiate the next instruction, as the nested type bridges the one-counter types in a way that allows the processes to be Turing complete.

The implementation of the state-aware instruction process using the counter protocol and the instantiation protocol will be given in Definition 4.6.

Instantiation tree process

Where the register and instruction process are structurally similar to those in the encoding given in Chapter 3, with the necessity of the nested instantiation protocol, a new process configuration is required to implement it. With the idea that we capture transitions concerning decrementing on the process level, we require such a process to know what comes after decrementing. This is necessary as an instruction process needs to be able to continue interacting with the registers and is only able to do so when it has a concrete type that exposes the choices, like $S[\alpha]$ or $0[]$. What such a process will not be able to know is what instruction comes next, and hence needs to be able to handle every possible way of decrementing.

The data structure that naturally follows is a tree structure where each node is a tuple of the values of the registers, where each child of a node is a decrement on one of the values of the parent up to zero. For a node that holds values greater than zero, it always has two children, while a node that only holds one value greater than zero only has one child, as those are the only decrements possible. The leaf of the tree correspond with the tuple with both values zero.

With such a tree data structure, whenever we decrement and need to instantiate the next instruction, we can simply use one branch to match the state of the registers. While for incrementing, we simply expand the tree with the new possible ways to decrement. From the perspective of ASP, this means two things: termination and duplication. Termination is relatively straightforward, as it simply traverses a branch, closing each process, which is necessary as we cannot leave residual processes, as that would violate session fidelity in ASP.

Duplication is slightly more complicated and is necessary to introduce new branches on a linear tree when incrementing the value of zero. Leafs and nodes with two children never need duplication, simply instantiating the appropriate next state directly or forwarding the expansion, respectively. A linear node, on the other hand, might forward the expansion or needs to create a branch to account for the new way to decrement. Whenever it needs to create a new branch, it duplicates the child first and then expands each copy for one of the registers, creating options for decrementing along each register (an example using the concrete implementation is given in Figure 4.6).

We formalise this into a nested protocol in Definition 4.4 and further implement this in Definition 4.7.

It has to be noted that, while this is not the most efficient data structure, it does capture the possible ways of decrementing two registers more naturally, directly showing the possible options to decrement the registers, which makes the definitions and transitions easier to understand. Furthermore, the additions of termination and duplications are only used internally by the processes composing the data structure, as the instruction process will never have a need for such options.

4.2 Encoding of registers

First, we encode registers using the counter protocol in Definition 4.2. It follows the same configuration structure as the encoding from the previous chapter, with the only difference being the definition of the case of decrement-jump, bypassing

the sharing of the state.

Definition 4.3 (Encoding register polymorphic). The encoding of $m \in \mathbb{N}$ into a configuration $\mathcal{S}$ in NST is given as follows, using Definition 4.2:

$$\llbracket m \rrbracket_r = \begin{cases} \text{proc}(r, R^0) & \text{if } m = 0 \\ \llbracket m - 1 \rrbracket_p, \text{proc}(r, R^s[\langle m - 1 \rangle / \alpha]) & \text{if } m > 0 \end{cases} \quad (4.2)$$

with the set of process definitions $\Sigma_{\mathcal{M}}$:

$$\begin{aligned} \cdot \vdash r^0[] &= R^0 :: (r : 0[]) \in \Sigma_{\mathcal{M}} \\ p : \alpha \vdash r^s[\alpha] &= R^s :: (r : S[\alpha]) \in \Sigma_{\mathcal{M}} \end{aligned}$$

and the following definitions for R^0 and R^s :

$$\begin{aligned} R^0 &= \text{case } r \text{ (incr} \Rightarrow c \leftarrow r^0[]; r \leftarrow r^s[0[]] \text{ } c \\ &\quad \text{jump} \Rightarrow r \leftarrow r^0[]) \\ R^s &= \text{case } r \text{ (incr} \Rightarrow c \leftarrow r^s[\alpha] \text{ } p; r \leftarrow r^s[S[\alpha]] \text{ } c \\ &\quad \text{decr} \Rightarrow p \leftrightarrow r) \end{aligned}$$

Unlike the previous encoding of registers, the correctness of the behaviour is guaranteed as it follows the session type, but only if they are well typed. To elaborate, whenever we have some $\llbracket m \rrbracket_r$, the channel r is of type $\langle m \rangle$. This means that whenever we increment the session type tells us that it continues to be type $\langle m + 1 \rangle$, which happens in the process due to the instantiation of $r^s[\langle m \rangle]$ (after the replication of the current process). Also, when a jump occurs, the state remains 0 for both the type and the process. Finally, for a decrement, according to Definitions 4.2 and 4.3 the encoding follows to be $\langle m - 1 \rangle$ and $\llbracket m - 1 \rrbracket_{r'}$, where r' matches the type of $m - 1$.

We want to formally verify that the encoding of registers can be typed in ASP.

Lemma 4.1. *Given any $\llbracket m \rrbracket_r$ then, from Definition 4.3, then*

$$\cdot \vdash_{\Sigma_{\mathcal{M}}} \llbracket m \rrbracket_r :: (r : \langle m \rangle)$$

Proof. Appendix B.1 gives type derivations for the processes of the encoding. $\square$

4.3 Instantiation of state-aware instructions

In this section, we define the concrete instantiation protocol for the tree data structure as discussed in the introduction. It is used to implement the definitions of the state-aware instruction processes and then the instantiation tree processes, which we define next. As the instantiation tree processes rely on the process definitions from the definition of the instruction processes, the definition of instruction processes is given first. We also separate the encodings of the instruction and tree processes for the sake of clarity and simplicity in discussing and defining the encoding.

Definition 4.4 (Instruction instantiation protocol). The definition of the instruction instantiation types:

$$\begin{aligned}
T^{0,0}[] &= \&\{ \text{exp}_0: T^{s,0}[0[]], & \text{exp}_1: T^{0,s}[0[]], \\
& \text{dup}: T^{0,0}[] \otimes T^{0,0}[], & \text{ter}: \mathbf{1} \} \\
T^{s,0}[\alpha] &= \&\{ \text{ins}_0: I[\alpha, 0[]], \\
& \text{exp}_0: T^{s,0}[S[\alpha]], & \text{exp}_1: T^{s,s}[\alpha, 0[]], \\
& \text{dup}: T^{s,0}[\alpha] \otimes T^{s,0}[\alpha], & \text{ter}: \mathbf{1} \} \\
T^{0,s}[\beta] &= \&\{ \text{ins}_1: I[0[], \beta], \\
& \text{exp}_0: T^{s,s}[0[], \beta], & \text{exp}_1: T^{0,s}[S[\beta]], \\
& \text{dup}: T^{0,s}[\beta] \otimes T^{0,s}[\beta], & \text{ter}: \mathbf{1} \} \\
T^{s,s}[\alpha, \beta] &= \&\{ \text{ins}_0: I[\alpha, S[\beta]], & \text{ins}_1: I[S[\alpha], \beta], \\
& \text{exp}_0: T^{s,s}[S[\alpha], \beta], & \text{exp}_1: T^{s,s}[\alpha, S[\beta]], \\
& \text{ter}: \mathbf{1} \}
\end{aligned}$$

With the type $I[\cdot]$ for instantiation given as:

$$I[\alpha, \beta] = \alpha \multimap \beta \multimap \mathbf{1} \multimap \&\{ \mathbf{i}: \mathbf{1} \}_{i \in L_{\mathcal{M}}}$$

As the instantiation type is not aware of the current instruction, it is extended with the receipt of the label, such that an instruction process can share this state. Here we make use of Definition 2.9, which defines the set of all instructions that can be reached from some configuration of a two-counter machine, similar to how the process definitions of instructions are defined in Definition 3.3. This works both for the cases where it moves on to $i + 1$ or when jumping to instruction k . Besides the register channels, it is also necessary to send the termination channel y to make the process typable.

We also define the instantiation protocol as an encoding of both register values:

Definition 4.5. The encoding of $m_0, m_1 \in \mathbb{N}$ into the tree instantiation protocol is given as follows, using Definition 4.4:

$$\mathcal{T}^{\mathcal{M}}(m_0, m_1) = \begin{cases} T^{0,0}[] & \text{if } m_0 = m_1 = 0 \\ T^{s,0}[\langle m_0 - 1 \rangle] & \text{if } m_0 > 0 \wedge m_1 = 0 \\ T^{0,s}[\langle m_1 - 1 \rangle] & \text{if } m_0 = 0 \wedge m_1 > 0 \\ T^{s,s}[\langle m_0 - 1 \rangle, \langle m_1 - 1 \rangle] & \text{if } m_0 > 0 \wedge m_1 > 0 \end{cases} \quad (4.3)$$

We add this encoding primarily as a substitute of the types given in Definition 4.4 whenever the state of the registers is unknown. This allows us to write a more general statement about the typability of processes implementing the protocol without having to consider the state of the registers.

4.3.1 Encoding of state-aware instructions

As the instantiation protocol will instantiate the instructions, they depend on the process definitions of the instructions. Only the definition of the instantiation types is required to give the formal definition.

Definition 4.6 (Instruction processes). Given some $i \in \mathbb{Z}^+$ and Minsky machine $\mathcal{M}$, the encoding of i into a configuration $\mathcal{S}$ in ASP is given as, depending on the values of the registers $m_0, m_1 \in \mathbb{N}$, using Definitions 4.2 and 4.4¹:

$$\llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}} = \begin{cases} \text{proc}(x, P_i^{0,0}) & \text{if } m_0 = m_1 = 0 \\ \text{proc}(x, P_i^{s,0}[\langle m_0 - 1 \rangle / \alpha]) & \text{if } m_0 > 0 \wedge m_1 = 0 \\ \text{proc}(x, P_i^{0,s}[\langle m_1 - 1 \rangle / \beta]) & \text{if } m_0 = 0 \wedge m_1 > 0 \\ \text{proc}(x, P_i^{s,s}[\langle m_0 - 1 \rangle / \alpha, \langle m_1 - 1 \rangle / \beta]) & \text{if } m_0 > 0 \wedge m_1 > 0 \end{cases}$$

With the set of process definitions $\Sigma_{\mathcal{M}}$:

$$\begin{aligned} r_0 : 0[], \quad r_1 : 0[], \quad t : T^{0,0}[], \quad y : \mathbf{1} \vdash p_{i \in L_{\mathcal{M}}}^{0,0}[] &= P_i^{0,0} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \\ r_0 : S[\alpha], \quad r_1 : 0[], \quad t : T^{s,0}[\alpha], \quad y : \mathbf{1} \vdash p_{i \in L_{\mathcal{M}}}^{s,0}[\alpha] &= P_i^{s,0} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \\ r_0 : 0[], \quad r_1 : S[\beta], \quad t : T^{0,s}[\beta], \quad y : \mathbf{1} \vdash p_{i \in L_{\mathcal{M}}}^{0,s}[\beta] &= P_i^{0,s} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \\ r_0 : S[\alpha], \quad r_1 : S[\beta], \quad t : T^{s,s}[\alpha, \beta], \quad y : \mathbf{1} \vdash p_{i \in L_{\mathcal{M}}}^{s,s}[\alpha, \beta] &= P_i^{s,s} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \end{aligned}$$

where processes $P_i^{0,0}$, $P_i^{s,0}$, $P_i^{0,s}$, and $P_i^{s,s}$ are given in Figure 4.2.

The instruction process behaves in much the same way between the different states concerning the interaction with the register processes, with the only difference being the interaction with the instantiation process. For each instruction process, except for $i > n$, the instruction sends the appropriate label to the register depending on the definition of the instruction and the value of the respective register. After the tree process can be used to update the state of the register or to instantiate the next instruction, with each case using the tree as follows:

- **Incrementing:** The tree receives the expansion label for the appropriate register to synchronise the new state of the register, and after continues to be the next instruction $i + 1$;
- **Decrementing:** The tree is used to instantiate the next instruction by receiving the instantiate label, ins_j , followed by the channels required for the next instruction process to be instantiated; register channels and the external channel y . This is followed by the label of the next instruction, $(i + 1)$. As the root of the tree becomes the next instruction, it is forwarded to take the place of the current instruction process;
- **Jumping:** Only the next instruction k needs to be instantiated as the state of the registers does not change.

The special case of termination, when $i > n$, is simulated by waiting on the external channel y , which leads to a poised state of the whole configuration, as no message will be received, similar to the implementation of Definition 3.3.

Similar to Definition 3.3, as the set of process definitions $\Sigma_{\mathcal{M}}$ can only be finite, we use the set of reachable instructions (Definition 2.9) to define the

¹The subscripts r_0 , r_1 , and t refer to the internal renaming of the channels for the register 0 and 1 and the instantiation tree in that order. Channels x and y never reduce; hence, no renaming is necessary.

$$\begin{aligned}
P_i^{0,0} &= \begin{cases} r_0.\text{incr}; t.\text{exp}_0; x \leftarrow p_{i+1}^{s,0}[0] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{INC}(0) \\ r_1.\text{incr}; t.\text{exp}_1; x \leftarrow p_{i+1}^{0,s}[0] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{INC}(1) \\ r_j.\text{jump}; x \leftarrow p_k^{0,0} \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{DECJ}(j, k) \\ \text{wait } y; Q_i^{0,0} & \text{if } i > n \end{cases} \\
P_i^{s,0} &= \begin{cases} r_0.\text{incr}; t.\text{exp}_0; x \leftarrow p_{i+1}^{s,0}[S[\alpha]] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{INC}(0) \\ r_1.\text{incr}; t.\text{exp}_1; x \leftarrow p_{i+1}^{s,s}[\alpha, 0] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{INC}(1) \\ r_0.\text{decr}; t.\text{ins}_0; \text{send } t \ r_0; \text{send } t \ r_1; \\ \quad \text{send } t \ y; t.(i+1); t \leftrightarrow x & \text{if } i : \text{DECJ}(0, k) \\ r_1.\text{jump}; x \leftarrow p_k^{s,0}[\alpha] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{DECJ}(1, k) \\ \text{wait } y; Q_i^{s,0} & \text{if } i > n \end{cases} \\
P_i^{0,s} &= \begin{cases} r_0.\text{incr}; t.\text{exp}_0; x \leftarrow p_{i+1}^{s,s}[0, \beta] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{INC}(0) \\ r_1.\text{incr}; t.\text{exp}_1; x \leftarrow p_{i+1}^{0,s}[S[\beta]] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{INC}(1) \\ r_0.\text{jump}; x \leftarrow p_k^{0,s}[\beta] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{DECJ}(0, k) \\ r_1.\text{decr}; t.1; \text{send } t \ r_0; \text{send } t \ r_1; \\ \quad \text{send } t \ y; t.(i+1); t \leftrightarrow x & \text{if } i : \text{DECJ}(1, k) \\ \text{wait } y; Q_i^{0,s} & \text{if } i > n \end{cases} \\
P_i^{s,s} &= \begin{cases} r_0.\text{incr}; t.\text{exp}_0; x \leftarrow p_{i+1}^{s,s}[S[\alpha], \beta] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{INC}(0) \\ r_1.\text{incr}; t.\text{exp}_1; x \leftarrow p_{i+1}^{s,s}[\alpha, S[\beta]] \quad r_0 \ r_1 \ t \ y & \text{if } i : \text{INC}(1) \\ r_0.\text{decr}; t.\text{ins}_0; \text{send } t \ r_0; \text{send } t \ r_1; \\ \quad \text{send } t \ y; t.(i+1); t \leftrightarrow x & \text{if } i : \text{DECJ}(0, k) \\ r_1.\text{decr}; t.\text{ins}_1; \text{send } t \ r_0; \text{send } t \ r_1; \\ \quad \text{send } t \ y; t.(i+1); t \leftrightarrow x & \text{if } i : \text{DECJ}(1, k) \\ \text{wait } y; Q_i^{s,s} & \text{if } i > n \end{cases}
\end{aligned}$$

Figure 4.2: Processes used in Definition 4.6.

processes (see Lemma 2.1). This allows the instruction process to instantiate the next instruction according to the definition of the machine.

We want to formally verify that the encoding of instructions can be typed in ASP.

Lemma 4.2. *Given any $\llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$ then, from Definition 4.6 and using Definition 4.5, then*

$$r_0 : \langle m_0 \rangle, r_1 : \langle m_1 \rangle, t : \mathcal{T}^{\mathcal{M}} \langle m_0, m_1 \rangle, y : \mathbf{1} \models_{\Sigma, \mathcal{M}} \llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}} :: (x : \mathbf{1})$$

Proof. Appendix B.2 gives type derivations for the processes of the encoding. $\square$

4.3.2 Encoding the instantiation tree

The addition of termination and duplication in the instantiation type reveals the necessity of communication between a node process and its predecessor(s), unlike

the definitions of the register processes. For example, termination needs to reach all processes in a branch. However, this is not the only reason; these processes also need to be aware of the previous state to instantiate the next instruction. As a consequence, each node processes need to be aware of the specific transition they implement. These dependencies are illustrated in Figure 4.3, where each edge represents a decrement on a register $j \in \{0, 1\}$. The notation of 0, 1 and s tells what the process precedes, for example:

1. The process $N^{0,0}$ is a leaf and has no dependencies, as no decrements are possible. It offers the protocol $T^{0,0}[]$;
2. The process $N^{1,0}$ uses a channel with protocol $T^{0,0}[]$, which is the decrement of the first register, which is only offered by process $N^{0,0}$. It offers the protocol $T^{s,0}[0[]]$;
3. The process $N^{s,0}$ uses a channel with protocol $T^{s,0}[\alpha]$ which is the decrement on the first register which processes $N^{s,0}$ and $N^{1,0}$ can offer. This depends on whether α is substituted by a type of the form $S[\cdot]$ or $0[]$ respectively. The exact substitution is given by the encoded value of the register from Definition 4.2, which determines the exact process, whether that is $N^{s,0}$ or $N^{1,0}$. It offers the protocol $T^{s,0}[S[\alpha]]$;
4. The process $N^{1,1}$ uses the channels with protocols $T^{0,s}[0[]]$ and $T^{s,0}[0[]]$ which decrement the first and second register respectively. Only processes $N^{0,1}$ and $N^{1,0}$ implement these protocols. It offers the protocol $T^{s,s}[0[], 0[]]$;
5. The process $N^{s,s}$ uses the channels with protocols $T^{s,s}[\alpha, S[\beta]]$ and $T^{s,s}[S[\alpha], \beta]$ which decrement the first and second register respectively. The first protocol is offered by the processes $N^{s,s}$ and $N^{1,s}$ with a similar dependency as (3); the second protocol follows naturally. It offers the protocol $T^{s,s}[S[\alpha], S[\beta]]$.

Hence, each node knows whether the previous state is greater than zero or not for both registers, but cannot look beyond that. This is enough for it to instantiate the next instruction when performing a decrement on either register.

We then give the formal definition of the instantiation process, which depends on the values of both registers.

Definition 4.7 (Instruction instantiation process). Given some $m_0, m_1 \in \mathbb{N}$ and using the process definitions given in Definition 4.6, the encoding of m_0 and m_1 into the a configuration $\mathcal{S}$ in ASP is given as, using Definitions 4.2

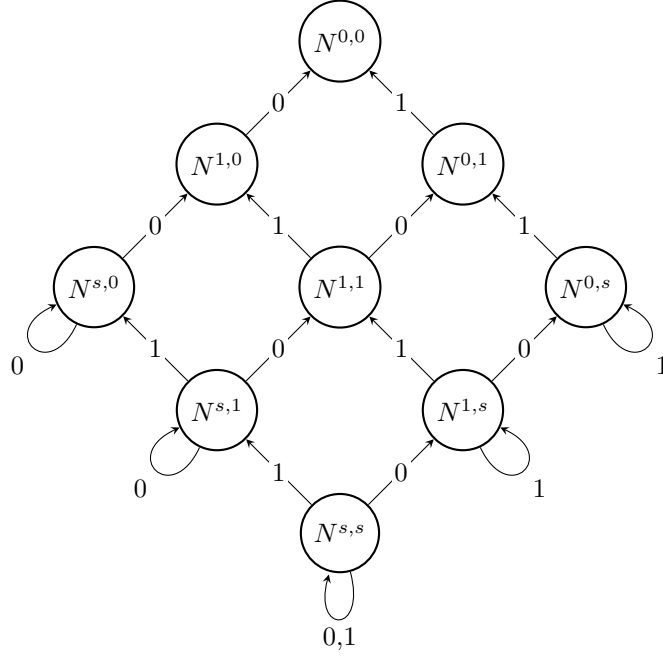


Figure 4.3: Definition 4.7: the relations of the node processes in the instantiation tree (see Figures 4.4 and 4.5). Each arrow denotes a channel which points to the process providing it.

and 4.4:

$$\{m_0, m_1\}_z^{\mathcal{M}} = \begin{cases} \text{proc}(z, N^{0,0}) & \text{if } m_0 = m_1 = 0 \\ \{0, 0\}_a^{\mathcal{M}}, \text{proc}(z, N^{1,0}) & \text{if } m_0 = 1 \wedge m_1 = 0 \\ \{m-1, 0\}_a^{\mathcal{M}}, \text{proc}(z, N^{s,0}[\lfloor m_0 - 2 \rfloor / \alpha]) & \text{if } m_0 > 1 \wedge m_1 = 0 \\ \{0, 0\}_b^{\mathcal{M}}, \text{proc}(z, N^{0,1}) & \text{if } m_0 = 0 \wedge m_1 = 1 \\ \{0, m_1 - 1\}_b^{\mathcal{M}}, \text{proc}(z, N^{0,s}[\lfloor m_1 - 2 \rfloor / \beta]) & \text{if } m_0 = 0 \wedge m_1 > 1 \\ \{0, 1\}_a^{\mathcal{M}}, \{1, 0\}_b^{\mathcal{M}}, \text{proc}(z, N^{1,1}) & \text{if } m_0 = m_1 = 1 \\ \{m_0 - 1, 1\}_a^{\mathcal{M}}, \{m_0, 0\}_b^{\mathcal{M}}, \text{proc}(z, N^{s,1}[\lfloor m_0 - 2 \rfloor / \alpha]) & \text{if } m_0 > 1 \wedge m_1 = 1 \\ \{0, m_1\}_a^{\mathcal{M}}, \{1, m_1 - 1\}_b^{\mathcal{M}}, \text{proc}(z, N^{1,s}[\lfloor m_1 - 2 \rfloor / \beta]) & \text{if } m_0 = 1 \wedge m_1 > 1 \\ \{m_0 - 1, m_1\}_a^{\mathcal{M}}, \{m_0, m_1 - 1\}_b^{\mathcal{M}}, \text{proc}(z, N^{s,s}[\lfloor m_0 - 2 \rfloor / \alpha, \lfloor m_1 - 2 \rfloor / \beta]) & \text{if } m_0 > 1 \wedge m_1 > 1 \end{cases} \quad (4.4)$$

With the set of process definitions $\Sigma_{\mathcal{M}}$:

$$\begin{aligned} \cdot; \cdot & \vdash t^{0,0}[] & = N^{0,0} :: (z : T^{0,0}[]) \in \Sigma_{\mathcal{M}} \\ \cdot; z : T^{0,0}[] & \vdash t^{1,0}[] & = N^{1,0} :: (z : T^{s,0}[0[]]) \in \Sigma_{\mathcal{M}} \end{aligned}$$

$$\begin{array}{lll}
\alpha; a : T^{s,0}[\alpha] & \vdash t^{s,0}[\alpha] & = N^{s,0} :: (z : T^{s,0}[S[\alpha]]) \in \Sigma_{\mathcal{M}} \\
\cdot; z : T^{0,0}[] & \vdash t^{0,1}[] & = N^{0,1} :: (z : T^{0,s}[0[]]) \in \Sigma_{\mathcal{M}} \\
\beta; b : T^{0,s}[\beta] & \vdash t^{0,s}[\beta] & = N^{0,s} :: (z : T^{0,s}[S[\beta]]) \in \Sigma_{\mathcal{M}} \\
\cdot; a : T^{0,s}[0[]], \quad b : T^{s,0}[0[]] & \vdash t^{1,1} & = N^{1,1} :: (z : T^{s,s}[0[], 0[]]) \in \Sigma_{\mathcal{M}} \\
\alpha; a : T^{s,s}[\alpha, 0[]], \quad b : T^{s,0}[S[\alpha]] & \vdash t^{s,1}[\alpha] & = N^{s,1} :: (z : T^{s,s}[S[\alpha], 0[]]) \in \Sigma_{\mathcal{M}} \\
\beta; a : T^{0,s}[\beta], \quad b : T^{s,s}[0[], S[\beta]] & \vdash t^{1,s}[\beta] & = N^{1,s} :: (z : T^{s,s}[0[], S[\beta]]) \in \Sigma_{\mathcal{M}} \\
\alpha, \beta; a : T^{s,s}[\alpha, S[\beta]], \quad b : T^{s,s}[S[\alpha], \beta] & \vdash t^{s,s}[\alpha, \beta] & = N^{s,s} :: (z : T^{s,s}[S[\alpha], S[\beta]]) \in \Sigma_{\mathcal{M}}
\end{array}$$

With the definitions for $N^{a,b}$ with $a, b \in \{0, 1, s\}$ given in Figures 4.4 and 4.5.

The instruction process will interact with the tree by either expanding the tree or by instantiating the next instruction (recall that duplication and termination are only used internally). Instantiation is straightforward by eliminating a branch by terminating the processes, and the root process becomes the next instruction. On the other hand, there is the expansion of the tree when a register is incremented.

To illustrate expansion, Figure 4.6 shows such an increment operation, for the specific situation in which the first register holds 2 and the second register (which is to be incremented) holds 0. Each process is colour-coded to highlight the lifecycle; for example, the root process is marked in blue, while red and green are processes that are duplicated and expanded, while white are the new leaves that are produced due to the expansion of the branches.

When incrementing the second register, the tree needs to become aware of the two ways the instruction can decrement the registers; in the initial state, there was only one way of decrementing (the first register). The root process duplicates the process below it, which in turn does the same until a leaf is reached. The duplicated processes correspond to processes that are missing one increment for both registers. Using this, the root process can then create the necessary branches by expanding one copy to an increment on the first register and one for the second register. Using this it has the necessary condition to instantiate the expanded state of the tree. The expansion of the branches works essentially the same as the root node when expanding the second register. For the expansion of the first register, no branching occurs; instead, the expansion is sent to the leaf, which instantiates a new leaf and the increment, which allows the top process in the branch to become the incremented state.

The branching example also shows the preservation of the dependencies of the processes. Before the expansion, the process from root to leaf is implemented as $N^{s,0}$, $N^{1,0}$ and $N^{0,0}$ respectively (Definition 4.7), where each process depends on the process to its right if any. After expansion the root node (blue) becomes $N^{s,1}$. The inner node (red) is duplicated and expanded for each register, producing the processes $N^{1,1}$ and $N^{s,0}$. Next, the leaves (green) get duplicated twice; once when the root node duplicates its branch, and once when the inner nodes duplicate. Each duplicate is expanded to create an instance of $N^{0,1}$ as a child of the node $N^{1,1}$ and two instances of $N^{1,0}$ for the instances $N^{1,1}$ and $N^{0,s}$. Finally, with the expansion of the leaf nodes, each of these nodes produces an instance of $N^{0,0}$ to preserve channels in the configuration of the encoding for the tree process, only exposing the root node. The process configurations before and after the expansion are in alignment with the relations given in Figure 4.3.

$$\begin{aligned}
N^{0,0} &= \text{case } z \text{ (exp}_0 \Rightarrow c \leftarrow t^{0,0}[]; z \leftarrow t^{1,0}[] c \\
&\quad \text{exp}_1 \Rightarrow c \leftarrow t^{0,0}[]; z \leftarrow t^{0,1}[] c \\
&\quad \text{dup} \Rightarrow c \leftarrow t^{0,0}[]; \text{send } z \text{ } c; z \leftarrow t^{0,0}[] \\
&\quad \text{ter} \Rightarrow \text{close } z) \\
N^{1,0} &= \text{case } z \text{ (ins}_0 \Rightarrow r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{0,0}[] r_0 \text{ } r_1 \text{ } a \text{ } y)_{i \in LM} \\
&\quad \text{exp}_0 \Rightarrow a.\text{exp}_0; z \leftarrow t^{s,0}[0] a \\
&\quad \text{exp}_1 \Rightarrow a.\text{dup}; b \leftarrow \text{recv } a; a.\text{exp}_1; b.\text{exp}_0; z \leftarrow t^{1,1}[] a \text{ } b \\
&\quad \text{dup} \Rightarrow a.\text{dup}; a' \leftarrow \text{recv } a; c \leftarrow t^{1,0}[] a; \text{send } z \text{ } c; z \leftarrow t^{1,0}[] a' \\
&\quad \text{ter} \Rightarrow a.1; \text{wait } a; \text{close } z) \\
N^{s,0} &= \text{case } z \text{ (ins}_0 \Rightarrow r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] r_0 \text{ } r_1 \text{ } a \text{ } y)_{i \in LM} \\
&\quad \text{exp}_0 \Rightarrow a.\text{exp}_0; z \leftarrow t^{s,0}[S[\alpha]] a \\
&\quad \text{exp}_1 \Rightarrow a.\text{dup}; b \leftarrow \text{recv } a; a.\text{exp}_1; b.\text{exp}_0; z \leftarrow t^{s,1}[\alpha] a \text{ } b \\
&\quad \text{dup} \Rightarrow a.\text{dup}; b \leftarrow \text{recv } a; c \leftarrow t^{s,0}[\alpha] a; \text{send } z \text{ } c; z \leftarrow t^{s,0}[\alpha] b \\
&\quad \text{ter} \Rightarrow a.\text{ter}; \text{wait } a; \text{close } z) \\
N^{0,1} &= \text{case } z \text{ (ins}_1 \Rightarrow r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{0,0}[] r_0 \text{ } r_1 \text{ } b \text{ } y)_{i \in LM} \\
&\quad \text{exp}_0 \Rightarrow b.\text{dup}; a \leftarrow \text{recv } b; a.\text{exp}_1; b.\text{exp}_0; z \leftarrow t^{1,1}[] a \text{ } b \\
&\quad \text{exp}_1 \Rightarrow b.\text{exp}_1; z \leftarrow t^{0,s}[0] b \\
&\quad \text{dup} \Rightarrow a.\text{dup}; a' \leftarrow \text{recv } a; c \leftarrow t^{0,1}[] a; \text{send } z \text{ } c; z \leftarrow t^{0,1}[] a' \\
&\quad \text{ter} \Rightarrow b.1; \text{wait } b; \text{close } z) \\
N^{0,s} &= \text{case } z \text{ (ins}_1 \Rightarrow r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] r_0 \text{ } r_1 \text{ } a \text{ } y)_{i \in LM} \\
&\quad \text{exp}_0 \Rightarrow b.\text{dup}; a \leftarrow \text{recv } b; b.\text{exp}_1; a.\text{exp}_0; z \leftarrow t^{1,s}[\beta] b \text{ } a \\
&\quad \text{exp}_1 \Rightarrow b.\text{exp}_1; z \leftarrow t^{0,s}[S[\beta]] a \\
&\quad \text{dup} \Rightarrow b.\text{dup}; a \leftarrow \text{recv } b; c \leftarrow t^{0,s}[\beta] b; \text{send } z \text{ } c; z \leftarrow t^{0,s}[\beta] a \\
&\quad \text{ter} \Rightarrow b.\text{ter}; \text{wait } b; \text{close } z)
\end{aligned}$$

Figure 4.4: Processes $N^{a,b}$ in Definition 4.7 (Part 1)

We want to formally verify that the encoding of the instantiation tree can be typed in ASP.

Lemma 4.3. *Given any $\{m_0, m_1\}_z^{\mathcal{M}}$ then, from Definition 4.7 and using Definition 4.5, we have*

$$\cdot \models_{\Sigma_{\mathcal{M}}} \{m_0, m_1\}_z^{\mathcal{M}} :: (z : \mathcal{T}^{\mathcal{M}}(m_0, m_1))$$

Proof. Appendix B.3 gives type derivations for the processes of the encoding.

$$\begin{aligned}
N^{1,1} &= \text{case } z \text{ (ins}_0 \Rightarrow b.\text{ter}; \text{wait } b; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{0,s}[\alpha] \text{ } r_0 \text{ } r_1 \text{ } a \text{ } y)_{i \in LM} \\
&\quad \text{ins}_1 \Rightarrow a.\text{ter}; \text{wait } a; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] \text{ } r_0 \text{ } r_1 \text{ } b \text{ } y)_{i \in LM} \\
&\quad \text{exp}_0 \Rightarrow a.\text{exp}_0; b.\text{exp}_0; z \leftarrow t^{s,1}[0[]] \text{ } a \text{ } b \\
&\quad \text{exp}_1 \Rightarrow a.\text{exp}_1; b.\text{exp}_1; z \leftarrow t^{1,s}[0[]] \text{ } a \text{ } b \\
&\quad \text{ter} \Rightarrow a.\text{ter}; b.\text{ter}; \text{wait } a; \text{wait } b; \text{close } z) \\
N^{s,1} &= \text{case } z \text{ (ins}_0 \Rightarrow b.\text{ter}; \text{wait } b; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{s,s}[\alpha] \text{ } r_0 \text{ } r_1 \text{ } a \text{ } y)_{i \in LM} \\
&\quad \text{ins}_1 \Rightarrow a.\text{ter}; \text{wait } a; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] \text{ } r_0 \text{ } r_1 \text{ } b \text{ } y)_{i \in LM} \\
&\quad \text{exp}_0 \Rightarrow a.\text{exp}_0; b.\text{exp}_0; z \leftarrow t^{s,1}[S[\alpha]] \text{ } a \text{ } b \\
&\quad \text{exp}_1 \Rightarrow a.\text{exp}_1; b.\text{exp}_1; z \leftarrow t^{s,s}[\alpha, 0[]] \text{ } a \text{ } b \\
&\quad \text{ter} \Rightarrow a.\text{ter}; b.\text{ter}; \text{wait } a; \text{wait } b; \text{close } z) \\
N^{1,s} &= \text{case } z \text{ (ins}_0 \Rightarrow b.\text{ter}; \text{wait } b; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{0,s}[\beta] \text{ } r_0 \text{ } r_1 \text{ } a \text{ } y)_{i \in LM} \\
&\quad \text{ins}_1 \Rightarrow a.\text{ter}; \text{wait } a; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{s,s}[\beta] \text{ } r_0 \text{ } r_1 \text{ } b \text{ } y)_{i \in LM} \\
&\quad \text{exp}_0 \Rightarrow a.\text{exp}_0; b.\text{exp}_0; z \leftarrow t^{s,s}[0[], \beta] \text{ } a \text{ } b \\
&\quad \text{exp}_1 \Rightarrow a.\text{exp}_1; b.\text{exp}_1; z \leftarrow t^{1,s}[S[\beta]] \text{ } a \text{ } b \\
&\quad \text{ter} \Rightarrow a.\text{ter}; b.\text{ter}; \text{wait } a; \text{wait } b; \text{close } z) \\
N^{s,s} &= \text{case } z \text{ (ins}_0 \Rightarrow b.\text{ter}; \text{wait } b; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{s,s}[\alpha, \beta] \text{ } r_0 \text{ } r_1 \text{ } a \text{ } y)_{i \in LM} \\
&\quad \text{ins}_1 \Rightarrow a.\text{ter}; \text{wait } a; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \\
&\quad \text{case } z \text{ (i} \Rightarrow z \leftarrow p_i^{s,s}[\alpha, \beta] \text{ } r_0 \text{ } r_1 \text{ } b \text{ } y)_{i \in LM} \\
&\quad \text{exp}_0 \Rightarrow a.\text{exp}_0; b.\text{exp}_0; z \leftarrow t^{s,s}[S[\alpha], \beta] \text{ } a \text{ } b \\
&\quad \text{exp}_1 \Rightarrow a.\text{exp}_1; b.\text{exp}_1; z \leftarrow t^{s,s}[\alpha, S[\beta]] \text{ } a \text{ } b \\
&\quad \text{ter} \Rightarrow a.\text{ter}; b.\text{ter}; \text{wait } a; \text{wait } b; \text{close } z)
\end{aligned}$$

Figure 4.5: Processes $N^{a,b}$ in Definition 4.7 (Part 2)

□

4.4 Encoding of the configuration

Using the definitions, we can redefine a new encoding around the new protocols.

Definition 4.8 (Encoding). For any configuration of a Minsky machine $(i, m_0, m_1)^{\mathcal{M}}$,

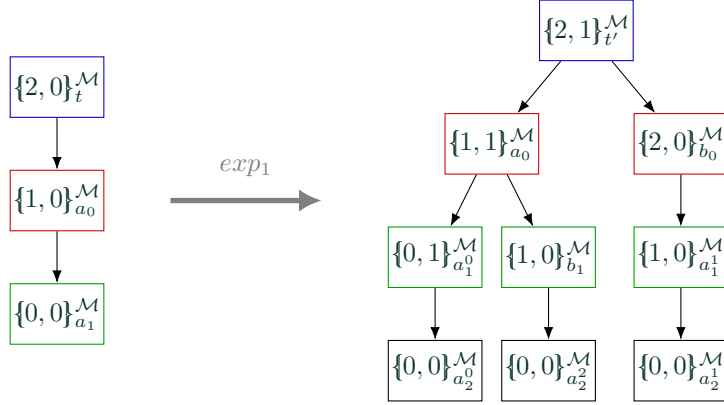


Figure 4.6: Instantiation tree: Example of expansion/branching in the process when incrementing.

a mapping is defined to an ASP process configuration.

$$\llbracket (\cdot) \rrbracket^{\mathcal{M}} : \mathbb{Z}^+ \times \mathbb{N} \times \mathbb{N} \mapsto \mathcal{S} \quad (4.5)$$

Using Definitions 4.3, 4.6 and 4.7, the concrete encoding is defined as:

$$\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} = \llbracket m_0 \rrbracket_{r_0}, \llbracket m_1 \rrbracket_{r_1}, \{m_0, m_1\}_t^{\mathcal{M}}, \llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}} \quad (4.6)$$

We want to formally verify that the encoding can be typed in ASP.

Theorem 4.1 (Typable). *Given any $(i, m_0, m_1)^{\mathcal{M}}$ then $y : \mathbf{1} \models_{\Sigma, \mathcal{M}} \llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} :: (x : \mathbf{1})$ from Definition 4.8.*

Proof. Appendix B gives type derivations of the processes in the encoding. $\square$

4.4.1 Process Interactions of the Encoding

To understand the encoding better, it is important to describe the interactions of the processes as a whole. We illustrate the interaction through sequence diagrams given in Figures 4.7 to 4.9. The diagram includes the register process that is manipulated, the instruction process, and the root process of the tree used to maintain the decrement states. The interaction between only the register and instruction processes is not informative, simply displaying the sequence of reductions of a two-counter machine. It is when the tree process is included that the internal actions to maintain the state become visible and show how the instruction process is able to perform the branching correctly.

Incrementing

Figure 4.7 shows the sequence of interactions of an instruction $i : \text{INC}(0)$. In this case, we do not have to consider the values for the registers as the interactions used are always available to the instruction process.

The first interaction comes from the instruction process (cf. Definition 4.6) as the other processes in the configuration are poised. It sends the **incr** label to the first register. Recall the asynchronous nature of ASP, which means that

the label is not directly consumed by the register process, but rather each arrow containing a label or channel is indicative of a message being sent. When the message is sent, the configuration allows two lines of reduction, on the:

- Register process: the consumption of the **incr** message;
- Instruction process: the interaction with the tree process to start synchronising the state by expanding it.

On the register process, the message allows the register process to consume the **incr** label and continue to reduce to the next state $m_0 + 1$ of the encoding for the register. The instruction process can reduce where a message is sent to the tree process with label **exp₀** and afterwards continue to reduce further to the next state $i + 1$ of the instruction process. The interaction with the tree process allows the tree process to reduce to its next state. Do note that where the instruction and register processes are composed of a singular process and where they have one linear path of reductions, the tree process is a composition of processes where the reduction is composed of internal message passing, allowing potentially for multiple concurrent lines of reductions. Whenever all reductions have been applied to the tree process, it will match the next state.

As ASP is confluent, the order of reductions does not alter the result. However, the asynchronous nature does allow one process, in particular the instruction process, to move ahead without having to wait for the other processes to consume the messages generated. This also holds for any intermediate states of the processes, which are not highlighted in the sequence diagram. For example, the state of the register after consuming the label, which needs an additional reduction to match the definition of the encoding.

Each encoding in the sequence diagram relates to one of the instances of the processes defined in Definitions 4.3, 4.6 and 4.7. The register process starts out as an instance of either R^0 or R^s , depending on whether m_0 equals or is greater than zero, respectively. This process is poised until it receives the label and allows it to reduce to the poised instance of R^s without requiring additional messages. The instruction process starts as an instance of $P_i^{a,b}$ with a and b depending on the state of the registers. Incrementing the state of the registers is not important, as incrementing only differs on the level of types. After incrementing and expanding the tree, the instruction process becomes $P_{i+1}^{s,b}$ as it guarantees that the state of the first register is greater than zero, with the preservation of b . Similarly, the tree process starts as either an instance of $N^{a,b}$ with a and b depending on the state of the registers. It is poised until it receives the expansion message, which allows it to reduce to an instance of $N^{a',b}$ where a' depends on the state of a . When $a = 0$ then $a' = 1$, otherwise $a' = s$.

Decrementing

In Figure 4.8, the sequence of interactions of a $i : \text{DECJ}(0, k)$ instruction is shown, with the condition that $m_0 > 0$.

The start of the sequence is the same as that for an increment instruction. The first reduction comes from the instruction process in the form of a message with label **decr** to the register. In much the same way, two lines of reductions are possible after: the register consuming the message and the instruction interacting with the tree. While the register reduction works much in the same

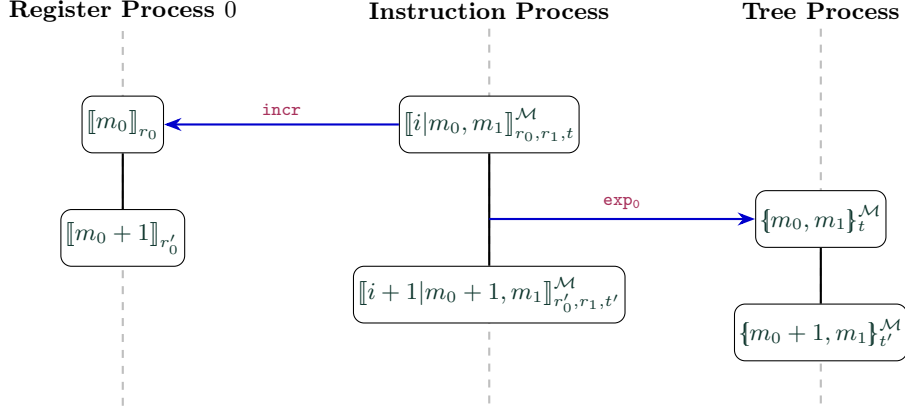


Figure 4.7: Sequence diagram of an instruction $i : \text{INC}(0)$ on register 0 with some value m_0 .

way as with incrementing, the difference lies in the interaction between the instruction and tree processes.

The instruction is aware of the current state of the register, but not the decremented state. It relies on the tree process to instantiate the next instruction as it stores the relation between the current state and the possible decremented states. It does this by first sending the label ins_0 to start the procedure to create the next instruction. This is followed by sending the register channels and the external channel y used for simulating termination, followed by the label $(i + 1)$ as the tree is not aware of the current instruction and cannot know what instruction comes next. The instruction process then has to wait for the tree process to consume the messages.

The tree process, in turn, can consume the messages generated by the instruction process as they come in. When the message for the next instruction is consumed, it can instantiate the next state $i + 1$ of the instruction process that can then be forwarded to take the place of the current instruction process, retaining the intuitionistic channel x . Internally, the tree process might terminate a branch if both registers hold values greater than zero; the remaining branch after forwarding becomes the new root of the tree, being in sync with the values of the registers.

This process relies on the matching of the generic type parameter α to itself. While the type parameter is generic, type parameters are instantiated with the values of the registers, with the values between the registers and the tree kept in sync and thus matching. Furthermore, as the tree process can receive multiple messages from the instruction process, recall that each reduction renames the channel to preserve the order in which a process can consume messages, where messages carry the continuation channel.

Again, we can relate the encoded states in the diagram to instances of process, similar to the incrementing case. Here, the register process starts as R^s as $m_0 > 0$, and after receiving the decrement message reduces to become an instance of R^0 or R^s , depending on whether m_0 becomes zero or not, respectively. The instruction process behaves much in the same way as when incrementing; it starts as an instance of $P_i^{s,b}$ and becomes $P_{i+1}^{0,b}$ or $P_{i+1}^{s,b}$ depending on whether

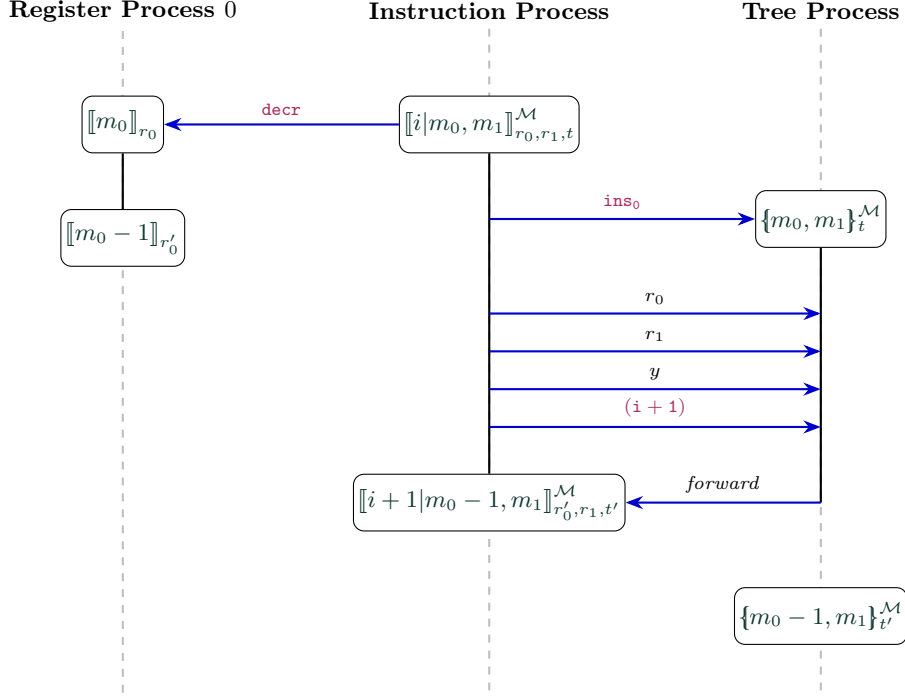


Figure 4.8: Sequence diagram of an instruction $i : \text{DECJ}(0, k)$ on register 0 with value $m_0 > 0$.

m_0 becomes zero or not respectively, with b corresponding to the state of the second register. The tree process starts out as an instance of either $N^{1,b}$ or $N^{s,b}$, depending on whether m_0 is one or is greater. In both cases, the process doesn't continue to be a tree process, but rather serves as an instantiation of the next instruction $P_{i+1}^{0,b}$ or $P_{i+1}^{s,b}$, which is forwarded to take the place of the instruction process. In either case, whether $N^{1,b}$ or $N^{s,b}$, a child process of the root is promoted as the new root node and takes the place of the forwarded process, which is either an instance of $N^{0,b}$ in case of $N^{1,b}$ or an instance of $N^{1,b}$ or $N^{s,b}$ depending on whether $m_0 = 2$ or was greater.

Jumping

In Figure 4.9 the sequence of interactions of a $i : \text{DECJ}(0, k)$ instruction is shown, with the condition that $m_0 = 0$.

Again, the start of the sequence is the same as that for an increment instruction. However, as the state of the register doesn't change, the tree does not have to be updated. Only the register and instruction processes have to move onto the next state, with only the instruction process changing its state to k .

We relate the encoded states in the diagram to an instance of the process, similar to other cases. The register process starts as an instance of R^0 and, after receiving the jump message, remains an instance of the same state. The instruction process starts out as $P_i^{0,b}$ and reduces to become $P_k^{0,b}$, preserving

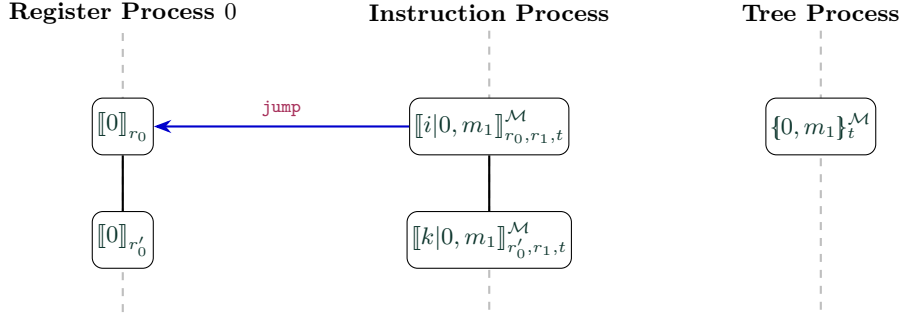


Figure 4.9: Sequence diagram of an instruction $i : \text{DECJ}(0, k)$ on register 0 with value 0.

the state of both registers, where b is the state of the second register. As no interaction happens with the tree, the instance of $N^{0,b}$ remains unchanged.

4.4.2 Examples

Using the example given in Section 2.2.3, we can further demonstrate the working of the encoding. Recall the definition of the machine $\mathcal{M}$ given as

$$1 : \text{INC}(1), 2 : \text{INC}(1), 3 : \text{INC}(0), 4 : \text{DECJ}(1, 3), 5 : \text{DECJ}(1, 1)$$

Termination

The first example starts at state $(3, 0, 1)$ and demonstrates termination and different ways of branching of the machine. The encoding of this state is given as

$$[(3, 0, 1)]^{\mathcal{M}} = [0]_{r_0}, [1]_{r_1}, \{0, 1\}_t^{\mathcal{M}}, [3|0, 1]_{r_0, r_1, t}^{\mathcal{M}}$$

Each component in the encoding corresponds to a process:

1. $[0]_{r_0}$ is an instance of R^0 ;
2. $[1]_{r_1}$ is an instance of R^s that depends on $[0]_p$ which is an instance of R^0 ;
3. $\{0, 1\}_t^{\mathcal{M}}$ is an instance of $N^{0,1}$ that depends on $\{0, 0\}_b^{\mathcal{M}}$ which is an instance of $N^{0,0}$;
4. $[3|0, 1]_{r_0, r_1, t}^{\mathcal{M}}$ is an instance of $P_3^{0,s}$ that depends on the processes listed above and the external channel y .

The instance of the instruction process is the only non-poised process in the configuration and reduces to $[4|1, 1]_{r'_0, r_1, t'}^{\mathcal{M}}$, enabling the reductions of the first register process, the instantiation process, and the instruction process itself. The reductions of the register and the instantiation process move the configuration to the next state of the encoding $[(4, 1, 1)]^{\mathcal{M}}$. However, choosing the reduction of the instruction process would start to let it make progress to the next state, mainly $[(5, 1, 0)]^{\mathcal{M}}$, and skipping the preceding state entirely. In ASP, we are free to choose any order of reduction (Theorem 2.3) and, at any

point, the configuration can converge to a state that matches the encoding. Choosing the reduction of the instruction process, it is possible to demonstrate the dependency it has on the tree process when decrementing. The instruction process will not be able to reduce to the next state of its encoding $\llbracket 5|1, 0 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$, but instead will get stuck at a forward action until the instantiation process has reduced, as it instantiates the process that becomes the instruction after renaming. This condition of synchronising the instruction and instantiation process is only required when decrementing, as the instruction process can instantiate the next instruction when incrementing and jumping, being able to fulfil the dependencies directly. For the register, no such condition exists; they only receive a label and move to their next state.

The instruction process then becomes $\llbracket 5|1, 0 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$, with at least the tree instruction process being $\{1, 0\}_t^{\mathcal{M}}$. The instruction 5 is an instance of $P_5^{s,0}$, which tries to decrement the second register, which it knows to be 0. Hence, it just progresses to $\llbracket 1|1, 0 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$ leaving the instantiation process untouched. The process for the second register is already at the state $\llbracket 0 \rrbracket_{r_1'}$; however, the configuration does not quite match the encoding $\llbracket (1, 1, 0) \rrbracket^{\mathcal{M}}$, as the register is composed against a jump message and not the instruction process. The register process can reduce, consuming the message and becoming $\llbracket 0 \rrbracket_{r_1''}$ such that the encoding of the state is able to match.

Further, increments allow the instruction process to $\llbracket 4, 2, 2 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$ and get ahead of the register and instruction process. The next two reductions depend on the tree process to catch up before it is able to perform further decrements. During the last decrement we see how the encoding deals with branching, before instruction 5 was an instance of $P_5^{s,0}$, however now it is an instance of $P_5^{s,s}$ hence the instruction knows that it deals with an successor and decrements instead of jumps using the instantiation process to move to the next state. When the instruction becomes $\llbracket 6|2, 0 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$, the process becomes poised and is not able to progress further as it waits on channel y to close, which is external, hence no message will ever be produced. Any reduction along the register and instantiation processes that is still possible can be performed, reaching the next final state $\llbracket (6, 2, 0) \rrbracket^{\mathcal{M}}$. This state is poised as the register and instantiation processes are poised awaiting a label to be produced by the instantiation process, which is poised itself, halting all progress.

While in this example, we consistently prioritised the instruction process when reducing, it is entirely possible to choose a different order such that every intermediate state in the original machine is passed in the encoding instead of being skipped.

Non-termination

For non-termination, we only have to consider some state $\llbracket (3, n, 0) \rrbracket^{\mathcal{M}}$ and see that it returns to the same instruction with an updated state for the first register. The encoding of this state is given as

$$\llbracket (3, n, 0) \rrbracket^{\mathcal{M}} = \llbracket n \rrbracket_{r_0}, \llbracket 1 \rrbracket_{r_1}, \{n, 0\}_t^{\mathcal{M}}, \llbracket 3|n, 0 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$$

Each component in the encoding corresponds to a process:

1. $\llbracket n \rrbracket_{r_0}$ is an instance of R^0 or R^s depending whether $n = 0$ or $n > 0$ respectively;

2. $\llbracket 0 \rrbracket_{r_1}$ is an instance of R^0 ;
3. $\{n, 0\}_t^{\mathcal{M}}$ is an instance of $N^{0,0}$, $N^{1,0}$, or $N^{s,0}$ depending whether $n = 0$, $n = 1$, or $n > 1$ respectively;
4. $\llbracket 3|n, 0 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$ is an instance of $P_3^{0,0}$ or $P_3^{s,0}$ depending on whether $n = 0$ or $n > 0$.

What is notable here is that the instruction process is differentiated on the state of the first register; however, it does not affect the action it performs; it simply increments. Furthermore, we will see that the instantiation process serves no purpose here but to mirror the state of the registers, as the instruction process only tries to decrement a register that it knows is zero, thus jumping.

Like the termination example, the only process that can reduce is the instruction process, producing reductions for the register, tree, and instruction processes. The instruction process reduces to $\llbracket 4|n+1, 0 \rrbracket_{r'_0, r_1, t'}^{\mathcal{M}}$ which always corresponds to an instance of $P_4^{s,0}$, where instruction 4 is a decrement on the second register. Hence, the instruction process can simply reduce to $\llbracket 3|n+1, 0 \rrbracket_{r'_0, r'_1, t''}^{\mathcal{M}}$ preserving the state of the registers. As an arbitrary n was chosen, it shows that we continue to perform reductions without waiting for the register and the instantiation processes. At any point, it is possible to reduce the register and tree processes such that the configuration matches the encoding of a subsequent state. For example, where we have $\llbracket 3|n+m, 0 \rrbracket_{r'_0, r'_1, t'}^{\mathcal{M}}$ and the other processes have been ignored when reducing:

1. The first register can reduce by consuming m messages with an increment label reducing it to $\llbracket n+m \rrbracket_{r'_0}$;
2. The second register can reduce by consuming m messages with a decrement label, reducing it to $\llbracket 0 \rrbracket_{r'_0}$, where the state remains unchanged, but the configuration can match the encoding of the state as the process is composed against the instruction process;
3. The tree can reduce by consuming m messages with an expansion label for the first register, reducing it to $\{n+m, 0\}_{t'}^{\mathcal{M}}$.

The combined result is a process configuration that matches the encoding for $\llbracket (3, n+m, 0) \rrbracket^{\mathcal{M}}$, which means the machine has performed m loops and can continue to progress further. Like with the termination example, a different order can be chosen such that every intermediate state of the machine is passed in the reduction of the encoding instead of being skipped.

4.4.3 Correctness

Unlike the encoding given in the previous chapter, the encoding using polymorphic types is much stronger in preserving the semantics of the Minsky machine. With the encoding of the register values as types, it generates a protocol that guarantees the exact semantics of a Minsky machine on a register whenever the instruction processes follow the definition of the machine.

By Definition 4.6 we know that the instruction do follow the definition of the machine. It follows the definition either directly, through instantiation of the next instruction process, or indirectly, through sending a label to the tree

instantiation process, and moves to the next state of the encoding. This is further highlighted in sequence diagrams given in Figures 4.7 to 4.9.

With the preservation of instructions in the encoding, the protocol defined in Definition 4.2 ensures that the two register processes correspond to the correct value after receiving the label from the instruction. This comes with the precondition of the property proven in Theorem 4.1, as otherwise it is not guaranteed that progress or the following of the protocol occurs. The interaction between instruction and register is simply an explicit statement which highlights the instruction from the underlying machine; it forms the basis of the guarantee of correctness while not actually introducing new behaviour, as the instruction process has to be state-aware using some other construct, mainly the instantiation tree.

The instantiation tree forms the mechanism to capture the state of the register, such that the instruction process can move to the next state whenever it decrements. The correctness comes from the proof given in Theorem 4.1. The theorem ensures that the states of the register and tree actually match. By this result, whenever a decrement occurs, the next instruction can be instantiated, matching the states of the registers. As there are no external channels, besides y , progress is guaranteed. Progress only halts once we reach a wait action on y , which only occurs in the encoding when the machine reaches $i > n$, which corresponds to the termination of the machine.

This combined ensures that we can state that the encoding is sound.

Theorem 4.2 (Soundness). *For some Minsky machine $\mathcal{M}$, using Definition 4.8: if $(i, m_0, m_1) \rightarrow (i', m'_0, m'_1)$, then $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \llbracket (i', m'_0, m'_1) \rrbracket^{\mathcal{M}}$.*

The proof can be illustrated by means of the sequence diagrams given in Figures 4.7 to 4.9. The configuration for each sequence diagram starts matching the encoding of a machine with state (i, m_0, m_1) . Depending on the state of the registers and the instruction, the machine state reduces to (i', m'_0, m'_1) . If it is possible to show a reduction path in the encoding leading to the encoded state, we have shown soundness. Similar to Theorem 3.2, there are 3 cases based on the reductions possible in the Minsky machine for some state: incrementing, decrementing and jumping.

For the case of incrementing, the machine's state reduces to $(i + 1, m_0 + 1, m_1)$. Using Figure 4.7, the instruction process reduces first and is able to reduce straight to the next state of its encoding. The messages generated by the instruction process allow the register and tree processes to reduce to their respective next states, matching the encoding of $(i + 1, m_0 + 1, m_1)$, showing soundness for increments.

For the case of decrementing, the machine's state reduces to $(i + 1, m_0 - 1, m_1)$, where $m_0 > 0$. Similar to the incrementing case, using Figure 4.8, the instruction process reduces first, but it does not reduce immediately to the next encoded state; instead waiting to forward the reduced tree process. The register process and tree process are able to reduce using the messages generated from the instruction process. Where the register process moves to its next state in the encoding, the tree process becomes the next encoded state of the instruction once forwarded, promoting a predecessor process containing the decremented tree as the new tree process. This then matches the next encoding of $(i + 1, m_0 - 1, m_1)$ showing soundness for decrements.

Finally, for the case of jumping, the machine's state reduces to $(k, 0, m_1)$. It follows the same steps as the incrementing case, without reductions on the tree process, as the state of the registers does not change. This proves soundness for jumps.

For completeness, similar to the completeness proven in Theorem 3.3, we have to consider the asynchronous nature of ASP. As ASP is confluent (Theorem 2.3), it means that some process might get ahead in the encoding. However, as we have already stated the property of soundness, completeness follows as whenever all messages in a configuration are consumed, and the processes are instantiated, required to reach the next state of the encoding, and completeness follows. Furthermore, there are no additional reductions possible, in particular when $i > n$, as waiting on the channel y causes the instruction process, and as a consequence, the whole configuration, to become poised.

Theorem 4.3 (Completeness). *If $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} \rightarrow^+ \mathcal{S}$ where $\mathcal{S}$ is a process configuration in ASP, then there exists (i', m'_0, m'_1) such that $\mathcal{S} \rightarrow^+ \llbracket (i', m'_0, m'_1) \rrbracket^{\mathcal{M}}$ and $(i, m_0, m_1) \rightarrow^+ (i', m'_0, m'_1)$ using Definition 4.8.*

Chapter 5

Discussion

The encodings of nested session types presented show that session types do not necessarily restrict Turing completeness of a programming language (Chapter 3). Furthermore, it was shown that nested session types, while not being able to capture two-counter types, make it possible to capture operational correspondence of a Turing-complete program and allow it to continue (Chapter 4).

Nested session types encompass prenex polymorphism, polymorphic type construction, recursively defined types and nested types. In the encoding from Chapter 3, the session types use the recursive type definitions present in the nested session types. The protocol is implemented in a one-counter process that is implemented as a stack. The stack is implemented as a recursive process using two definitions and captures the value at the level of the top process. Turing completeness follows by verifying the soundness and completeness of the encoding, ensuring preservation of the semantics.

While nested session types cannot express two-counter types, they can still play an important role in ensuring correctness in Turing-complete machines. The encoding from Chapter 4 introduced one-counter types to ensure the correctness of the machine, while nested types in combination with processes mirror the counters to enable the machine to progress when decrementing. As established, the nested types in the encoding without one-counter types do lose meaning as the type parameters are only extended and never used, essentially reducing to the first encoding with a more complex composition. Hence, nested types are required to capture the state of two-counter machines.

Notable difference between the two encodings is that the encoding using nested types adds the requirement that the instruction process needs to be aware of the state, and as a consequence, an instruction instantiation process that is structurally equivalent to a decrement tree was introduced. While the data structure could be further optimised both in space and time complexity, the same data structure would also be a requirement in a process calculus that allows for two-counter session types, as it merely consolidates the 2 one-counter types. Structurally, from the perspective of processes, the consolidated register process will follow a similar data structure to that of the instantiation process. While the two-counter type can predict the continuation state, the processes providing the type have as a minimum requirement that it needs to be aware of the transition and state to be able to implement the protocol.

An encoding using nested types and two-counter types both have the same

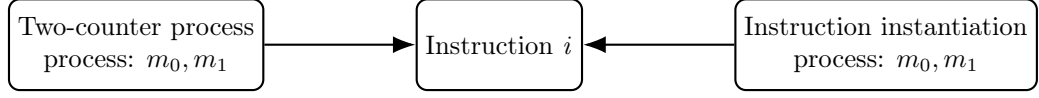


Figure 5.1: Hypothetical structure of the process configuration of the encoded Minsky machine for two-counter types

complexity, as the worst complexity comes from the tree structure. However, the complexity of the register processes alone is better for the encoding using nested types as it is always $O(1)$, while that of the register using a two-counter type is at worst $O(2^{m_0+m_1})$ as it needs to traverse the tree. Furthermore, the two encodings both have the requirement that the instruction process needs to be aware of the state to send the appropriate labels when decrementing, as dictated by the session type of the register.

Hence, nested session types can enable the capturing of the state of a Turing machine in session types and allow the machine to continue. Moreover, where type equivalence in nested session types is decidable, where for two-counter types they are not.

Chapter 6

Conclusion

In this thesis, two arguments for the Turing completeness of the formal language of Nested Session Types were shown. Both arguments use the ability to simulate two-counter Minsky machines in ASP as the premise of Turing completeness, as such a machine is by itself Turing complete.

The first argument encodes a Minsky machine in ASP and proves the correctness of the encoding using only the recursive properties of NSTs. The second argument uses the nested types of NSTs for which a stronger protocol is developed that captures the exact semantics of registers for some state. While this is a more involved argument due to the need for additional processes to enable the simulation compared to the first encoding, it captures the correctness of the states of registers concerning jumping and decrementing, guaranteeing the correctness of transitions of the state of an encoded Minsky machine.

The second argument limits itself to a combination of one-counter and nested types to enable an encoding with sufficient meaning. One-counter types alone are not enough to instantiate the next instruction in the context of ASP, while the nested types alone do not capture the semantics of decrements in a meaningful way. The types complement each other to capture the semantics of a two-counter Minsky machine, restricting the typed programs to give better guarantees for the correctness and allow the program to progress.

Where nested types are allowed, and two-counter types can both enable an encoding of a Minsky machine, nested types are more efficient while also allowing type equivalence.

Bibliography

- [1] P. Wadler, “Propositions as sessions,” in *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP ’12. New York, NY, USA: Association for Computing Machinery, 2012, p. 273–286. [Online]. Available: <https://doi.org/10.1145/2364527.2364568>
- [2] A. Das, H. DeYoung, A. Mordido, and F. Pfenning, “Nested session types,” *ACM Trans. Program. Lang. Syst.*, vol. 44, no. 3, pp. 19:1–19:45, 2022. [Online]. Available: <https://doi.org/10.1145/3539656>
- [3] S. J. Gay, D. Poças, and V. T. Vasconcelos, “The different shades of infinite session types,” in *Foundations of Software Science and Computation Structures - 25th International Conference, FOSSACS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings*, ser. Lecture Notes in Computer Science, P. Bouyer and L. Schröder, Eds., vol. 13242. Springer, 2022, pp. 347–367. [Online]. Available: https://doi.org/10.1007/978-3-030-99253-8_18
- [4] L. Caires and F. Pfenning, “Session types as intuitionistic linear propositions,” in *CONCUR 2010 - Concurrency Theory, 21th International Conference, CONCUR 2010, Paris, France, August 31-September 3, 2010. Proceedings*, ser. Lecture Notes in Computer Science, P. Gastin and F. Laroussinie, Eds., vol. 6269. Springer, 2010, pp. 222–236. [Online]. Available: https://doi.org/10.1007/978-3-642-15375-4_16
- [5] M. L. Minsky, *Computation: finite and infinite machines*. USA: Prentice-Hall, Inc., 1967.
- [6] I. Lanese, J. A. Pérez, D. Sangiorgi, and A. Schmitt, “On the expressiveness and decidability of higher-order process calculi,” *Information and Computation*, vol. 209, no. 2, pp. 198–226, 2011. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0890540110001598>

Appendices

Appendix A

Proof of Theorem 3.1

A.1 Register processes

Proof of Lemma 3.1 using Definition 3.2.

A.1.1 Register encoding

We need to consider the following case for the encoding:

Case 1 (Zero). For $\llbracket m \rrbracket_r$ where $m = 0$:

$$\frac{;\cdot \vdash R^0 :: (r : C[])}{\cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m \rrbracket_r = \text{proc}(r, R^0) :: (r : C[]) } \text{proc}$$

Case 2 (Successor). For $\llbracket m \rrbracket_r$ where $m > 0$:

$$\frac{\cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m-1 \rrbracket_p :: (p : C[]) \quad \frac{;\cdot \vdash R^s :: (r : C[])}{p : C[] \models_{\Sigma_{\mathcal{M}}} \text{proc}(r, R^s) :: (r : C[]) } \text{proc}}{\cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m \rrbracket_r = (\llbracket m-1 \rrbracket_p, \text{proc}(r, R^s)) :: (r : C[]) } \text{comp}$$

A.1.2 Register process definitions

We need to consider the following case for the process definitions:

Case 1 (Zero). For process definition $z[]$:

$$\frac{;\cdot \vdash R^0 :: (r : C[]) }{\cdot \vdash z[] = R^0 :: (r : C[]) \in \Sigma_{\mathcal{M}}}$$

Case 2 (Successor). For process definition $s[]$:

$$\frac{;\cdot \vdash R^s :: (r : C[]) }{p : C[] \vdash s[] = R^s :: (r : C[]) \in \Sigma_{\mathcal{M}}}$$

A.1.3 Constituent register processes

We consider the following cases for the processes used in the encoding and the definition set:

Case 1 (Zero). For R^0 :

$$\frac{\forall l \in L \quad \cdot \vdash R_l^0 :: (r : A_l^0)}{\cdot \vdash R^0 = \text{case } r \text{ (} \mathbf{1} \Rightarrow R_l^0)_{l \in L} :: (r : C[] = \&\{\mathbf{1} : A_l^0\}_{l \in L})} \&R$$

where $L = \{\text{inc}, \text{decj}\}$.

1. For R_{inc}^0 ¹:

$$\frac{\cdot \vdash z[] = R^0 :: (r : C[]) \in \Sigma_{\mathcal{M}} \quad \cdot \vdash \cdot \quad \frac{p : C[] \vdash s[] = R^s :: (r : C[]) \in \Sigma_{\mathcal{M}} \quad \cdot \vdash (c : C[]) \diamond}{\cdot \vdash (c : C[]) \vdash r \leftarrow s[] \quad c :: (r :: C[]) } \text{def}}{\cdot \vdash R_{\text{incr}}^0 = c \leftarrow z[]; r \leftarrow s[] \quad c :: (r : A_{\text{incr}}^0 = C[])} \text{def}$$

2. For R_{decj}^0 :

$$\frac{\cdot \vdash z[] = R^0 :: (r : C[]) \in \Sigma_{\mathcal{M}} \quad \cdot \vdash \cdot}{\cdot \vdash r \leftarrow z[] :: (r : C[]) } \text{def} \quad \frac{\cdot \vdash R_{\text{jump}}^0 = r.\text{jump}; r \leftarrow z[] :: (r : A_{\text{jump}}^0 = \oplus\{\text{jump} : C[], \dots\})}{\cdot \vdash R_{\text{jump}}^0 = r.\text{jump}; r \leftarrow z[] :: (r : A_{\text{jump}}^0 = \oplus\{\text{jump} : C[], \dots\})} \oplus R$$

Case 2 (Successor). For R^s :

$$\frac{\forall l \in L \quad \cdot \vdash p : C[] \vdash R_l^s :: (r : A_l^s)}{\cdot \vdash p : C[] \vdash R^s = \text{case } r \text{ (} \mathbf{1} \Rightarrow R_l^s)_{l \in L} :: (r : C[] = \&\{\mathbf{1} : A_l^s\}_{l \in L})} \&R$$

¹We write type equivalence using a diamond as it follows naturally, as no substitution occurs and any definition is reflexive.

where $L = \{\text{inc}, \text{decj}\}$.

1. with R_{inc}^s :

$$\frac{p : C[] \vdash s[] = R^s :: (r : C[]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash (p : C[]) \diamond \quad \frac{p : C[] \vdash s[] = R^s :: (r : C[]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash (c : C[]) \diamond}{\cdot; c : C[] \vdash r \leftarrow s[] \quad c :: (r :: C[]) } \text{def}}{\cdot; p : C[] \vdash R_{\text{inc}}^s = c \leftarrow s[] \quad p; r \leftarrow s[] \quad c :: (r : A_{\text{inc}}^s = C[]) } \text{def}$$

2. with R_{decj}^s :

$$\frac{\frac{\cdot; \cdot \vdash (C[]) \diamond}{\cdot; p : C[] \vdash r \leftrightarrow p :: (r : C[]) } \text{id}}{\cdot; p : C[] \vdash R_{\text{decj}}^s = r.\text{next}; \quad r \leftrightarrow p :: (r : A_{\text{decj}}^s = \oplus\{\dots, \text{next} : C[]\}) } \oplus R$$

62

A.2 Instruction processes

Proof of Lemma 3.3 using Definition 3.3.

A.2.1 Instruction encoding

For the encoding of instruction $i \in \mathbb{Z}^+$:

$$\frac{\cdot; r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash P_i :: (x : \mathbf{1})}{r_0 : C[], r_1 : C[], y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} = \text{proc}(x, P_i) :: (x : \mathbf{1}) } \text{proc}$$

A.2.2 Instruction process definition

For the process definition of instruction $i \in L_{\mathcal{M}}$:

$$\frac{\cdot; r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash P_i :: (x : \mathbf{1})}{r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash p_i[] = P_i :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}}}$$

A.2.3 Constituent instruction processes

We consider the following constituent process used in the encoding and definition set:

1. Incrementing;
2. Decrementing-jump;
3. Out of bounds.

Case 1 (Incrementing). For P_i where $i : \text{INC}(j)$:

$$\frac{\frac{r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash p_{i+1}[] = P_{i+1} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash (r_0 : C[], r_1 : C[], y : \mathbf{1}) \diamond}{\cdot; r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash x \leftarrow p_{i+1}[] \quad r_0 \ r_1 \ y :: (x : \mathbf{1})} \text{def}}{\cdot; r_j : C[] = \&\{\text{inc} : C[], \dots\}, r_{1-j} : C[], y : \mathbf{1} \vdash P_i = r_j.\text{inc}; \ x \leftarrow p_{i+1}[] \quad r_0 \ r_1 \ y :: (x : \mathbf{1})} \&\text{L}$$

By Definition 2.9, $i + 1$ is reachable, hence p_{i+1} is defined in $\Sigma_{\mathcal{M}}$.

Case 2 (Decrement-jump). For P_i where $i : \text{DECJ}(j, k)$:

$$\frac{\frac{(l \in L) \quad \cdot; r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash x \leftarrow p_{i_l}[] \quad r_0 \ r_1 \ y :: (x : \mathbf{1})}{\cdot; r_j : \oplus\{\text{jump} : C[], \text{next} : C[]\}, r_{1-j} : C[], y : \mathbf{1} \vdash \text{case } r_j \ (\mathbf{1} \Rightarrow x \leftarrow p_{i_l}[] \quad r_0 \ r_1 \ y)_{l \in L} :: (x : \mathbf{1})} \oplus\text{R}}{\cdot; r_j : C[] = \&\{\dots, \text{decj} : \oplus\{\mathbf{1} : C[]\}_{l \in L}\}, r_{1-j} : C[], y : \mathbf{1} \vdash P_i = r_j.\text{decj}; \ \text{case } r_j \ (\mathbf{1} \Rightarrow x \leftarrow p_{i_l}[] \quad r_0 \ r_1 \ y)_{l \in L} :: (x : \mathbf{1})} \&\text{L}$$

where $L = \{\text{jump}, \text{next}\}$, $i_{\text{jump}} = k$ and $i_{\text{next}} = i + 1$.
 with $x \leftarrow p_{i_l} \parallel r_0 \ r_1 \ y$:

$$\frac{r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash p_{i_l} \parallel = P_{i_l} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash (r_0 : C[], r_1 : C[], y : \mathbf{1}) \diamond}{\cdot; r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash x \leftarrow p_{i_l} \parallel r_0 \ r_1 \ y :: (x : \mathbf{1})} \text{def}$$

By Definition 2.9, $i + 1$ and k are reachable, hence p_{i+1} and p_k are defined in $\Sigma_{\mathcal{M}}$.

Case 3 (Out of bounds). For P_i where $i > n$:

$$\frac{\cdot; r_0 : C[], r_1 : C[] \vdash Q_i :: (x : \mathbf{1})}{\cdot; r_0 : C[], r_1 : C[], y : \mathbf{1} \vdash \text{wait } y; Q_i :: (x : \mathbf{1})} \mathbf{1L}$$

A.3 Encoding of configuration

Proof of Theorem 3.1 using Definition 3.4.

For $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}}$:

$$\frac{\cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m_0 \rrbracket_{r_0} :: (r_0 : C[]) \quad \frac{\cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m_1 \rrbracket_{r_1} :: (r_1 : C[]) \quad r_0 : C[], r_1 : C[], y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} :: (x : \mathbf{1})}{r_0 : C[], y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \llbracket m_1 \rrbracket_{r_1}, \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}} :: (x : \mathbf{1})} \text{comp}}{y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} = (\llbracket m_0 \rrbracket_{r_0}, (\llbracket m_1 \rrbracket_{r_1}, \llbracket i \rrbracket_{r_0, r_1}^{\mathcal{M}})) :: (x : \mathbf{1})} \text{comp}$$

Appendix B

Proof of Theorem 4.1

B.1 Register processes

Proof of Lemma 4.1 using Definition 4.3.

B.1.1 Register encoding

We use Definition 4.2:

1. For $\llbracket m \rrbracket_r$ where $m = 0$:

$$\frac{\cdot; \cdot \vdash R^0 :: (r : \langle 0 \rangle = 0 \square)}{\cdot \models_{\Sigma_{\mathcal{M}}} \llbracket 0 \rrbracket_r = \text{proc}(r, R^0) :: (r : \langle 0 \rangle)} \text{proc}$$

2. For $\llbracket m \rrbracket_r$ where $m > 0$:

$$\frac{\cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m-1 \rrbracket_p :: (p : \langle m-1 \rangle) \quad \frac{\cdot; p : \langle m-1 \rangle \vdash R^s[\langle m-1 \rangle / \alpha] :: (r : \langle m \rangle)}{p : \langle m-1 \rangle \models_{\Sigma_{\mathcal{M}}} \text{proc}(r, R^s[\langle m-1 \rangle / \alpha]) :: (r : \langle m \rangle)} \text{proc}}{\cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m \rrbracket_r = (\llbracket m-1 \rrbracket_p, \text{proc}(r, R^s[\langle m-1 \rangle / \alpha])) :: (r : \langle m \rangle)} \text{comp}$$

B.1.2 Register process definitions

1. For process definition $r^0 \square \in \Sigma_{\mathcal{M}}$:

$$\frac{\cdot; \cdot \vdash R^0 :: (r : 0 \square)}{\cdot \vdash r^0 \square = R^0 :: (r : 0 \square) \in \Sigma_{\mathcal{M}}}$$

2. For process definition $r^s \square \in \Sigma_{\mathcal{M}}$:

$$\frac{\alpha; p : \alpha \vdash R^s :: (r : S[\alpha])}{p : \alpha \vdash r^s[\alpha] = R^s :: (r : S[\alpha]) \in \Sigma_{\mathcal{M}}}$$

B.1.3 Constituent register processes

We consider two cases:

1. With process R^0 :

$$\frac{(\forall l \in \{\text{incr}, \text{jump}\}) \quad \cdot; \cdot \vdash R_l^0 :: (r : A_l^0)}{\cdot; \cdot \vdash R^0 = \text{case } r \text{ (incr} \Rightarrow R_{\text{incr}}^0, \text{jump} \Rightarrow R_{\text{jump}}^0) :: (r : 0[])} \&\text{R}$$

with R_{incr}^0 ¹:

$$\frac{\cdot \vdash r^0[] = R^0 :: (r : 0[]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash \cdot \quad \frac{p : \alpha \vdash r^s[\alpha] = R^s :: (r : S[\alpha]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash (c : 0[]) \diamond}{\cdot; c : 0[] \vdash r \leftarrow r^s[0[]] \quad c :: (r : S[0[]])} \text{def}}{\cdot; \cdot \vdash R_{\text{incr}}^0 = c \leftarrow r^0[]; r \leftarrow r^s[0[]] \quad c :: (r : A_{\text{incr}}^0 = S[0[]])} \text{def}$$

with R_{jump}^0 :

$$\frac{\cdot \vdash r^0[] = R^0 :: (r : 0[]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash \cdot}{\cdot; \cdot \vdash r \leftarrow r^0[] :: (r : A_{\text{jump}}^0 = 0[])} \text{def}$$

2. With process R^s :

$$\frac{(\forall l \in \{\text{incr}, \text{decr}\}) \quad \alpha; p : \alpha \vdash R_l^s :: (r : A_l^s)}{\alpha; p : \alpha \vdash R^s = \text{case } r \text{ (incr} \Rightarrow R_{\text{incr}}^s, \text{decr} \Rightarrow R_{\text{decr}}^s) :: (r : S[\alpha])} \&\text{R}$$

¹We write type equivalence using a diamond as it follows naturally, as no substitution occurs and any definition is reflexive.

with R_{incr}^s ²:

$$\frac{p : \alpha \vdash r^s[\alpha] = R^s :: (r : S[\alpha]) \in \Sigma_{\mathcal{M}} \quad \cdot; \alpha \vdash (p : \alpha) \diamond \quad \frac{p : \alpha \vdash r^s[\alpha] = R^s :: (r : S[\alpha]) \in \Sigma_{\mathcal{M}} \quad \cdot; \alpha \vdash (c : S[\alpha]) \diamond}{\cdot; c : S[\alpha] \vdash r \leftarrow r^s[S[\alpha]] \quad c :: (r : S[S[\alpha]])} \text{def}}{\alpha; p : \alpha \vdash R_{\text{incr}}^0 = c \leftarrow r^0[]; r \leftarrow r^s[0[]] \quad c :: (r : A_{\text{incr}}^0 = S[S[\alpha]])} \text{def}$$

with R_{decr}^s :

$$\frac{\cdot; \alpha \vdash (\alpha) \diamond}{\alpha; p : \alpha \vdash R_{\text{decr}}^s = r \leftrightarrow p :: (r : A_{\text{decr}}^s = \alpha)} \text{id}$$

89

B.2 Instruction processes

Proof of Lemma 4.2 using Definition 4.6.

B.2.1 Instruction encoding

Four cases:

1. For $\llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$ with $m_0 = 0$ and $m_1 = 0$:

$$\frac{\cdot; r_0 : 0[], r_1 : 0[], t : T^{0,0}[], y : \mathbf{1} \vdash P_i^{0,0} :: (x : \mathbf{1})}{r_0 : 0[], r_1 : 0[], t : T^{0,0}[], y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \text{proc}(x, P_i^{0,0}) :: (x : \mathbf{1})} \text{proc}$$

2. For $\llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$ with $m_0 > 0$ and $m_1 = 0$:

²We write type equivalence using a diamond as it follows naturally, as only substitution is applied on the same definition, which makes it reflexive.

$$\frac{\cdot; r_0 : S[\langle m_0 - 1 \rangle], r_1 : 0[], t : T^{s,0}[\langle m_0 - 1 \rangle], y : \mathbf{1} \vdash P_i^{s,0}[\langle m_0 - 1 \rangle / \alpha] :: (x : \mathbf{1})}{r_0 : S[\langle m_0 - 1 \rangle], r_1 : 0[], t : T^{s,0}[\langle m_0 - 1 \rangle], y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \text{proc}(x, P_i^{s,0}[\langle m_0 - 1 \rangle / \alpha]) :: (x : \mathbf{1})} \text{proc}$$

3. For $\llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$ with $m_0 = 0$ and $m_1 > 0$:

$$\frac{\cdot; r_0 : 0[], r_1 : S[\langle m_1 - 1 \rangle], t : T^{0,s}[\langle m_1 - 1 \rangle], y : \mathbf{1} \vdash P_i^{0,s}[\langle m_1 - 1 \rangle / \beta] :: (x : \mathbf{1})}{r_0 : 0[], r_1 : S[\langle m_1 - 1 \rangle], t : T^{0,s}[\langle m_1 - 1 \rangle], y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \text{proc}(x, P_i^{0,s}[\langle m_1 - 1 \rangle / \beta]) :: (x : \mathbf{1})} \text{proc}$$

4. For $\llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}$ with $m_0 > 0$ and $m_1 > 0$:

$$\frac{\cdot; r_0 : S[\langle m_0 - 1 \rangle], r_1 : S[\langle m_1 - 1 \rangle], t : T^{s,s}[\langle m_0 - 1 \rangle, \langle m_1 - 1 \rangle], y : \mathbf{1} \vdash P_i^{s,s}[\langle m_0 - 1 \rangle / \alpha, \langle m_1 - 1 \rangle / \beta] :: (x : \mathbf{1})}{r_0 : S[\langle m_0 - 1 \rangle], r_1 : S[\langle m_1 - 1 \rangle], t : T^{s,s}[\langle m_0 - 1 \rangle, \langle m_1 - 1 \rangle], y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \text{proc}(x, P_i^{s,s}[\langle m_0 - 1 \rangle / \alpha, \langle m_1 - 1 \rangle / \beta]) :: (x : \mathbf{1})} \text{proc}$$

B.2.2 Instruction process definitions (Figure 4.2)

1. For $p_{i \in L_{\mathcal{M}}}^{0,0}[] \in \Sigma_{\mathcal{M}}$:

$$\frac{\cdot; r_0 : 0[], r_1 : 0[], t : T^{0,0}[], y : \mathbf{1} \vdash P_i^{0,0} :: (x : \mathbf{1})}{r_0 : 0[], r_1 : 0[], t : T^{0,0}[], y : \mathbf{1} \vdash p_{i \in L_{\mathcal{M}}}^{0,0}[] = P_i^{0,0} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}}}$$

2. For $p_{i \in L_{\mathcal{M}}}^{s,0}[\alpha] \in \Sigma_{\mathcal{M}}$:

$$\frac{\cdot; r_0 : S[\alpha], r_1 : 0[], t : T^{s,0}[\alpha], y : \mathbf{1} \vdash P_i^{s,0} :: (x : \mathbf{1})}{r_0 : S[\alpha], r_1 : 0[], t : T^{s,0}[\alpha], y : \mathbf{1} \vdash p_{i \in L_{\mathcal{M}}}^{s,0}[\alpha] = P_i^{s,0} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}}}$$

3. For $p_{i \in L_{\mathcal{M}}}^{0,s}[\beta] \in \Sigma_{\mathcal{M}}$:

$$\frac{.; r_0 : 0[], r_1 : S[\beta], t : T^{0,s}[], y : \mathbf{1} \vdash P_i^{0,s} :: (x : \mathbf{1})}{r_0 : 0[], r_1 : S[\beta], t : T^{0,s}[\beta], y : \mathbf{1} \vdash p_{i \in L\mathcal{M}}^{0,s}[\beta] = P_i^{0,s} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}}}$$

4. For $p_{i \in L\mathcal{M}}^{s,s}[\alpha, \beta] \in \Sigma_{\mathcal{M}}$:

$$\frac{.; r_0 : S[\alpha], r_1 : S[\beta], t : T^{s,s}[\alpha, \beta], y : \mathbf{1} \vdash P_i^{s,s} :: (x : \mathbf{1})}{r_0 : S[\alpha], r_1 : S[\beta], t : T^{s,s}[\alpha, \beta], y : \mathbf{1} \vdash p_{i \in L\mathcal{M}}^{s,s}[] = P_i^{s,s} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}}}$$

B.2.3 Constituent instruction processes

We only consider the following cases, regardless of state, as they follow the same construction, hence their proofs are analogous:

1. Incrementing;
2. Decrementing (only for states $s, 0$, $0, s$ and s, s);
3. Jump (only for state $0, 0$, $0, s$ and $s, 0$);
4. Out of bounds.

Case 1 (Increment). For $P_i^{0,0}$ with $i : \text{INC}(i)$. We show for $i = 0$

$$\frac{\begin{array}{c} r_0 : S[\alpha], r_1 : 0[], t : T^{s,0}[\alpha], y : \mathbf{1} \vdash p_{i+1}^{s,0}[\alpha] = P_{i+1}^{s,0} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \quad ; \cdot \vdash (r_0 : S[0[]], r_1 : 0[], t : T^{s,0}[0[]], y : \mathbf{1}) \diamond \\ (\text{exp}_0 \in L_1) \quad ; r_0 : S[0[]], r_1 : 0[], t : T^{s,0}[0[]], y : \mathbf{1} \vdash x \leftarrow p_{i+1}^{s,0}[0[]] \quad r_0 \ r_1 \ t \ y \end{array}}{\begin{array}{c} ; r_0 : S[0[]], r_1 : 0[], t : T^{0,0}[] = \&\{\text{exp}_0 : T^{s,0}[0[]], \dots\}, y : \mathbf{1} \vdash t.\text{exp}_0; \ x \leftarrow p_{i+1}^{s,0}[0[]] \quad r_0 \ r_1 \ t \ y \\ (\text{incr} \in L_0) \quad ; r_0 : 0[] = \&\{\text{incr} : S[0[]], \dots\}, r_1 : 0[], t : T^{0,0}[], y : \mathbf{1} \vdash P_i^{0,0} = r_0.\text{incr}; \ t.\text{exp}_0; \ x \leftarrow p_{i+1}^{s,0}[0[]] \quad r_0 \ r_1 \ t \ y :: (x : \mathbf{1}) \end{array}} \&L \text{ def}$$

with $L_0 = \{\text{incr}, \text{jump}\}$ and $L_1 = \{\text{ins}_0, \text{exp}_0, \text{exp}_1, \text{dup}, \text{ter}\}$.

By Definition 2.9, $i + 1$ is reachable, hence $p_{i+1}^{s,0}$ is defined in $\Sigma_{\mathcal{M}}$.

Proof for incrementing case for $P_i^{0,s}$, $P_i^{s,0}$, and $P_i^{s,s}$ is analogous.

Case 2 (Decrement). For $P_i^{s,0}$ with $i : \text{DECJ}(0, k)$:

$$\begin{array}{c} \vdash \\ \hline \frac{\frac{\frac{\frac{\frac{\frac{\cdot \vdash (1) \diamond}{\cdot; t : \mathbf{1} \vdash t \leftrightarrow x :: (x : \mathbf{1})} \text{id}}{\cdot; \& \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}} \vdash t.(\mathbf{i} + \mathbf{1}); t \leftrightarrow x :: (x : \mathbf{1})} \&L}{\cdot; \vdash (1) \diamond} \&L}{\cdot; t : \mathbf{1} \otimes \& \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}}, y : \mathbf{1} \vdash \text{send } t \ y; t.(\mathbf{i} + \mathbf{1}); t \leftrightarrow x :: (x : \mathbf{1})} \otimes L}{\cdot; \vdash (0[]) \diamond} \otimes L}{\cdot; r_1 : 0[], t : 0[] \otimes \mathbf{1} \otimes \& \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}}, y : \mathbf{1} \vdash \text{send } t \ r_1; \text{send } t \ y; t.(\mathbf{i} + \mathbf{1}); t \leftrightarrow x :: (x : \mathbf{1})} \otimes L}{\frac{\alpha; r_0 : \alpha, r_1 : 0[], t : I[\alpha, 0[]] = \alpha \otimes 0[] \otimes \mathbf{1} \otimes \& \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}}, y : \mathbf{1} \vdash \text{send } t \ r_0; \text{send } t \ r_1; \text{send } t \ y; t.(\mathbf{i} + \mathbf{1}); t \leftrightarrow x :: (x : \mathbf{1})}{\alpha; r_0 : \alpha, r_1 : 0[], t : T^{s,0}[\alpha] = \& \{\text{ins}_0 : I[\alpha, 0[]], \dots\}, y : \mathbf{1} \vdash t.\text{ins}_0; \text{send } t \ r_0; \text{send } t \ r_1; \text{send } t \ y; t.(\mathbf{i} + \mathbf{1}); t \leftrightarrow x :: (x : \mathbf{1})} \&L} \&L}{\alpha; r_0 : S[\alpha] = \& \{\dots, \text{decr} : \alpha\}, r_1 : 0[], t : T^{s,0}[\alpha], y : \mathbf{1} \vdash P_i^{s,0} = r_0.\text{decr}; t.\text{ins}_0; \text{send } t \ r_0; \text{send } t \ r_1; \text{send } t \ y; t.(\mathbf{i} + \mathbf{1}); t \leftrightarrow x :: (x : \mathbf{1})} \&L} \&L$$

Proof for decrementing case for $P_i^{0,s}$ and $P_i^{s,s}$ is analogous.

Case 3 (Jump). For $P_i^{0,0}$ with $i : \text{DECJ}(j, k)$:

$$\frac{\text{(jump} \in L) \quad \frac{r_0 : 0[], r_1 : 0[], t : T^{0,0}[], y : \mathbf{1} \vdash p_{i+1}^{0,0}[] = P_{i+1}^{0,0} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \quad \cdot \vdash (r_0 : 0[], r_1 : 0[], t : T^{0,0}[], y : \mathbf{1}) \diamond}{\cdot; r_0 : 0[], r_1 : 0[], t : t : T^{0,0}[], y : \mathbf{1} \vdash x \leftarrow p_k^{0,0}[] \quad r_0 \ r_1 \ t \ y :: (x : \mathbf{1})} \text{def}}{\cdot; r_j : 0[] = \oplus\{\dots, \text{jump} : 0[]\}, r_{1-j} : 0[], t : t : T^{0,0}[], y : \mathbf{1} \vdash P_i^{0,0} = r_j.\text{jump}; x \leftarrow p_k^{0,0}[] \quad r_0 \ r_1 \ t \ y :: (x : \mathbf{1})} \&L$$

with $L = \{\text{incr}, \text{jump}\}$.

By Definition 2.9, k is reachable, hence $p_{i+1}^{0,0}$ is defined in $\Sigma_{\mathcal{M}}$.

Proof for jump case for $P_i^{0,s}$ and $P_i^{s,0}$ is analogous.

Case 4 (Out of bounds). For $P_i^{0,0}$ with $i > n$:

$$\frac{\cdot; r_0 : 0[], r_1 : 0[], t : t : T^{0,0}[] \vdash Q_i^{0,0} :: (x : \mathbf{1})}{\cdot; r_0 : 0[], r_1 : 0[], t : t : T^{0,0}[], y : \mathbf{1} \vdash P_i^{0,0} = \text{wait } y; Q_i^{0,0} :: (x : \mathbf{1})} \text{1L}$$

where $Q_i^{0,0}$ is any typable infinite recursive process.

Proof for out of bound case for $P_i^{0,0}$, $P_i^{0,s}$, $P_i^{s,0}$, and $P_i^{s,s}$ is analogous.

B.3 State tree processes

Proof of Lemma 4.3 using Definition 4.7.

B.3.1 State tree encoding

72

1. Leaf (e.g. $0,0$);
2. Unary node (e.g. $m,0$ or $0,m$ where $m > 0$);
3. Binary node (e.g. m_0,m_1 where $m_0 > 0$ and $m_1 > 0$).

Case 1 (Leaf). For $\{0,0\}_z^{\mathcal{M}}$:

$$\frac{\cdot; \cdot \vdash N^{0,0} :: (z : T^{0,0}[]) }{\cdot \models_{\Sigma_{\mathcal{M}}} \{0,0\}_z^{\mathcal{M}} = \text{proc}(z, N^{0,0}) :: (z : \mathcal{T}^{\mathcal{M}}(0,0) = T^{0,0}[]) } \text{proc}$$

Case 2 (Unary node). For $\{1,0\}_z^{\mathcal{M}}$:

$$\frac{\cdot \models_{\Sigma_{\mathcal{M}}} \{0,0\}_a^{\mathcal{M}} :: (a : \mathcal{T}^{\mathcal{M}}(0,0)) \quad \frac{\cdot; a : T^{0,0}[] \vdash N^{1,0} :: (z : T^{s,0}[0[]])}{a : \mathcal{T}^{\mathcal{M}}(0,0) = T^{0,0}[] \models_{\Sigma_{\mathcal{M}}} \text{proc}(z, N^{1,0}) :: (z : T^{s,0}[0[]])} \text{proc}}{\cdot \models_{\Sigma_{\mathcal{M}}} \{1,0\}_z^{\mathcal{M}} = (\{0,0\}_a^{\mathcal{M}}, \text{proc}(z, N^{1,0})) :: (z : \mathcal{T}^{\mathcal{M}}(1,0) = T^{s,0}[0[]])} \text{comp}$$

Proofs for the cases $\{0, 1\}_z^{\mathcal{M}}$, $\{m_0, 0\}_z^{\mathcal{M}}$, and $\{0, m_1\}_z^{\mathcal{M}}$ are analogous, with $m_0 > 1$ and $m_1 > 1$.

Case 3 (Binary node). For $\{1, 1\}_z^{\mathcal{M}}$:

$$\frac{\cdot \models_{\Sigma_{\mathcal{M}}} \{0, 1\}_a^{\mathcal{M}} :: (a : \mathcal{T}^{\mathcal{M}}(\emptyset, 1)) \quad \frac{\cdot \models_{\Sigma_{\mathcal{M}}} \{1, 0\}_b^{\mathcal{M}} :: (b : \mathcal{T}^{\mathcal{M}}(1, 0)) \quad \frac{\cdot; a : T^{0,s}[\langle \emptyset \rangle], b : T^{s,0}[\langle \emptyset \rangle] \vdash N^{1,1} :: (z : T^{s,s}[\langle \emptyset \rangle, \langle \emptyset \rangle])}{a : T^{0,s}[\langle \emptyset \rangle], b : \mathcal{T}^{\mathcal{M}}(1, 0) = T^{s,0}[\langle \emptyset \rangle] \models_{\Sigma_{\mathcal{M}}} \text{proc}(z, N^{1,1}) :: (z : T^{s,s}[\langle \emptyset \rangle, \langle \emptyset \rangle])} \text{proc}}{a : \mathcal{T}^{\mathcal{M}}(\emptyset, 1) = T^{0,s}[\langle \emptyset \rangle] \models_{\Sigma_{\mathcal{M}}} (\{1, 0\}_b^{\mathcal{M}}, \text{proc}(z, N^{1,1})) :: (z : T^{s,s}[\langle \emptyset \rangle, \langle \emptyset \rangle])} \text{comp}} \text{comp}$$

Proofs for the cases $\{1, m_1\}_z^{\mathcal{M}}$, $\{m_0, 1\}_z^{\mathcal{M}}$, and $\{m_0, m_1\}_z^{\mathcal{M}}$ are analogous, with $m_0 > 1$ and $m_1 > 1$.

B.3.2 State tree process definition

$\cong$

B.3.3 Constituent state tree processes

1. Leaf (offering expansion, duplication and termination);
2. Unary node (offering decrement+instantiation on one register, expansion, duplication and termination);
3. Binary node (offering decrement+instantiation on both registers, expansion, and termination).

Case 1 (Leaf). For $N^{0,0}$:

$$\frac{(\forall l \in L) \quad \cdot; \vdash N_l^{0,0} :: (t : A_l)}{\cdot; \vdash N^{0,0} = \text{case } t \text{ (} \mathbf{1} \Rightarrow N_l^{0,0})_{l \in L} :: (t : T^{0,0}[] = \oplus \{\mathbf{1} : A_l\})} \&\mathbf{R}$$

with $L = \{\text{exp}_0, \text{exp}_1, \text{dup}, \text{ter}\}$.

1. For $N_{\text{exp}_0}^{0,0}$:

$$\frac{\cdot \vdash t^{0,0}[] = N^{0,0} :: (z : T^{0,0}[]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash \cdot \quad \frac{c : T^{0,0}[] \vdash t^{1,0}[] = N^{1,0} :: (z : T^{s,0}[0[]]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash c : T^{0,0}[]}{\cdot; c : T^{0,0}[] \vdash z \leftarrow t^{1,0}[] \quad c :: (z : A_{\text{exp}_0} = T^{s,0}[0[]])} \text{def}}{\cdot; \cdot \vdash N_{\text{exp}_0}^{0,0} = c \leftarrow t^{0,0}[]; z \leftarrow t^{1,0}[] \quad c :: (z : A_{\text{exp}_0})} \text{def}$$

Proof for $N_{\text{exp}_1}^{0,0}$ is structurally analogous to $N_{\text{exp}_0}^{0,0}$.

2. For $N_{\text{dup}}^{0,0}$:

$$\frac{\cdot \vdash t^{0,0}[] = N^{0,0} :: (z : T^{0,0}[]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash \cdot \quad \frac{\cdot \vdash t^{0,0}[] = N^{0,0} :: (z : T^{0,0}[]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash \cdot}{\cdot; \cdot \vdash z \leftarrow t^{0,0}[] :: (z : T^{0,0}[]) } \text{def}}{\cdot \vdash t^{0,0}[] = N^{0,0} :: (z : T^{0,0}[]) \in \Sigma_{\mathcal{M}} \quad \cdot; \cdot \vdash \cdot \quad \frac{\cdot; \cdot \vdash (T^{0,0}[]) \diamond \quad \cdot; \cdot \vdash z \leftarrow t^{0,0}[] :: (z : T^{0,0}[]) }{\cdot; c : T^{0,0}[] \vdash \text{send } z \ c; z \leftarrow t^{0,0}[] :: (z : A_{\text{dup}} = T^{0,0}[] \otimes T^{0,0}[]) } \otimes L}}{\cdot; \cdot \vdash N_{\text{dup}}^{0,0} = c \leftarrow t^{0,0}[]; \text{send } z \ c; z \leftarrow t^{0,0}[] :: (z : A_{\text{dup}})} \text{def}$$

3. For $N_{\text{ter}}^{0,0}$:

$$\frac{}{\cdot; \cdot \vdash N_{\text{ter}}^{0,0} = \text{close } z :: (z : A_{\text{ter}} = \mathbf{1})} \text{1R}$$

Case 2 (Linear). For $N^{s,0}$:

$$\frac{(\forall l \in L) \quad \alpha; \cdot \vdash N_l^{s,0} :: (t : A_l)}{\alpha; a : T^{s,0}[\alpha] \vdash N^{s,0} = \text{case } t \ (1 \Rightarrow N_l^{s,0})_{l \in L} :: (t : T^{s,0}[S[\alpha]] = \oplus \{ \mathbf{1} : A_l \})} \&\text{R}$$

with $L = \{\text{ins}_0, \text{exp}_0, \text{exp}_1, \text{dup}, \text{ter}\}$.

1. For $N_{\text{ins}_0}^{s,0}$:

$$\begin{array}{c}
\frac{r_0 : S[\alpha], r_1 : 0[], t : T^{s,0}[\alpha], y : \mathbf{1} \vdash p_i^{s,0}[\alpha] = P_i^{s,0} :: (x : \mathbf{1}) \in \Sigma_{\mathcal{M}} \quad ; \alpha \vdash r_0 : S[\alpha], r_1 : 0[], t : T^{s,0}[\alpha], y : \mathbf{1}}{(i \in L_{\mathcal{M}})} \text{def} \\
\frac{\alpha; r_0 : S[\alpha], r_1 : 0[], t : T^{s,0}[\alpha], y : \mathbf{1} \vdash z \leftarrow p_i^{s,0}[\alpha] \ r_0 \ r_1 \ a \ y :: (z : \mathbf{1})}{\alpha; r_0 : S[\alpha], r_1 : 0[], t : T^{s,0}[\alpha], y : \mathbf{1} \vdash \text{case } z \ (\mathbf{i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] \ r_0 \ r_1 \ a \ y)_{i \in L_{\mathcal{M}}} :: (z : \oplus \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}})} \&\text{R} \\
\frac{\alpha; r_0 : S[\alpha], r_1 : 0[], t : T^{s,0}[\alpha] \vdash y \leftarrow \text{recv } z; \text{case } z \ (\mathbf{i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] \ r_0 \ r_1 \ a \ y)_{i \in L_{\mathcal{M}}} :: (z : \mathbf{1} \multimap \oplus \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}})}{\alpha; r_0 : S[\alpha], t : T^{s,0}[\alpha] \vdash r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \text{case } z \ (\mathbf{i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] \ r_0 \ r_1 \ a \ y)_{i \in L_{\mathcal{M}}} :: (z : 0[] \multimap \oplus \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}})} \multimap\text{R} \\
\frac{\alpha; r_0 : S[\alpha], t : T^{s,0}[\alpha] \vdash r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \text{case } z \ (\mathbf{i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] \ r_0 \ r_1 \ a \ y)_{i \in L_{\mathcal{M}}} :: (z : 0[] \multimap \oplus \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}})}{\alpha; t : T^{s,0}[S[\alpha]] \vdash N_{\text{ins}_0}^{s,0} = r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; \text{case } z \ (\mathbf{i} \Rightarrow z \leftarrow p_i^{s,0}[\alpha] \ r_0 \ r_1 \ a \ y)_{i \in L_{\mathcal{M}}} :: (z : A_{\text{ins}_0} = I[\alpha, 0[]])} \multimap\text{R}
\end{array}$$

2. For $N_{\text{exp}_0}^{s,0}$:

$$\begin{array}{c}
\frac{a : T^{s,0}[\alpha] \vdash t^{s,0}[\alpha] = N^{s,0} :: (z : T^{s,0}[S[\alpha]]) \quad ; \alpha \vdash (a : T^{s,0}[S[\alpha]]) \diamond}{(\text{exp}_0 \in L) \quad \alpha; a : T^{s,0}[S[\alpha]] \vdash z \leftarrow t^{s,0}[S[\alpha]] \ a :: (z : A_{\text{exp}_0} = T^{s,0}[S[S[\alpha]])} \text{def}} \\
\frac{\alpha; a : T^{s,0}[\alpha] = \oplus \{..., \text{exp}_0 : T^{s,0}[S[\alpha]], ...\} \vdash N_{\text{exp}_0}^{s,0} = a.\text{exp}_0; \ z \leftarrow t^{s,0}[S[\alpha]] \ a :: (z : A_{\text{exp}_0})}{\&\text{L}}
\end{array}$$

with $L = \{\text{ins}_0, \text{exp}_0, \text{exp}_1, \text{dup}, \text{ter}\}$

3. For $N_{\text{exp}_1}^{s,0}$:

$$\begin{array}{c}
\frac{a : T^{s,s}[\alpha, 0[]], b : T^{s,0}[S[\alpha]] \vdash \quad t^{s,1}[\alpha, \beta] = N^{s,1} :: (z : T^{s,s}[S[\alpha], 0[]]) \in \Sigma_{\mathcal{M}} \quad ; \alpha \vdash (a : T^{s,s}[\alpha, 0[]], b : T^{s,0}[S[\alpha]]) \diamond}{(\text{exp}_0 \in L) \quad \alpha; a : T^{s,s}[\alpha, 0[]], b : T^{s,0}[S[\alpha]] \vdash z \leftarrow t^{s,1}[\alpha] \ a \ b :: (z : A_{\text{exp}_1} = T^{s,s}[S[\alpha], 0[]])} \text{def}} \\
\frac{(\text{exp}_1 \in L) \quad \alpha; a : T^{s,s}[\alpha, 0[]], b : T^{s,0}[\alpha] = \oplus \{..., \text{exp}_0 : b : T^{s,0}[S[\alpha]], ...\} \vdash b.\text{exp}_0; \ z \leftarrow t^{s,1}[\alpha] \ a \ b :: (z : A_{\text{exp}_1})}{\&\text{L}} \\
\frac{\alpha; a : T^{s,0}[\alpha] = \oplus \{..., \text{exp}_1 : T^{s,s}[\alpha, 0[]], ...\}, b : T^{s,0}[\alpha] \vdash a.\text{exp}_1; \ b.\text{exp}_0; \ z \leftarrow t^{s,1}[\alpha] \ a \ b :: (z : A_{\text{exp}_1})}{\&\text{L}} \\
\frac{(\text{dup} \in L) \quad \alpha; a : T^{s,0}[\alpha] \otimes T^{s,0}[\alpha] \vdash b \leftarrow \text{recv } a; a.\text{exp}_1; \ b.\text{exp}_0; \ z \leftarrow t^{s,1}[\alpha] \ a \ b :: (z : A_{\text{exp}_1})}{\otimes\text{L}} \\
\frac{\alpha; a : T^{s,0}[\alpha] = \oplus \{..., \text{dup} : T^{s,0}[\alpha] \otimes T^{s,0}[\alpha], ...\} \vdash N_{\text{exp}_1}^{s,0} = a.\text{dup}; \ b \leftarrow \text{recv } a; a.\text{exp}_1; \ b.\text{exp}_0; \ z \leftarrow t^{s,1}[\alpha] \ a \ a' :: (z : A_{\text{exp}_1})}{\&\text{L}}
\end{array}$$

with $L = \{\text{ins}_0, \text{exp}_0, \text{exp}_1, \text{dup}, \text{ter}\}$

4. For $N_{\text{dup}}^{s,0}$:

$$\frac{(\text{dup} \in L) \quad \frac{\Pi}{\alpha; a : T^{s,0}[\alpha] = \oplus\{\dots, \text{dup} : T^{s,0}[\alpha], \dots\} \vdash a' \leftarrow \text{recv } a; c \leftarrow t^{s,0}[\alpha] \ a; \text{send } z \ c; z \leftarrow t^{s,0}[\alpha] \ a' :: (z : A_{\text{dup}})} \otimes\text{L}}{\alpha; a : T^{s,0}[\alpha] \vdash N_{\text{dup}}^{s,0} = a.\text{dup}; \ a' \leftarrow \text{recv } a; c \leftarrow t^{s,0}[\alpha] \ a; \text{send } z \ c; z \leftarrow t^{s,0}[\alpha] \ a' :: (z : A_{\text{dup}})} \&\text{L}}$$

Where Π :

$$\frac{a : T^{s,0}[\alpha] \vdash t^{s,0}[\alpha] = N^{s,0} :: (z : T^{s,0}[S[\alpha]]) \in \Sigma_{\mathcal{M}} \quad ; \alpha \vdash (a : T^{s,0}[\alpha]) \diamond \quad \Pi'}{\alpha; a : T^{s,0}[\alpha], a' : T^{s,0}[\alpha] \vdash c \leftarrow t^{s,0}[\alpha] \ a; \text{send } z \ c; z \leftarrow t^{s,0}[\alpha] \ a' :: (z : A_{\text{dup}})} \text{def}$$

and Π' :

$$\frac{\frac{a : T^{s,0}[\alpha] \vdash t^{s,0}[\alpha] = N^{s,0} :: (z : T^{s,0}[S[\alpha]]) \in \Sigma_{\mathcal{M}} \quad ; \alpha \vdash (a' : T^{s,0}[\alpha]) \diamond}{\alpha; a' : T^{s,0}[\alpha] \vdash z \leftarrow t^{s,0}[\alpha] \ a' :: (z : A_{\text{dup}})} \text{def}}{\alpha; a' : T^{s,0}[\alpha], c : T^{s,0}[S[\alpha]] \vdash \text{send } z \ c; z \leftarrow t^{s,0}[\alpha] \ a' :: (z : A_{\text{dup}} = T^{s,0}[S[\alpha]] \otimes T^{s,0}[S[\alpha]])} \otimes\text{R}}$$

5. For $N_{\text{ter}}^{s,0}$:

$$\frac{(\text{ter} \in L) \quad \frac{\frac{\overline{; \cdot \vdash \text{close } z :: (z : A_{\text{ter}} = 1)} \text{1R}}{; a : 1 \vdash \text{wait } a; \text{close } z :: (z : A_{\text{ter}})} \text{1L}}{\alpha; a : T^{s,0}[\alpha] = \oplus\{\dots, \text{ter} : 1\} \vdash N_{\text{ter}}^{s,0} = a.\text{ter}; \text{wait } a; \text{close } z :: (z : A_{\text{ter}})} \&\text{L}}$$

Proofs for $N^{1,0}$, $N^{0,1}$, and $N^{0,s}$ are analogous.

Case 3 (Branch). For $N^{s,s}$:

$$\frac{(\forall l \in L) \quad \alpha, \beta; a : T^{s,s}[\alpha, S[\beta]], b : T^{s,s}[S[\alpha], \beta] \vdash N_l^{s,s} :: (t : A_l)}{\alpha, \beta; a : T^{s,s}[\alpha, S[\beta]], b : T^{s,s}[S[\alpha], \beta] \vdash N^{s,s} = \text{case } t \text{ (} \mathbf{1} \Rightarrow N_l^{s,s} \text{)}_{l \in L} :: (t : T^{s,s}[S[\alpha], S[\beta]] = \oplus \{\mathbf{1} : A_l\}_{l \in L})} \&\text{R}$$

with $L = \{\text{ins}_0, \text{ins}_1, \text{exp}_0, \text{exp}_1, \text{ter}\}$.

1. For $N_{\text{ins}_0}^{s,s}$:

$$\frac{\frac{\frac{\frac{\Pi}{\alpha, \beta; r_0 : S[\alpha], t : T^{s,s}[\alpha, \beta] \vdash r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; N' :: (z : \multimap \mathbf{1} \multimap A')} \multimap \text{R}}{\alpha, \beta; t : T^{s,s}[\alpha, \beta] \vdash r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; N' :: (z : A_{\text{ins}_0})} \multimap \text{R}}{\alpha, \beta; t : T^{s,s}[\alpha, \beta], b : \mathbf{1} \vdash \text{wait } b; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; N' :: (z : A_{\text{ins}_0})} \mathbf{1L}}{\alpha, \beta; t : T^{s,s}[\alpha, \beta], b : T^{s,s}[S[\alpha], \beta] = \oplus \{\dots, \text{ter} : \mathbf{1}\} \vdash N_{\text{ins}_0}^{s,s} = b.\text{ter}; \text{wait } b; r_0 \leftarrow \text{recv } z; r_1 \leftarrow \text{recv } z; y \leftarrow \text{recv } z; N' :: (z : A_{\text{ins}_0})} \&\text{L}$$

77

with $L = \{\text{ins}_0, \text{ins}_1, \text{exp}_0, \text{exp}_1, \text{ter}\}$.

Where Π :

$$\frac{\frac{(\forall i \in L_{\mathcal{M}}) \quad \frac{r_0 : S[\alpha], r_1 : S[\beta], t : T^{s,s}[\alpha, \beta], y : \mathbf{1} \vdash p_{i \in L_{\mathcal{M}}}^{s,s}[\alpha, \beta] = P_i^{s,s} :: (z : \mathbf{1}) \in \Sigma_{\mathcal{M}} \quad ; \alpha, \beta \vdash (r_0 : S[\alpha], r_1 : S[\beta], a : T^{s,s}[\alpha, \beta], y : \mathbf{1}) \diamond}{\alpha, \beta; r_0 : S[\alpha], r_1 : S[\beta], t : T^{s,s}[\alpha, \beta], y : \mathbf{1} \vdash z \leftarrow p_i^{s,s}[\alpha, \beta] \text{ } r_0 \text{ } r_1 \text{ } a \text{ } y :: (z : \mathbf{1})} \text{def}}{\alpha, \beta; r_0 : S[\alpha], r_1 : S[\beta], t : T^{s,s}[\alpha, \beta], y : \mathbf{1} \vdash N' = \text{case } z \text{ (} \mathbf{i} \Rightarrow z \leftarrow p_i^{s,s}[\alpha, \beta] \text{ } r_0 \text{ } r_1 \text{ } a \text{ } y \text{)}_{i \in L_{\mathcal{M}}} :: (z : A' = \oplus \{\mathbf{i} : \mathbf{1}\}_{i \in L_{\mathcal{M}}})} \&\text{R}}{\alpha, \beta; r_0 : S[\alpha], r_1 : S[\beta], t : T^{s,s}[\alpha, \beta] \vdash y \leftarrow \text{recv } z; N' :: (z : \mathbf{1} \multimap A')} \multimap \text{R}$$

Proof for $N_{\text{ins}_1}^{s,s}$ is analogous to the proof above.

2. For $N_{\text{exp}_0}^{s,s}$:

$$\frac{(\mathbf{exp}_0 \in L_a) \quad \Pi}{\alpha, \beta; a : T^{s,s}[\alpha, S[\beta]] = \oplus\{\dots, \mathbf{exp}_0 : T^{s,s}[S[\alpha], S[\beta]], \dots\}, b : T^{s,s}[S[\alpha], \beta] \vdash N_{\mathbf{exp}_0}^{s,s} = a.\mathbf{exp}_0; b.\mathbf{exp}_0; z \leftarrow t^{s,s}[S[\alpha], \beta] \ a \ b :: (z : A_{\mathbf{exp}_0})} \&R$$

Where Π :

$$\frac{(\mathbf{exp}_0 \in L_b) \quad \frac{\alpha, \beta; a : T^{s,s}[\alpha, S[\beta]], b : T^{s,s}[S[\alpha], \beta] \vdash t^{s,s}[\alpha, \beta] = N^{s,s} :: (z : T^{s,s}[S[\alpha], S[\beta]]) \in \Sigma_{\mathcal{M}} \quad ; \alpha, \beta \vdash (a : T^{s,s}[S[\alpha], S[\beta]], b : T^{s,s}[S[S[\alpha]], \beta]) \diamond}{\alpha, \beta; a : T^{s,s}[S[\alpha], S[\beta]], b : T^{s,s}[S[S[\alpha]], \beta] \vdash z \leftarrow t^{s,s}[S[\alpha], \beta] \ a \ b :: (z : A_{\mathbf{exp}_0} = T^{s,s}[S[S[\alpha]], S[\beta]])} \text{def}}{\alpha, \beta; a : T^{s,s}[S[\alpha], S[\beta]], b : T^{s,s}[S[\alpha], \beta] = \oplus\{\dots, \mathbf{exp}_0 : T^{s,s}[S[S[\alpha]], \beta], \dots\} \vdash b.\mathbf{exp}_0; z \leftarrow t^{s,s}[S[\alpha], \beta] \ a \ b :: (z : A_{\mathbf{exp}_0})} \&R$$

Proof for $N_{\mathbf{exp}_1}^{s,s}$ is analogous to the proof above.

3. For $N_{\mathbf{ter}}^{s,s}$:

$$\frac{(\mathbf{ter} \in L_a) \quad \frac{(\mathbf{ter} \in L_b) \quad \frac{\frac{; \cdot \vdash \mathbf{close} \ z :: (z : A_{\mathbf{ter}} = \mathbf{1})}{; b : \mathbf{1} \vdash \mathbf{wait} \ b; \mathbf{close} \ z :: (z : A_{\mathbf{ter}})} \mathbf{1L}}{; a : \mathbf{1}, b : \mathbf{1} \vdash \mathbf{wait} \ a; \mathbf{wait} \ b; \mathbf{close} \ z :: (z : A_{\mathbf{ter}})} \mathbf{1L}}{\beta; a : \mathbf{1}, b : T^{s,s}[S[\alpha], \beta] = \oplus\{\dots, \mathbf{ter} : \mathbf{1}\} \vdash b.\mathbf{ter}; \mathbf{wait} \ a; \mathbf{wait} \ b; \mathbf{close} \ z :: (z : A_{\mathbf{ter}})} \oplus R}{\alpha, \beta; a : T^{s,s}[\alpha, S[\beta]] = \oplus\{\dots, \mathbf{ter} : \mathbf{1}\}, b : T^{s,s}[S[\alpha], \beta] \vdash N_{\mathbf{ter}}^{s,s} = a.\mathbf{ter}; b.\mathbf{ter}; \mathbf{wait} \ a; \mathbf{wait} \ b; \mathbf{close} \ z :: (z : A_{\mathbf{ter}})} \oplus R$$

Proofs for $N^{1,1}$, $N^{1,s}$, and $N^{s,1}$ are analogous.

B.4 Encoding of the configuration

Proof of Theorem 4.1 using Definition 4.8. Following Appendix B.1, let Π_0 and Π_1 denote the following:

- $\Pi_0 = \cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m_0 \rrbracket_{r_0} :: (r_0 : \langle m_0 \rangle)$
- $\Pi_1 = \cdot \models_{\Sigma_{\mathcal{M}}} \llbracket m_1 \rrbracket_{r_1} :: (r_1 : \langle m_1 \rangle)$

For $\llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}}$ (Definition 4.8):

$$\begin{array}{c}
\frac{\cdot \models_{\Sigma_{\mathcal{M}}} \{m_0, m_1\}_t^{\mathcal{M}} :: (t : \mathcal{T}^{\mathcal{M}} \langle m_0, m_1 \rangle) \quad r_0 : \langle m_0 \rangle, r_1 : \langle m_1 \rangle, t : \mathcal{T}^{\mathcal{M}} \langle m_0, m_1 \rangle, y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}} :: (x : \mathbf{1})}{\frac{\Pi_1 \quad r_0 : \langle m_0 \rangle, r_1 : \langle m_1 \rangle, y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} (\{m_0, m_1\}_t^{\mathcal{M}}, \llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}) :: (x : \mathbf{1})}{\text{comp}}} \text{comp} \\
\frac{\Pi_0 \quad r_0 : \langle m_0 \rangle, y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} (\llbracket m_1 \rrbracket_{r_1}, (\{m_0, m_1\}_t^{\mathcal{M}}, \llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}})) :: (x : \mathbf{1})}{\text{comp}} \text{comp} \\
y : \mathbf{1} \models_{\Sigma_{\mathcal{M}}} \llbracket (i, m_0, m_1) \rrbracket^{\mathcal{M}} = (\llbracket m_0 \rrbracket_{r_0}, (\llbracket m_1 \rrbracket_{r_1}, (\{m_0, m_1\}_t^{\mathcal{M}}, \llbracket i | m_0, m_1 \rrbracket_{r_0, r_1, t}^{\mathcal{M}}))) :: (x : \mathbf{1})
\end{array}$$